

Sistemi Operativi

Davide Cuna
Leonardo Ciscato Pajello
Tommaso Trentin

March 2025

Contents

1	Introduzione:	7
1.1	Cos'è un Sistema Operativo	7
1.1.1	Definizione:	7
1.1.2	Punti di vista sul Sistema Operativo	7
1.1.3	Obiettivi primari di un OS	7
1.2	Organizzazione di un Sistema di Calcolo	7
1.2.1	Schema di un Sistema di Calcolo	7
1.2.2	Dispositivi di I/O	8
1.2.3	Memoria	8
1.2.4	Accesso Diretto alla Memoria (DMA)	8
1.3	Sistemi Mono/Multi Programmati	9
1.3.1	Sistema Monoprogrammato	9
1.3.2	Sistema Multiprogrammato	9
1.3.3	Time Sharing (o multitasking)	9
1.3.4	Nozioni di Processo	9
1.4	Principali Funzionalità di un Sistema Operativo	10
1.4.1	Modalità di Funzionamento	10
1.4.2	Tipi di Funzionalità offerte dall'OS	10
2	Struttura e Servizi degli OS	11
2.1	Servizi offerti dall'OS	11
2.1.1	Servizi di Base	11
2.1.2	Interfaccia con l'utente	12
2.2	System Call	12
2.2.1	Passaggio di parametri	12
2.2.2	Tipi di System Call	12
2.2.3	Programmi di sistema	13
2.3	Struttura di un OS	13
2.3.1	Struttura Semplice - Monolitica	13
2.3.2	Struttura Stratificata	14
2.3.3	Struttura a MicroKernel	14
2.3.4	Struttura Modulare	15
2.3.5	Caso Linux	16
2.3.6	Struttura Ibrida?	16
2.4	Virtual Machines	16
3	Processi	17
3.1	Nozioni di Processo	17
3.1.1	Ciclo di vita di un Processo	18
3.1.2	Transizioni tra gli Stati	18
3.1.3	PCB - Process Control Block	18
3.1.4	Process Table	19
3.1.5	Context Switch	19
3.1.6	Code di Processi e Risorse	19
3.1.7	Scheduler	20
3.1.8	Swapping e Medium-term Scheduler	20
3.1.9	Transizione tra gli Stati (caso UNIX)	21
3.2	Operazioni sui Processi	21
3.2.1	Creazione di un Processo	22
3.2.2	Creazione di Processi	22
3.2.3	Terminazione di Processi	22
3.2.4	Gerarchia di Processi (Android)	22
3.2.5	Meccanismo di Terminazione (UNIX)	22
3.3	Comunicazione tra Processi	23
3.3.1	Processi Indipendenti o Cooperanti	23
3.3.2	Comunicazione tra Processi Concorrenti	23
3.3.3	Shared Memory	23
3.3.4	Message Passing	23
3.3.5	Caratteristiche della Comunicazione	24
4	Thread	25
4.1	Dai Processi ai Threads	25

4.1.1	Multithreading	25
4.1.2	Single e Multi Thread	26
4.1.3	Thread di un Processo	26
4.1.4	Vantaggi dei Thread	26
4.1.5	Svantaggi dei Thread	26
4.1.6	Scheduling e Dispatching: Thread o Processo?	26
4.2	Modelli di supporto al multithreading	27
4.2.1	Stati e Operazioni dei Thread	27
4.2.2	Esempio di Programmazione con Thread	27
4.2.3	Sincronizzazione tra Thread	27
4.2.4	User-level Thread e Kernel-level Thread	28
4.2.5	Vantaggi e Svantaggi di ULT	28
4.2.6	Svantaggi KLT	29
4.2.7	Approcci combinati?	29
4.2.8	Problematiche nella gestione dei Thread	29
5	Scheduling	29
5.1	Scheduling dei Processi	29
5.1.1	Concetti Generali	29
5.1.2	Compiti del Dispatcher	30
5.1.3	Parametri Generali	30
5.1.4	Obiettivi	30
5.1.5	Criteri per CPU Scheduling	30
5.1.6	CPU-burst e I/O-burst	31
5.1.7	Scheduling Preemptive e Non-preemptive	31
5.2	Algoritmi di Scheduling Processi	31
5.2.1	FCFS - First Come First Served	31
5.2.2	SJF - Shortest Job First	32
5.2.3	SRTF - Shortest Remaining Time First	32
5.2.4	Caratteristiche SJF / SRTF	32
5.2.5	Stima durata CPU burst	33
5.2.6	HRRF - Highest Response Ratio First	33
5.2.7	Scheduling con Priorità	33
5.2.8	Round Robin (circolare)	34
5.2.9	Durata quanto di tempo	34
5.2.10	Scheduling con Code Multiple	34
5.2.11	Scheduling con Code Multiple -Retroazione	34
5.2.12	Comparazione tra algoritmi	35
5.2.13	Fair Share Scheduling	35
5.3	Scheduling dei Thread	35
5.3.1	Kernel-Thread e User-Thread	35
5.4	Scheduling per Sistemi Multiprocessore	36
5.4.1	Scheduling in Sistemi Multiprocessore	36
5.4.2	Bilanciamento del Carico	36
5.4.3	Sistemi Eterogenei	37
5.5	Scheduling Real-time	37
5.5.1	Sistemi Real-Time	37
5.6	Scheduling in UNIX e Linux	37
5.6.1	Scheduling di UNIX Tradizionale	37
5.6.2	Scheduling in O(1) di Linux	37
6	Sincronizzazione	38
6.1	Il problema della Sezione Critica	38
6.1.1	Esecuzione Concorrente	38
6.1.2	Concetti Essenziali	38
6.1.3	Possibile soluzione al problema della SC	38
6.1.4	Soluzione di Peterson	39
6.1.5	Soluzioni basate sugli Interrupt	39
6.1.6	Soluzione con Supporto Hardware	39
6.1.7	Memory Barriers	39
6.1.8	Test and Set: istruzione atomica	39
6.1.9	Swap: istruzione atomica	40
6.1.10	CompareAndSwap: istruzione atomica	40

6.2	Semafori	41
6.2.1	Semafori di Mutex e di Sincronizzazione	41
6.2.2	Spinlock e Busy Waiting	41
6.2.3	Implementazione dei Semafori senza Busy Waiting	41
6.2.4	Atomicità di wait e signal	42
6.3	Tipici Problemi di Sincronizzazione	42
6.3.1	Produttori e Consumatori con Buffer Limitato	42
6.3.2	Soluzione con Priorità ai Lettori	42
6.3.3	Soluzione con Priorità agli Scrittori	43
6.3.4	Problema dei Cinque Filosofi	43
6.3.5	Soluzione Cinque Filosofi con Semafori	44
6.3.6	Osservazioni Finali sui Semafori	44
6.4	Monitor	45
6.4.1	Sincronizzazione nei Monitor: Condition Variables	45
6.4.2	Risveglio del Processo	46
6.4.3	Monitor che invocano altri monitor	46
6.4.4	Esempi di uso dei Monitor - 5 Filosofi	46
6.4.5	Esempi di uso dei Monitor - Produttori_Consumatori	47
6.4.6	Esempi di uso dei Monitor - Lettori_Scrittori	47
6.5	IPC di UNIXLinux	47
7	Deadlock	48
7.1	Il problema del Deadlock	48
7.1.1	Esempio ponte	48
7.1.2	Varie tipologie di risorse	48
7.1.3	Protocollo d'uso delle risorse	48
7.1.4	Il Deadlock	49
7.1.5	Caratterizzazione delle Situazioni Critiche	49
7.1.6	Grafo di Allocazione delle Risorse	49
7.1.7	Proprietà del Grafo	50
7.1.8	Immagini di esempio grafo ciclico con e senza deadlock	50
7.2	Gestione del Deadlock	51
7.2.1	The Ostrich Algorithm	51
7.2.2	Prevenzione del Deadlock	51
7.2.3	Evitare il Deadlock	52
7.2.4	L'idea di Base	52
7.2.5	Stato Safe	52
7.2.6	Stati Safe, Unsafe e Deadlock	53
7.2.7	Algoritmi per la Deadlock-Avoidance	53
7.2.8	Alg Grafo assegnazione risorse	53
7.2.9	Algoritmo del banchiere	54
7.2.10	Esempio Alg Banchiere	55
7.2.11	Rilevare il Deadlock	55
7.2.12	Algoritmo di Rilevamento	56
7.2.13	Esempio Alg Rilevamento	56
7.2.14	Quando attivare l'algoritmo di rilevamento?	57
7.2.15	Tecniche di ripristino	57
7.2.16	Approccio Combinato per Gestione Deadlock	58
8	Memoria Centrale	58
8.1	Generalità e Gestione	58
8.1.1	Gestione della memoria	58
8.1.2	Indirizzi di memoria e binding	58
8.1.3	Caricamento e collegamento	59
8.1.4	Swapping	59
8.1.5	Considerazioni	60
8.2	Allocazione Contigua	60
8.2.1	Concetti generali	60
8.2.2	Partizioni fisse	61
8.2.3	Partizioni variabili	62
8.2.4	Buddy System	63
8.3	Paginazione	64
8.3.1	Generalità	64

8.3.2	Architettura	64
8.3.3	Gestione dei processi	64
8.3.4	Page Table	65
8.4	Segmentazione	67
8.4.1	Generalità	67
8.4.2	Architettura per Segmentazione	67
8.4.3	Considerazioni	68
8.5	Segmentazione Paginata	68
8.5.1	Generalità	68
8.5.2	Architettura	68
9	Memoria Vituale	69
9.1	Motivazioni	69
9.1.1	Memoria Virtuale	69
9.2	Paginazione su Richiesta	69
9.2.1	Schema di Funzionamento	69
9.2.2	Tabella delle Pagine	69
9.2.3	Esempio	70
9.2.4	Gestione del Page Fault	70
9.2.5	Supporto alla Paginazione su Richiesta	71
9.3	Rimpiazzamento Pagine	71
9.3.1	Prestazioni della Paginazione su Richiesta	71
9.3.2	Esempio	71
9.3.3	Problematiche	71
9.3.4	Frequenza di page Fault	72
9.4	Algoritmi di Rimpiazzamento Pagine	72
9.4.1	FIFO	72
9.4.2	Anomalia di Belady	73
9.4.3	Algoritmo ottimale	73
9.4.4	LRU: Least Recently Used	73
9.4.5	Implementazione LRU	73
9.4.6	Approssimazioni LRU	74
9.4.7	Tecniche basate su conteggio	74
9.4.8	Algoritmo di seconda Chance	74
9.5	Allocazione del Frame	75
9.5.1	Schemi di allocazione	75
9.5.2	Contesto di rimpiazzamento	75
9.5.3	Thrashing	75
9.5.4	Località dei riferimenti	75
9.5.5	Modello del Working set	76
9.5.6	Page fault Frequency	76
9.5.7	Compressione della Memoria	76
10	Disk Scheduling	77
10.1	Struttura di un disco	77
10.2	Tempo di accesso a disco	77
10.3	Disk Scheduling	78
10.4	Algoritmi di Disk Scheduling	78
10.4.1	FCFS	78
10.4.2	SSTF	78
10.4.3	SCAN	79
10.4.4	SCAN variante 1	79
10.4.5	SCAN variante 2	80
10.4.6	SCAN variante 3	80
10.4.7	LIFO	80
11	Il File System (FS)	80
11.1	File	81
11.1.1	Struttura dei file	81
11.1.2	Attributi dei file	81
11.2	Operazioni sui file	82
11.2.1	File aperti	82
11.2.2	Lock dei file aperti	83

11.2.3	Tipi di file	83
11.2.4	Supporto alla struttura dei file	83
11.2.5	Struttura interna dei file	84
11.2.6	Modalità di accesso	84
11.3	Directory	84
11.3.1	Info e operazioni sulle Directory	84
11.3.2	Directory a un livello	85
11.3.3	Directory a due livelli	85
11.3.4	Directory con struttura ad Albero	85
11.3.5	Directory a grafo aciclico	86
11.3.6	Directory a grafo generico	86
11.4	Montaggio di un FS	87
11.4.1	Procedura di montaggio	87
11.4.2	Considerazioni	87
11.5	File Sharing	87
11.5.1	File Sharing multiutente	88
11.6	File System remoti	88
11.7	Malfunzionamenti	88
11.8	Semantica della coerenza	89
11.8.1	Protezione nel File System	89
11.9	Caso UNIX	89
11.9.1	Review del caso di UNIX	89
11.9.2	UNIX	89
11.9.3	Unix: Operazioni su file e directory	90
11.9.4	Unix: links	90
11.9.5	Protezione in UNIX	90
12	Realizzazione del File System (FS)	91
12.1	Struttura del File System	91
12.1.1	I/O control	91
12.1.2	Basic File System	92
12.1.3	File-Organization module	92
12.1.4	Logical File System	92
12.1.5	File Control Block	92
12.1.6	Tipi di File System	92
12.1.7	Strutture dati del File System	92
12.2	Realizzazione Directory	93
12.3	Metodi di allocazione dei files	93
12.3.1	Allocazione contigua	93
12.3.2	Allocazione concatenata	93
12.3.3	File-Allocation Table (FAT)	94
12.3.4	Allocazione indicizzata	94
12.4	Gestione dello spazio libero	95
12.4.1	Ripristino	96
12.4.2	File System con Log	96
12.5	Network File System (NFS)	96
12.5.1	Vantaggi NFS	96

1 Introduzione:

1.1 Cos'è un Sistema Operativo

1.1.1 Definizione:

Per Sistema Operativo [SO][OS] si intende l'insieme di programmi che operano sull'hardware di un **sistema di calcolo**. Gli obiettivi principali di un OS sono:

- semplificare all'utente l'uso e lo sviluppo dei programmi
- rendere efficiente l'utilizzo delle risorse hardware

Un Sistema di calcolo è composto da:

- **Dispositivi Fisici:** CPU, Memorie, Periferiche
- **Sistema Operativo:** gestisce e controlla l'utilizzo dell'hardware da parte dei programmi:
- **Programmi di Sistema:** forniscono una interfaccia verso le risorse gestite dall' OS, semplificando lo sviluppo e l'esecuzione di programmi applicativi (gestione file, configurazione del sistema, editor, compilatori, ...)
- **Programmi Applicativi:** applicazioni che risolvono problemi specifici (es: DBMS, Word processor, Web browser, ...)
- **Utenti:** qualsiasi agente che usi il sistema di calcolo per risolvere un problema (utente umano, altro OS, ...)

1.1.2 Punti di vista sul Sistema Operativo

- **Utente:** L' OS è un'interfaccia verso l'hardware che semplifica e rende più efficiente la risoluzione di un problema
- **Sistema di calcolo:** L' OS è il "programma" più "vicino" ai dispositivi fisici. Esso svolge funzioni di:
 - **Assegnazione** di risorse
 - **Controllo** dell'utilizzo del sistema di calcolo stesso

1.1.3 Obiettivi primari di un OS

Un OS può essere definito come il programma che permette di:

- Realizzare un sistema di calcolo che **supporti** l'esecuzione dei programmi utente in modo più efficiente e controllato
- Realizzare e supervisionare le funzioni necessarie per la **gestione** ed **assegnazione** delle risorse agli utilizzatori

Solitamente l' OS può essere definito come il programma sempre in esecuzione nel sistema di calcolo [si usa il termine kernel].

Il resto è costituito da programmi **di sistema** o **applicativi**.

1.2 Organizzazione di un Sistema di Calcolo

1.2.1 Schema di un Sistema di Calcolo

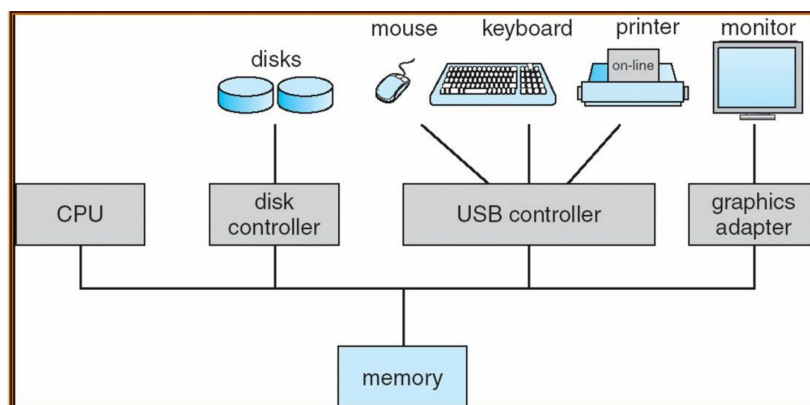


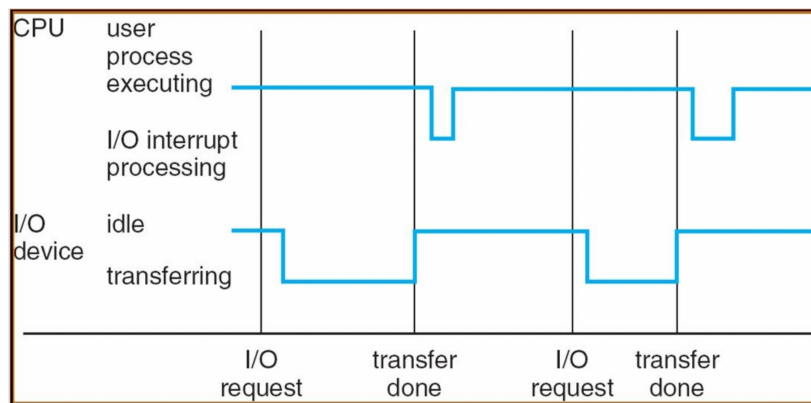
Figure 1: CPU's e Device Controller possono concorrere per l'uso del bus e per l'accesso alla memoria tramite il bus

1.2.2 Dispositivi di I/O

- CPU e Device operano concorrentemente
- Un controller gestisce uno specifico device ed utilizza un buffer locale per interagire con la CPU
- Il controller fa operazioni di R/W dei dati nel suo buffer locale
- La CPU fa operazioni di R/W nel buffer locale del controller
- La sincronizzazione tra controller e CPU avviene tramite **Interrupt**

Gestione degli Interrupt:

1. Il segnale di Interrupt viene inviato dal controller
2. Il processore intercetta il segnale e determina di che Interrupt si tratta
3. Il processore invoca l'esecuzione della corretta **procedura di servizio** per la gestione dell'Interrupt
4. Lo stato della CPU viene salvato e ripristinato al termine della gestione
5. La tecnica usata più di frequente sfrutta un **vettore degli Interrupt**



1.2.3 Memoria

Si distingue tra:

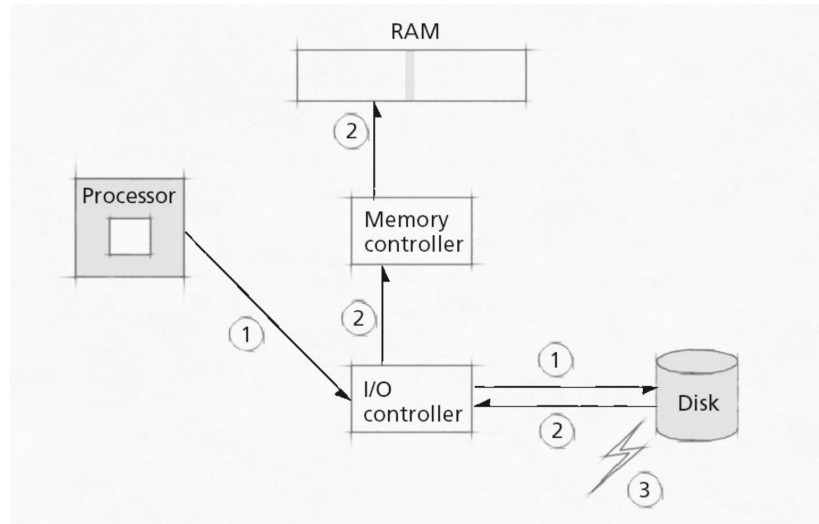
- **Memoria Principale** (RAM): l'unica memoria (di grandi dimensioni) accessibile in maniera diretta dalla CPU [Volatile]
- **Memoria Secondaria** (dischi, nastri, flash, ...) Sopprime alle limitazioni della memoria principale [Permanente]

I supporti ad oggi più diffusi sono i dischi magnetici (HD) ed i dispositivi di memoria non-volatile (flash, SSD). Sono dotati di un controller che ne gestisce l'interazione con il resto del sistema.

1.2.4 Accesso Diretto alla Memoria (DMA)

Consiste in una tecnica alternativa all'I/O guidato dalle interruzioni con la quale:

1. Un processore manda una richiesta di I/O al controller I/O, che a sua volta manda la richiesta al disco. Il processore continua ad eseguire istruzioni.
2. Il disco manda dati al controller I/O; i dati sono piazzati nell'indirizzo di memoria specificato dalla richiesta di DMA.
3. Il disco manda un Interrupt al processore per indicare che la I/O è completa.



1.3 Sistemi Mono/Multi Programmati

1.3.1 Sistema Monoprogrammato

- Oltre all'OS in memoria risiede **al più** un applicativo
- Il sistema esegue un **job** alla volta
- **Sistemi batch:** L'OS è rudimentale e, dunque, si limita a trasferire il controllo da un job al successivo

1.3.2 Sistema Multiprogrammato

MultiProgrammazione: possibilità di gestire più programmi residenti **contemporaneamente** nella memoria principale (RAM).

In memoria risiedono parti dei lavori da eseguire. L'OS ne seleziona uno e lo esegue fino a che termina o inizia l'attesa di un evento esterno (es. I/O). Se questa condizione si avvera, l'OS passa ad eseguire un altro lavoro (tra quelli in memoria)

1.3.3 Time Sharing (o multitasking)

Versione più raffinata della multiprogrammazione.

- In questa tecnica i **Context Switch** tra i job avvengono ad intervalli di tempo determinati dall'OS stesso.
- Il tempo della CPU (inteso come risorsa) viene suddivisa tra più job/utenti
- I Context Switch occorrono ad una frequenza t.c. ogni utente abbia la percezione di utilizzare un sistema di calcolo a lui dedicato
- Consente un rapporto Utente/OS più interattivo

1.3.4 Nozioni di Processo

Definizione → **Processo:** Un'attività unitaria di elaborazione, caratterizzata da un singolo **flusso sequenziale di esecuzione**, uno stato corrente, ed una collezione di **risorse** assegnate dal sistema.

- Per ogni user possono esistere uno o più processi contemporanei in memoria centrale.
- La memoria centrale potrebbe non essere sufficientemente ampia per contenere tutti i processi. → Alcuni processi vengono fatti risiedere in memoria di massa (secondaria) in una zona dedicata [**job pool**].
- L'OS sceglie quali processi caricare nella memoria principale (RAM) [**job scheduling**]
- L'OS sceglie quale processo eseguire tra quelli in RAM [**CPU scheduling**]
- L'OS può decidere di spostare dei processi dalla RAM alla memoria secondaria [**swap-out**]

Tecniche a supporto della MultiProgrammazione data la necessità di mantenere molti processi in RAM ed il fatto che la memoria principale è limitata, risulta necessario impiegare delle politiche di **assegnazione della memoria** (intesa come **risorsa**):

- **Swapping:** migrazione di processi tra RAM e memoria secondaria (avvicendamento dei processi).

- **Memoria Virtuale:** introduce uno spazio di indirizzamento logico con il quale si può eseguire un processo non completamente caricato in RAM.

1.4 Principali Funzionalità di un Sistema Operativo

1.4.1 Modalità di Funzionamento

Problema In presenza di più processi che condividono risorse è necessario garantire che ogni processo non danneggi gli altri.

A tal scopo i moderni OS possono operare in diverse **modalità di funzionamento** [utente & kernel]

Per rendere possibile queste modalità c'è bisogno di un **supporto hardware** opportuno come un **bit di modalità** della CPU tramite il quale si abilita o meno l'esecuzione delle **istruzioni privilegiate** (eseguibili solo in modalità kernel) come lo switch da modalità utente a modalità kernel, la gestione degli interrupt, la gestione e assegnazione delle risorse, ...

Domanda Se la gestione delle risorse avviene tramite istruzioni privilegiate, come fa un processo utente ad ottenere le risorse?

Si usano le **System Call**:

- tramite l'esecuzione di una system call un processo invoca l'intervento dell'OS [richiesta di un servizio]
- la chiamata genera un interrupt (trap) → tramite il vettore degli interrupt il controllo passa alla apposita routine di gestione [interna all'OS & opera in modalità kernel]
- in quel momento si analizza la richiesta e (eventualmente) si fornisce il servizio richiesto
- ritorno in modalità utente

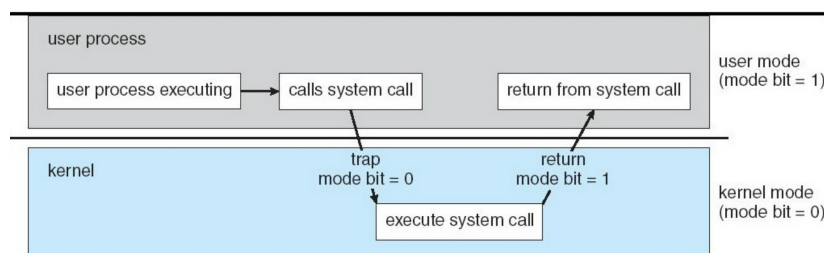


Figure 2: Visualizzazione di una System Call tipica

1.4.2 Tipi di Funzionalità offerte dall'OS

Gestione dei Processi L'OS deve fornire le funzionalità per la **gestione dei processi**:

- **creazione** e **cancellazione** dei processi
- **sospensione** e **ripristino** dei processi
- meccanismi per la **comunicazione** e **sincronizzazione** tra processi
- **gestione** delle situazioni di **stallo**

Gestione della RAM L'OS deve fornire le funzionalità per la **gestione della memoria principale**:

- Per la gestione della Memoria Secondaria (Dischi):
 - tenere traccia delle porzioni di RAM sono utilizzate e da quali processi
 - decidere quali processi e dati devono essere caricati in RAM e quali spostare altrove
 - **assegnazione** e **revoca** dello **spazio di memoria** ai processi a seconda delle necessità
- Per la gestione della Memoria Terziaria (Nastri, Supporti ottici):
 - gestione dello spazio libero
 - assegnazione dello spazio
 - scheduling del disco

Gestione di I/O L'OS deve fornire le funzionalità per la gestione dell' Input/Output:

- gestione dei buffer (zone di memoria utilizzate per lo scambio di dati con i dispositivi)
- gestione del caching (spostamento di dati in memorie più veloci)
- gestione dello spooling (consentire l'esecuzione di processi e in contemporanea l'esecuzione asincrona di diverse operazioni di I/O)

L'OS deve "nascondere" le caratteristiche peculiari dei dispositivi hardware, usando i **device-driver** (interfacce software verso i controllori hardware)

Gestione della Memoria di massa e dei files Le informazioni sono organizzate in **files** a loro volta organizzati in **file systems**. L'OS fornisce funzionalità per la loro gestione:

- creazione, cancellazione e accesso a file e directory
- associazione dei file ai dispositivi di memoria secondaria
- gestione dell'affidabilità (es. creazione copie di backup)

Protezione e Sicurezza **Protezione:** strategia di controllo dell'accesso alle risorse del sistema di calcolo da parte dei processi o utenti.

Sicurezza: strategia di difesa del sistema da accessi o operazioni dannose o distruttive, provenienti dall'interno o dall'esterno [es. DOS → Denial Of Service; Worm; Virus; Identity Theft; ...].

È necessario che l'OS sia in grado di distinguere tra gli utilizzatori del sistema di calcolo, per farlo si impiegano user IDs e group IDs assegnati agli utenti e ai loro processi.

Tra le proprietà che si vogliono garantire ci sono:

- **Disponibilità:** protezione del sistema contro le interruzioni del funzionamento.
- **Privatezza:** assicura che ogni utente compia solo accessi autorizzati ai dati.
- **Integrità dei dati:** protezione dei dati da modifiche non autorizzate o incontrollate.
- **Autenticazione:** gestione delle identità degli utenti e delle autenticità dei loro dati e messaggi.

2 Struttura e Servizi degli OS

2.1 Servizi offerti dall'OS

2.1.1 Servizi di Base

- **Interfaccia Utente:** Può essere di diverso genere [CLI, GUI o batch (interfaccia a lotti)].
- **Esecuzione di Programmi:** Caricamento in memoria ed esecuzione di programmi.
- **Operazioni di Input/Output:** L'esecuzione di un programma può richiedere operazioni di I/O. [L'OS fa da interfaccia tra utente/programma e i dispositivi I/O].
- **Gestione del File System:** Permette accesso ai file, creazione, read, write, ricerche [con gestione di accessi e privilegi].
- **Comunicazioni:** Scambio dati ed informazioni tra processi [shared memory & message passing]
- **Rilevamento e Gestione degli errori:** Dovuti a guasti HW, errori o violazioni SW. L'OS deve eseguire le giuste operazioni per assicurare che il sistema di calcolo funzioni correttamente e con consistenza.
- **Assegnazione delle risorse:** I processi competono per le risorse del sistema. L'OS le concede seguendo determinate strategie in modo tale da massimizzarne l'efficienza.
- **Contabilizzazione dell'uso delle risorse:** Le informazioni sull'impiego del sistema possono essere utilizzate per:
 - migliorare le prestazioni
 - controllare l'attività degli utenti
- **Protezione e Sicurezza:** Particolarmente utile nei sistemi multiutente

2.1.2 Interfaccia con l'utente

Interprete di Comandi A volte integrato nel kernel ma più spesso è un programma o processo effettivo [detto shell]. Acquisisce un comando in forma di stringa, lo decodifica e lo esegue. Ciò può causare l'esecuzione di un frammento di codice della shell stessa (built-in) o l'invocazione di un programma (solitamente di sistema [tipico nelle shell di Linux/UNIX])

Interfaccia Grafica L'interfaccia è visuale e modella un Desktop in cui programmi, file, directories, etc sono rappresentati da oggetti grafici. Combina e coordina l'uso di diversi tipi di dispositivi, quali tastiera, schermo, mouse, etc)

2.2 System Call

Costituiscono **l'interfaccia tra i processi e l'OS**.

Assembly e altri linguaggi di alto livello possono effettuare direttamente delle System Call.

Solitamente la **run-time support interface** di un compiler include una **interfaccia alle System Call**:

1. ad ogni System Call viene associato un numero che fa da indice in una tabella per determinare il codice corrispondente da eseguire
2. l'interfaccia invoca la System Call nel kernel e, al suo termine, restituisce il **return status** ed eventuali **return values**
3. il chiamante non necessita di conoscere i dettagli di implementazione della System Call [basta conoscere le modalità di Call e Return]

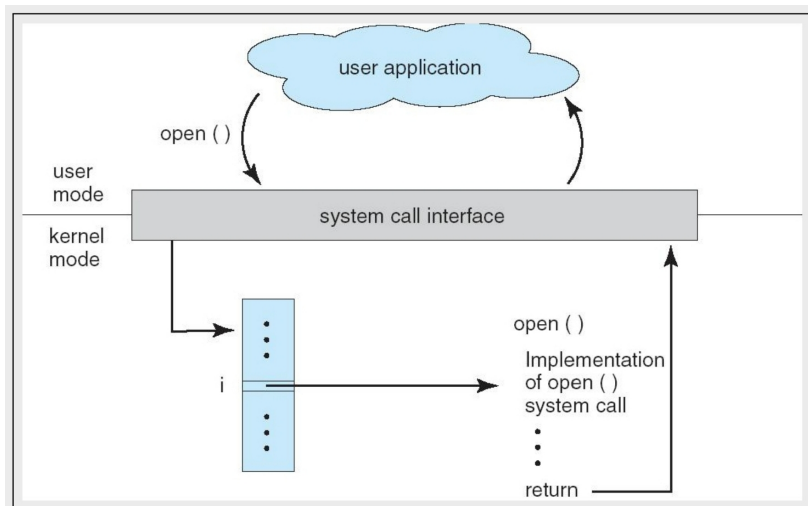


Figure 3: Esempio schematico di una System Call a open()

2.2.1 Passaggio di parametri

Per poter passare dei parametri ad una System Call ci sono tre metodi generali :

- tramite **registri**
- usando un **blocco di memoria** e passandone l'indirizzo tramite registri
- usando lo **stack** [push - pop]

2.2.2 Tipi di System Call

- **Controllo dei processi:**
 - Creazione & Terminazione di processi
 - Esecuzione di programmi
 - Allocazione & Deallocazione di memoria
 - Segnalazione dei processi & Attesa degli eventi
 - Impostazione degli attributi dei processi

- **Gestione dei file:** Riferito alla gestione degli attributi dei file:

- Creazione & Rimozione
- Apertura & Chiusura
- Posizionamento
- Read & Write
- Accesso

- **Gestione dei dispositivi:**

- Allocazione & Rilascio dei dispositivi
- Posizionamento
- Read & Write
- Accesso & Impostazione degli attributi
- Mount & UnMount dei dispositivi

- **Accesso a informazioni sul sistema:**

- Informazioni su HW & SW installato
- Accesso & Impostazione degli attributi di processi ed utenti
- Data & Ora del Sistema

- **Comunicazione:**

- Creazione & Chiusura di connessioni
- Invio & Ricezione di messaggi
- Accesso ad Informazioni & Attributi delle connessioni
- Gestione della memoria condivisa

2.2.3 Programmi di sistema

I programmi di sistema forniscono un ambiente per lo sviluppo e l'esecuzione di programmi utente. Possono essere complessi ma anche delle semplici interfacce per le System Call. Degli esempi sono:

- Gestione & Manipolazione dei file [creazione, spostamenti, editing, navigazione del file system]
- Informazioni sullo stato del Sistema [monitoraggio, configurazione, accounting]
- Supporto a linguaggi di programmazione [compiler, interpreti]
- Caricamento & Esecuzione di programmi [linker, loader, debugger]
- Comunicazioni [email, remote access, web browser]

2.3 Struttura di un OS

2.3.1 Struttura Semplice - Monolitica

Struttura usata dai primi OS [più semplici] e ha le seguenti caratteristiche:

- L'OS è costituito da un **singolo programma** composto da un insieme di procedure che ne fanno da componenti
- Ogni componente dell'OS è contenuta nel Kernel e può interagire con ciascuna delle altre componenti [tramite funzioni e chiamate a procedure]
- Funzioni come progettazione, realizzazione, debugging, manutenzione risultano molto complesse
- Il sistema è più sensibile a errori lato software per via dell'operare in modalità privilegiata delle componenti del kernel

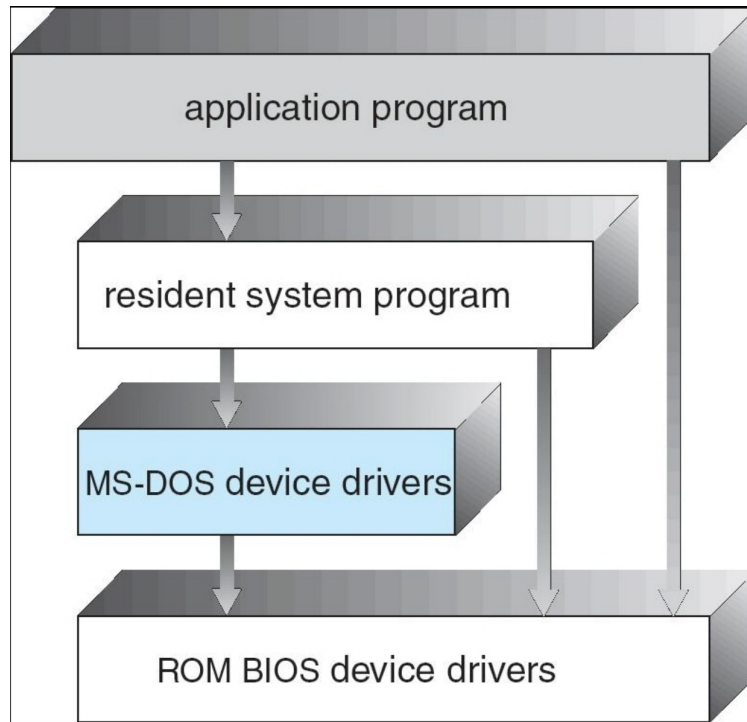


Figure 4: Esempio di OS Monolitico [MS-DOS]

Un esempio di OS quasi monolitico sono i primi sistemi UNIX che avevano solo 2 livelli: Programmi di sistema & Kernel; quest'ultimo consisteva di tutto ciò che stava tra l'interfaccia alle system call e l'HW ed implementava tutto [file system, scheduling, gestione della memoria]

2.3.2 Struttura Stratificata

Le idee alla base di questa struttura sono le seguenti:

- L'OS va partizionato in una gerarchia a livelli [0 - HW ... N - UI]
- Ognuno dei livelli raggruppa funzionalità omogenee
- Ogni livello può invocare **solo** le funzionalità dei livelli inferiori [da N a 0]
- Le implementazioni non sono esplicite ai livelli superiori e ciò comporta ad una maggiore modularità ed indipendenza di funzioni come debugging, implementazioni e modifiche
- Ne risentono le performance che degradano per via del forzato utilizzo delle interfacce tra layer

2.3.3 Struttura a MicroKernel

L'idea alla base è quella di mantenere il kernel il più essenziale possibile [il microkernel mantiene solo le funzionalità essenziali]

- **Componenti Interne:**

- Gestione di memoria di basso livello
- Sincronizzazione & Comunicazione tra processi

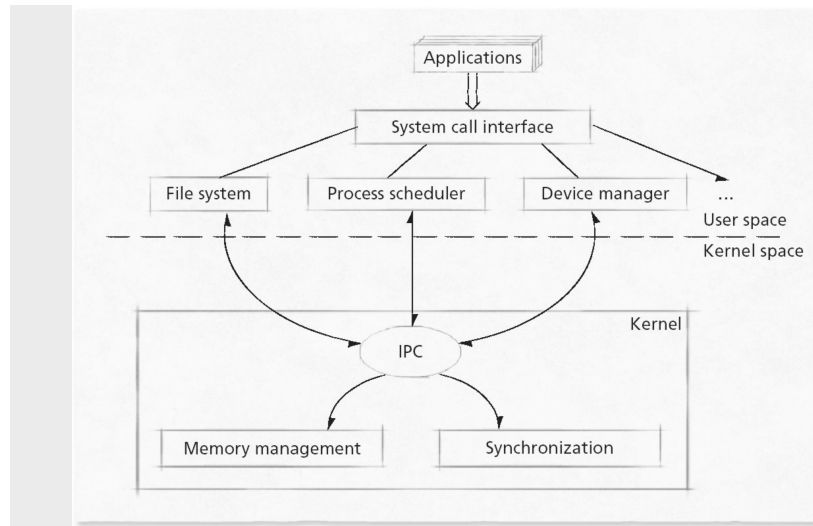
- **Componenti Esterne:**

- Gestione Processi
- File System
- Networking & Dispositivi [eseguiti fuori dal Kernel]

- **Vantaggi:**

- **Modularità:** il kernel resta funzionante anche se una componente esterna fallisce
- **Portabilità:** supporto a nuovo hardware ma con modifiche limitate
- **Sicurezza:** Meno codice in modalità privilegiata riduce i rischi

- **Svantaggi:** La performance può degradare per via dell'utilizzo delle interfacce tra/con le componenti esterne al microkernel



2.3.4 Struttura Modulare

Questa struttura punta a combinare i vantaggi della struttura a livelli e l'approccio a microkernel:

- Si adottano i principi della programmazione orientata agli oggetti
- Il kernel fornisce ed implementa le funzionalità essenziali
- Altre funzionalità vengono aggiunte **dinamicamente** [in avvio o in caso di necessità] caricando gli opportuni moduli
- I moduli sono interfacciabili con gli altri [ciò comporta ad una maggiore flessibilità rispetto alla struttura a livelli]
- Come nella struttura a microkernel le funzionalità essenziali sono accessibili in modo protetto e controllato.
- I moduli possono comunicare anche direttamente tra loro senza passare per il kernel [maggiore flessibilità, portabilità ed efficienza]

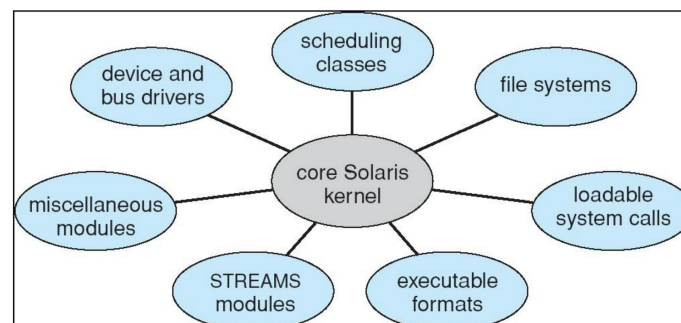


Figure 5: Schema dei moduli caricabili per il sistema Solaris

2.3.5 Caso Linux

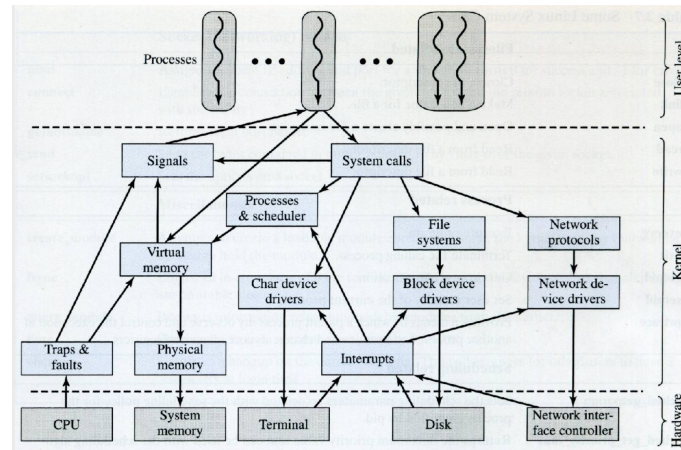


Figure 6: Schema del kernel (con moduli) di Linux

2.3.6 Struttura Ibrida?

I sistemi operativi reali (general purpose) solitamente adottano una struttura ibrida:

- pone come obiettivo il miglioramento dell'efficienza ed efficacia
- Linux, Solaris, Windows → hanno una struttura monolitica ma con l'aggiunta della possibilità di caricare moduli aggiungendoli in modo dinamico (al kernel o al microkernel)
- Mac OS X, iOS → combinano caratteristiche delle strutture a microkernel, stratificata e modulare (kernel è derivato da Unix BSD)
- Android → simile a iOS. Il kernel è derivato da Linux (le applicazioni Java eseguono su una macchina virtuale, Dalvik, ART)

2.4 Virtual Machines

Una VM è un'applicazione che emula le funzionalità di un sistema di elaborazione.

- Sulla Virtual Machine è possibile eseguire applicazioni non compatibili con il vero HW della macchina.
- La VM fornisce una interfaccia identica ad una macchina fisica ed è possibile condividere le risorse tra più VM.
- L'OS crea l'illusione di processi multipli, ciascuno in esecuzione su un proprio processore con una propria memoria.
- Ogni VM può girare un OS diverso.
- **Vantaggi:**
 - Uso di HW comune
 - Protezione delle risorse comuni
 - Maggiore sicurezza (VM isolate tra loro)
- **Efficienza?**
 - CPU virtuale e dunque più **lenta** di una fisica
 - I/O **op** più **veloce** dato che i dispositivi sono emulati

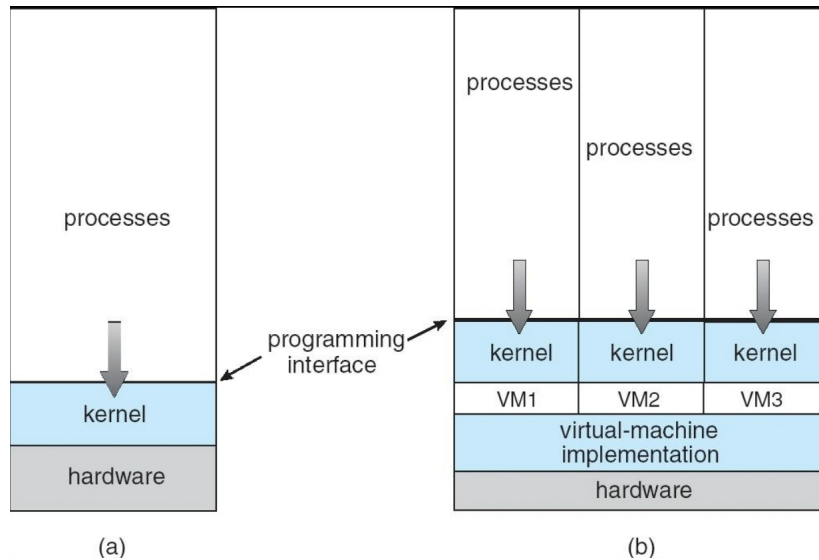


Figure 7: Modello di macchina non virtuale (a) e di macchina virtuale (b)

3 Processi

3.1 Nozioni di Processo

Programma → entità **statica**.

Processo → entità **dinamica**.

Ad **un** processo corrisponde **un** programma.

Ad **un** programma possono corrispondere **più** processi.

Per esempio lo stesso programma impiegato con input diversi[[**istanza**]].

Definiamo **processo** come una unità di elaborazione che viene eseguita **sequenzialmente**.

Le **proprietà** che lo caratterizzano sono:

- **Code Region** [il programma]
- **Program Counter** [instruction pointer]
- contenuto dei **registri**
- **Data Region** [sezione dati]
- **Stack Region** [Stack]
- insieme di **attributi** [informazioni per la sua gestione]

Al processo possono essere anche assegnate delle risorse come:

- memoria allocata
- file aperti
- dispositivi di I/O
- ...

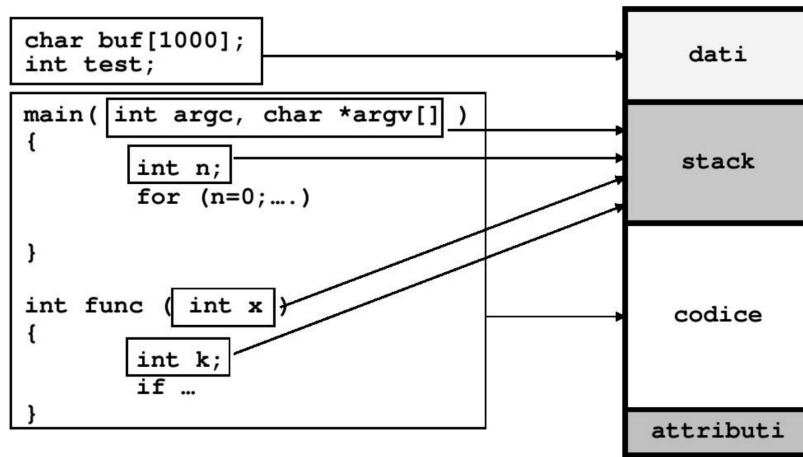


Figure 8: Un programma C e la rappresentazione della sua **immagine**

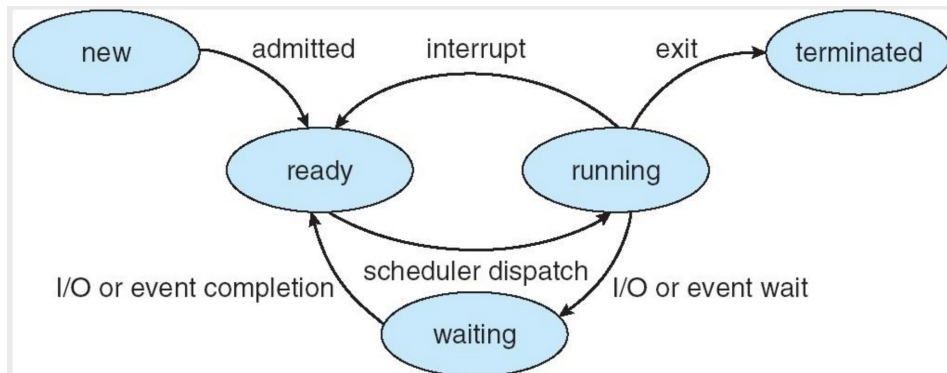
3.1.1 Ciclo di vita di un Processo

Ogni processo **evolve** durante la sua esistenza: in ogni istante si trova in un determinato **stato**. Gli stati principali sono:

- **new**: appena creato
- **ready**: pronto per essere eseguito
- **running**: in esecuzione (detiene l'uso della risorsa 'processore')
- **waiting**: in attesa di qualche evento (terminazione I/O, ...)
- **terminated**: ha finito la sua esecuzione

Il passaggio 'ready → running' è detto **dispatching** e viene eseguito da un particolare modulo dell'OS detto **dispatcher**.

3.1.2 Transizioni tra gli Stati



3.1.3 PCB - Process Control Block

Il **PCB** [process descriptor] è una struttura contenente le informazioni su un processo nell'OS [descrive l' **execution context** del processo]. Tra esse:

- **PID** [process ID, integer]
- lo **stato** [running, ready, waiting, ...]
- il valore del **Program Counter**
- il contenuto dei **registri** della CPU
- info utili allo **scheduling** di CPU
- info sulla **gestione** della **memoria**
- info sull'**accounting**

- info sullo **stato** dell'I/O

3.1.4 Process Table

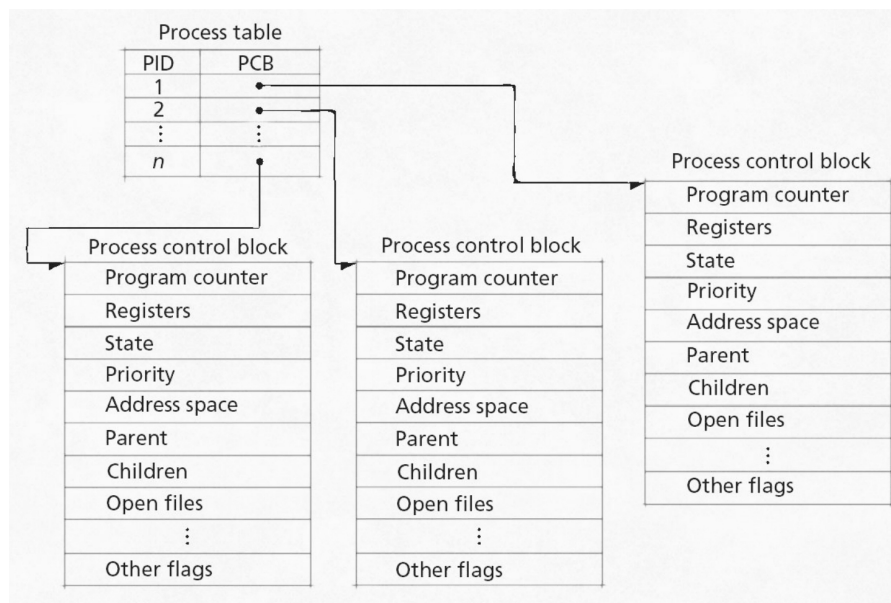
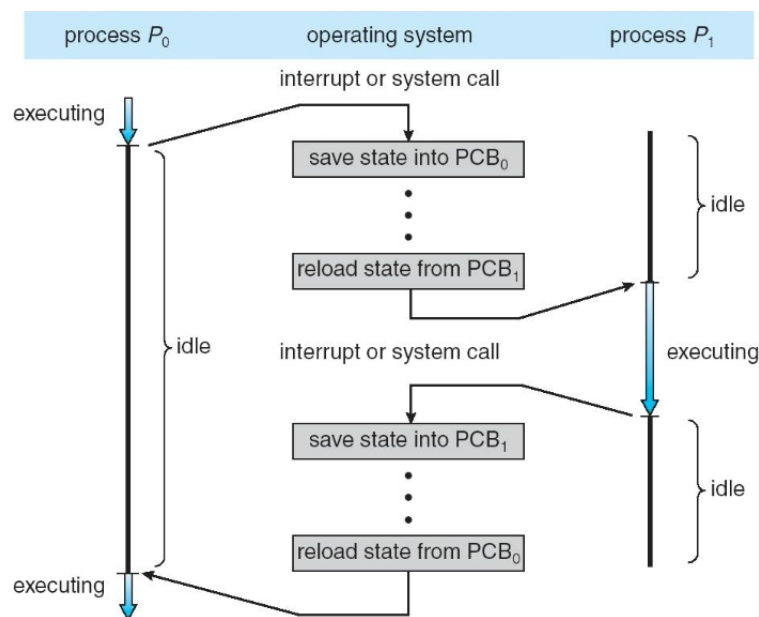


Figure 9: Tutti i PCB sono raccolti in una PT (global o per-user)

3.1.5 Context Switch

Durante un **Context Switch** l'OS salva l'**execution context** del processo P_0 [running] e ripristina quello del processo P_1 "entrante" [ready $\rightarrow$ running].



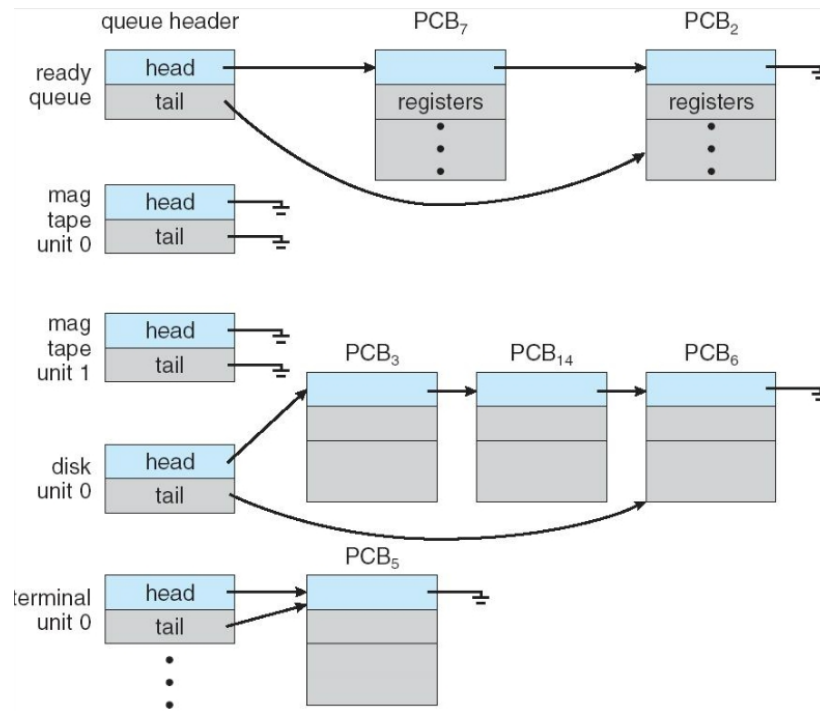
3.1.6 Code di Processi e Risorse

L'OS gestisce le richieste dei processi tramite:

- code di scheduling
- politiche di scheduling

Tra i vari tipi di queue [associate differenti tipi di risorse] troviamo:

- **job queue** [coda dei processi] raccoglie i PCB dei processi presenti nel sistema
- **ready queue** [coda dei processi pronti] raccoglie tutti i PCB dei processi ready
- **device queue** [coda del dispositivo] raccoglie tutti i PCB dei processi in attesa di un I/O device



Per poter inserire in coda un processo correlato al verificarsi di un evento si usa il **diagramma di accodamento**:

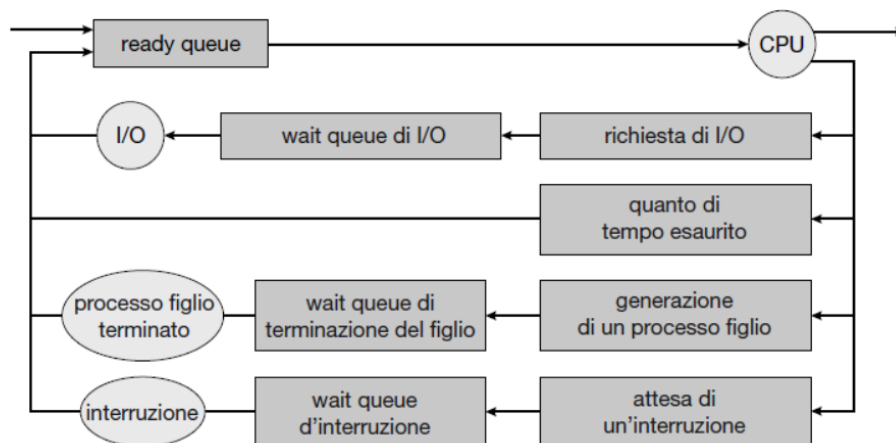


Figure 10: Diagramma di accodamento per lo scheduling dei processi

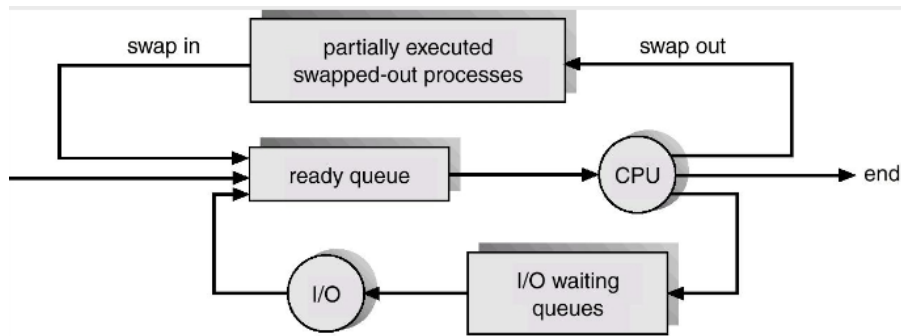
3.1.7 Scheduler

Nel momento in cui una risorsa si rende libera, l'OS ha il compito di scegliere uno dei processi in attesa nella coda corrispondente. I due principali scheduler sono:

- **Long Term Scheduler** [job scheduler] decide quali processi (non ancora iniziati) vanno caricati in RAM, quali vanno inseriti in 'ready queue' e quali devono restare nella memoria secondaria. Determina il **grado di multiprogrammazione** del sistema e deve bilanciare in maniera efficiente tra processi **CPU-Bound** e **I/O-Bound**.
- **Short Term Scheduler** [CPU scheduler] sceglie a quale processo [nella ready queue] assegnare la CPU

3.1.8 Swapping e Medium-term Scheduler

Oltre ai due scheduler menzionati sopra, spesso vi è uno ulteriore chiamato **Medium Term Scheduler** che ha il compito di modulare il carico a cui è soggetto il sistema spostando dei processi dalla RAM alla memoria secondaria o viceversa [swap-in & swap-out] (solitamente eseguiti solo su processi già **parzialmente eseguiti**).



Lo **swapping** viene attivato quando è necessario cambiare in maniera **dinamica** il grado di multi-programmazione del sistema per ottenere un miglior utilizzo della CPU.

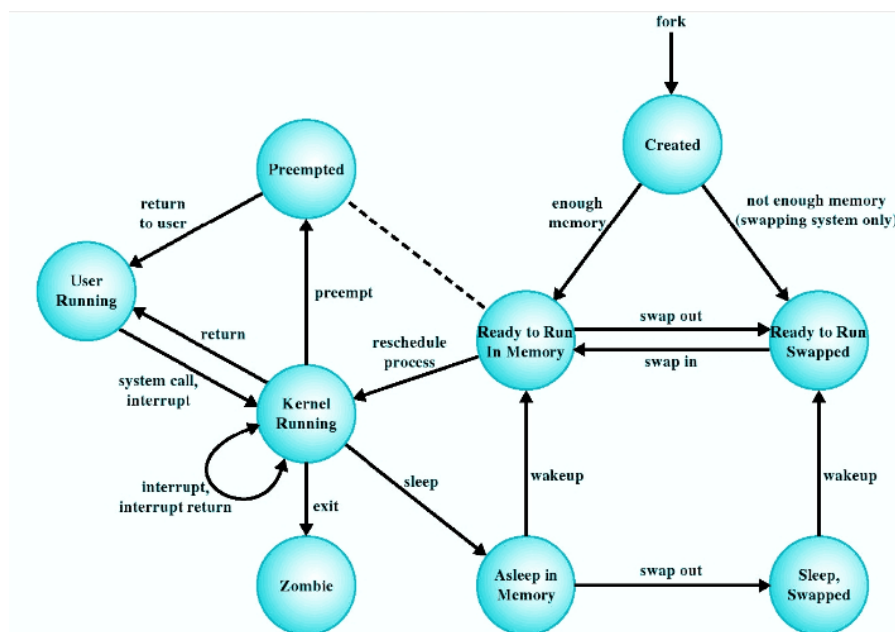
Perchè fare swap-out?

- **(ri)bilanciare** il rapporto tra CPU-Bound e I/O-Bound
- la RAM non è sufficiente a soddisfare le richieste di tutti i processi
- **sotto richiesta** dallo user o da qualche processo [es. accesso/modifica execution context in debugging]

Dopo essere stato '**swapped out**' un processo passa in uno stato detto **swapped** (o **suspended**) creando quindi la possibilità di avere dei processi '**ready+swapped**' o '**waiting+swapped**'

3.1.9 Transizione tra gli Stati (caso UNIX)

- **created:** il processo è stato solo creato.
- **user/kernel running:** in esecuzione in modalità user o kernel.
- **ready in memory:** pronto per andare in esecuzione
- **preempted:** bloccato dal kernel al fine di eseguire un altro processo.
- **asleep in memory:** caricato in memoria ma in attesa di un evento.
- **ready swapped:** pronto per eseguire ma attualmente 'swapped-out' (su disco).
- **sleeping swapped:** attualmente 'swapped-out' e in attesa di un evento.
- **zombie:** terminato ma in attesa che il padre riceva [tramite 'wait()'] lo status di ritorno.



3.2 Operazioni sui Processi

L'OS deve implementare delle operazioni fondamentali sui processi.

3.2.1 Creazione di un Processo

Ogni processo può creare altri processi tramite l'invocazione di una apposita **system call** (es. 'fork()' in UNIX).

- il processo creatore è detto **parent** mentre quello creato è detto **child**.
- il nuovo processo avrà un nuovo PID e PCB. [in UNIX tutti i processi discendono da 'init' (PID = 1)]
- il PCB del parent contiene informazioni per identificare i suoi child e viceversa. [si creano gerarchie di processi].

3.2.2 Creazione di Processi

Cosa accade quando un processo ne crea un altro?

Come procedono?

- padre e figlio continuano ad eseguire in modo concorrente
- padre attende la terminazione del figlio

Che codice esegue il figlio?

- il processo figlio è un *duplicato* (quasi) identico al padre
- il figlio carica ed esegue un codice diverso da quello del padre

Quali risorse vengono condivise?

- il processo figlio condivide tutte (o parte) delle risorse del padre
- nessuna risorsa viene condivisa (il figlio dovrà acquisire le risorse che necessità)

3.2.3 Terminazione di Processi

In condizioni normali un processo termina eseguendo l'ultima istruzione del suo programma e richiedendo la sua eliminazione tramite una systemcall 'exit()' [UNIX]. A seguito di una invocazione di 'exit()' l'OS:

- disalloca le risorse concesse al processo terminato
- trasmette al processo padre eventuali informazioni/dati relativi alla terminazione del figlio

La terminazione può avvenire per cause "anormali":

- errori o violazioni compiute dal processo → OS lo uccide
- richiesta di terminazione da parte di un altro processo
- in alcuni OS se il creatore del processo termina, anche il figlio lo fa

3.2.4 Gerarchia di Processi (Android)

Gli OS di dispositivi mobili possono dover gestire scarsità di risorse ed una soluzione consiste nel terminare uno o più processi. La scelta del processo da terminare avviene partizionando i processi in classi:

- **processo in primo piano**: visibile su schermo ed utilizzato dallo user
- **processo visibile**: non visibile in primo piano ma funzionale a processi in primo piano
- **processo di servizio**: che esegue in background funzionalità evidenti all'utente
- **processo in background**: attività non visibili all'utente
- **processo vuoto**: non contiene componenti attive associate ad app

3.2.5 Meccanismo di Terminazione (UNIX)

In UNIX ogni processo ha un **padre**. Così facendo si può sempre *trasmettere lo status di uscita di un processo al padre*:

- il padre esegue una systemcall 'wait()' per ricevere dall'OS il PID (ed exit status) del processo terminato [pid_t wait(int status)]
- così facendo ogni processo può venire a conoscenza di quali tra i suoi figli sono terminati
- la completa deallocazione delle risorse avviene SOLO se il padre esegue 'wait()'
- durante l'attesa del 'wait()' il processo terminato rimane in uno stato detto **zombie**

- per permettere questo meccanismo, nel caso il padre termini prima del figlio, quest'ultimo viene adottato dal processo init

3.3 Comunicazione tra Processi

3.3.1 Processi Indipendenti o Cooperanti

Processi Indipendenti due processi si dicono indipendenti se l'esecuzione di uno non influenza l'esecuzione dell'altro [riproducibilità della computazione]

Processi Concorrenti due processi si dicono concorrenti se l'esecuzione di uno influenza l'esecuzione del secondo o viceversa. [riproducibilità non garantita] l'esito dipende dalla velocità reciproca dei due processi. I processi concorrenti possono essere cooperanti e/o competitori

Processi Cooperanti - Utilità

- condivisione di informazioni e risorse
- miglioramento dei tempi di computazione
- modularità nell'implementazione/soluzione di un problema
- convenienza nel compiere diverse azioni in contemporanea

3.3.2 Comunicazione tra Processi Concorrenti

Processi concorrenti necessitano di meccanismi che supportino lo scambio di informazioni e dati [IPC - Inter Process Communication].

I modelli principali di IPC sono due:

- **Shared Memory**
- **Message Passing**

3.3.3 Shared Memory

Modello di comunicazione che prevede l'esistenza di un *ambiente globale* accessibile dai processi (alcune risorse sono **condivise**). [esiste uno spazio di memoria indirizzabile da tutti i processi che cooperano].

- il blocco di memoria è concesso dall'OS come risposta ad una invocazione da parte di un processo via syscall:
 - **UNIX** → shmget()
 - **POSIX** → shm_open()
- ogni processo che voglia usare lo stesso blocco di memoria lo dovrà annettere al proprio spazio di indirizzamento (es. syscall 'shmat()' di UNIX)
- l'OS delega ai processi uso/gestione della memoria condivisa (es. programma C sotto UNIX sarà accessibile tramite puntatore restituito da 'shmat()')

Gli accessi alla *Shared Memory* vanno **regolamentati**.

3.3.4 Message Passing

Modello di comunicazione che al posto di un ambiente globale utilizza un **communication link**, la comunicazione avviene tramite *invio* e *ricezione* di messaggi.

Le due operazioni di base sono:

- **send**
- **receive**

Dei punti da chiarire per caratterizzare il tipo di comunicazione sono i seguenti:

- il *link* può essere usato da più di due processi?
- tra due processi ci può essere più di un *link*?
- il *link* è monodirezionale o bidirezionale?
- come vengono gestiti i messaggi?

3.3.5 Caratteristiche della Comunicazione

Gli aspetti che caratterizzano diverse tipologie di comunicazione sono:

- **naming:** *diretta* o *indiretta*
- **sincronizzazione:** operazioni *sincrone* o *asincrone*
- **buffering:** buffer di messaggi *gestiti automaticamente (OS)* o *gestiti esplicitamente(processi)*

Comunicazione Diretta Si nomina esplicitamente il proceso partner con cui si sta scambiando il messaggio. Quindi:

- **send(P, msg)** → spedisco il msg a 'P'
- **receive(Q, msg)** → ricevo il msg da 'Q'

Lo schema è simmetrico, ogni *link* è associato **esattamente** due processi [un link per coppia]

Comunicazione Indiretta Si utilizzano le **mailbox**: ogni messaggio è inviato ad una mailbox univocamente identificata:

- **send(Box, msg)** → spedisco il msg alla Box
- **receive(Box, msg)** → prelevo un msg da Box

Lo schema è tale che:

- due processi comunicano condividendo la stessa *mailbox*
- più link possono essere usati dagli stessi due processi
- ogni link può essere utilizzato da più di due processi

Nel caso la mailbox venga usata da più di due processi, sono necessarie delle politiche:

- come determinare il destinatario nel caso di multipli *receive*
- chi "possiede" la mailbox? [OS oppure processi?]

Primitive Sincrone o Asincrone Sia **send** che **receive** possono essere *sincrone (bloccanti)* o *asincrone (non bloccanti)*:

- **send**
 - **sincrono:** il mittente si blocca in attesa della ricezione (da destinatario o mailbox)
 - **asincrono:** il mittente non attende la ricezione
- **receive:**
 - **sincrono:** il destinatario dopo aver eseguito una receive si blocca in attesa che vi sia un messaggio disponibile
 - **asincrono:** il destinatario dopo aver seguito una receive non si blocca (riceve *il messaggio* oppure *notifica di assenza di messaggi*)

Tipologie di Code di Messaggi Le code di messaggi sono *temporanee* e gestite dall'OS, la loro **capacità** può essere:

- **nulla:** non ci possono essere "messaggi di attesa" → *send bloccante*(no buffering).
- **limitata:** vi è un limite massimo al numero di passaggi in coda → *send bloccante se la coda è piena*
- **illimitata:** la send può essere sempre non bloccante

Esempio di Comunicazione [unnamed pipe di UNIX] In UNIX si può usare la syscall *int pipe(int pipefd[2])*:

- definisce canale monodirezionale [unnamed pipe]
- due descrittori:
 - **send** → pipefd[0] → 'write()'
 - **receive** → pipefd[1] → 'read()'

Comunicazione nei Sistemi ClientServer Altre tipologie di link sono:

- **Socket:**
 - due socket → link bidirezionale tra due processi
 - ogni processo crea una delle due estremità del link
 - in UNIX tramite systemcall 'socket()' si possono avere:
 - * tra processi dello stesso OS (unixsocket), tramite filesystem
 - * tra processi su due nodi di una rete (internetsocket), specificando IP+port
 - solitamente definiscono un processo *server* ed uno *client*
- **Remote procedure call** (invocazione di una procedura da esecutore remoto):
 - solitamente '*clientserver*'
 - un processo specifico riceve le richieste ed eventuali dati in input tramite una *specifica* porta
 - invia la richiesta all'esecutore locale per poi rispedire il risultato al processo client
 - il client opera come se invocasse una chiamata locale

4 Thread

4.1 Dai Processi ai Threads

Finora abbiamo considerato un processo come una unità elementare di elaborazione. Tale entità racchiude in sé due aspetti distinti:

- **Il possessore delle risorse:** ad un processo vengono assegnate delle risorse quali uno spazio di inserimento (codice, dati, stack) e degli attributi.
- **Il flusso di esecuzione:** un processo evolve seguendo un flusso di esecuzione (eventualmente in time sharing con altri). Il processo ha quindi uno stato, una priorità ed è soggetto a scheduling e dispatching.

Questi due aspetti possono essere considerati singolarmente come due unità distinte. Ci si può riferire alle **unità detentrici delle risorse** con i termini:

- process
- task

mentre l'unità soggetta a dispatching viene detta:

- thread
- lightweight process

N.B. la teminologia non è univoca quindi bisogna fare attenzione a cosa si intende per *thread*, *task*, *lightweight*.

4.1.1 Multithreading

Definizione: Con questo termine si indica la capacità di un OS di supportare molteplici files in esecuzione (thread) all'interno di un *singolo* processo.

I sistemi che abbiamo studiato finora, in cui vi è solo un thread per processo si chiamano *single-threaded*

- **MS-DOS** é un sistema che supporta un singolo processo che è single-threaded
- **UNIX** supporta multiprocessing per ogni utente, ma di solito ogni processo è single-threaded

N.B. Windows, Solaris, Linux, Mach,... supportano il multithreading.

In presenza di multithreading il processo viene definito come quell'unità che raccoglie allocazione delle risorse e meccanismi di protezione:

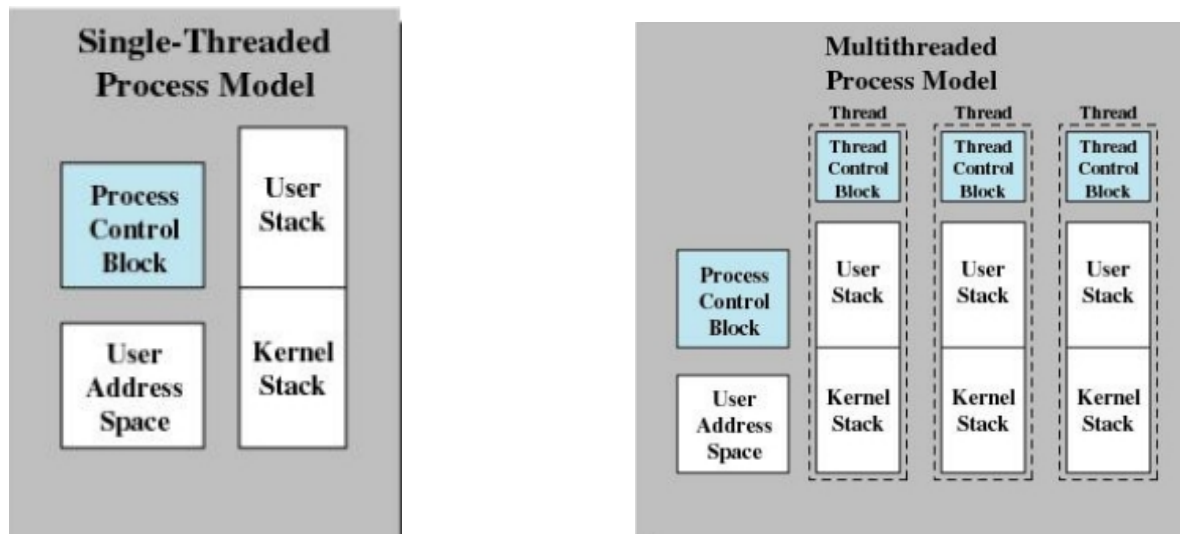
- spazio degli indirizzi
- accesso protetto (mediatamente dal OS) a CPU, ad altri processi, files e dispositivi di I/O.

Un processo può comporsi di più thread, **ognuno** dei quali possiede un *thread control block* (TCB) che descrive:

- lo stato del thread (*thread state*: ready, running, waiting ..)
- il thread context (program counter)

- lo stack (execution stack) del thread
- una eventuale zona di memoria per variabili locali, ovvero private ed accessibili **solo ai thread**.

4.1.2 Single e Multi Thread



4.1.3 Thread di un Processo

E' importante ricordare sempre che:

- **ogni** thread ha accesso alla memoria e alle risorse allocate al suo processo.
- suddette risorse sono **condivise** da tutti i thread del processo, le modifiche effettuate dai thread sono visibili agli altri. Ex: un thread ottiene i privilegi di lettura su un file ==> tutti i fratelli possono leggere suddetto file.
- è necessario utilizzare due meccanismi opportuni per controllare l'interazione/interferenza tra i thread.

4.1.4 Vantaggi dei Thread

I principali *vantaggi* dei thread derivano dalla maggior efficienza:

- in una applicazione multi-threaded si ha *maggior parallelismo*
- *creazione* di un thread in un processo: costa molto meno che creare un intero processo.
- *terminazione* di un thread: costa molto meno che teminare un processo.
 - N.B. La creazione o terminazione di un thread non implicano assegnazione o rilascio di risorse
- *switch di contesto* : lo switch tra thread **dello stesso processo** è molto più veloce di quello tra processi.
- le *comunicazioni* tra thread **dello stesso processo** sono solitamente più efficienti in quanto si svolgono sfruttando il contesto del processo. Le comunicazioni tra processi invece coinvolgono chiamate di sistema e la mediazione del kernel.

4.1.5 Svantaggi dei Thread

I principali *svantaggi* dell'impiego dei thread derivano dalla maggiore difficoltà e complessità di progettazione e implementazione delle applicazioni. In particolare:

- occorre pensarle fin dall'inizio in **modo parallelo**.
- facendo sì che i flussi di esecuzione non **interferiscano** in modo incontrollato; e che tutti gli accessi ai dati condivisi siano **protetti**.

4.1.6 Scheduling e Dispatching: Thread o Processo?

Solitamente lo scheduling e il dispatching avvengono a livello di thread(stato, priorità,...), quindi le informazioni corrispondenti sono mantenute nei TBC. Ci sono però alcune operazioni che l'OS compie e che riguardano tutti i thread di un processo, queste operazioni devono essere gestite a livello di processo ad esempio: Swapping o Terminazione di processo.

4.2 Modelli di supporto al multithreading

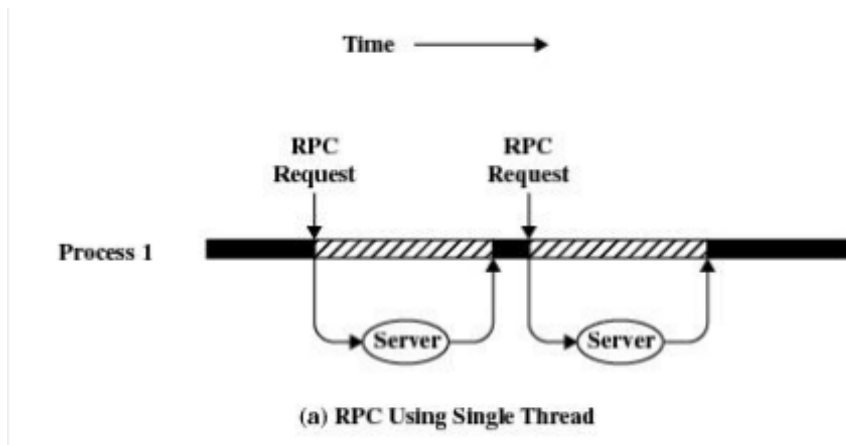
4.2.1 Stati e Operazioni dei Thread

I principali stati di un thread sono tipicamente *running*, *ready* e *blocked*. Le operazioni invece sono:

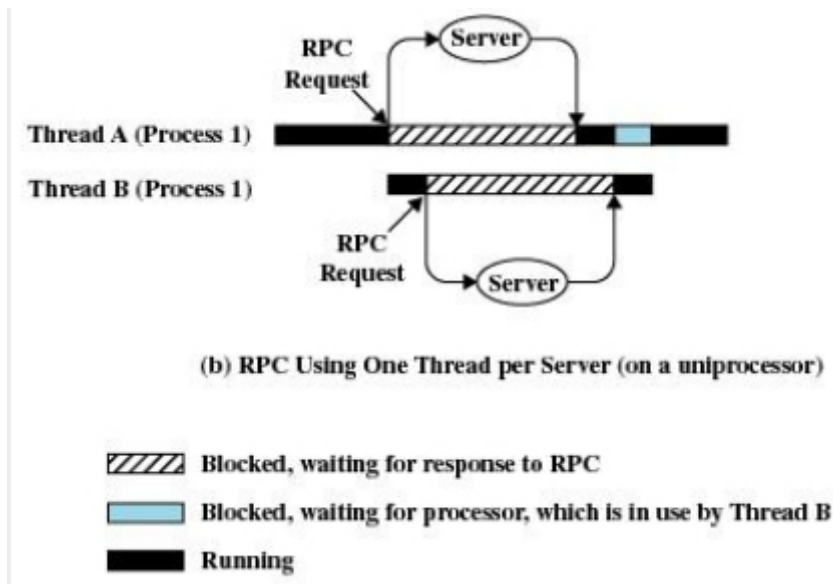
- **Spawn** creazione di un thread, di solito alla creazione di un processo viene associata quella di un thread. In seguito quest'ultimo può generare thread fratelli.
- **Block**: il thread si mette in attesa di un evento.
- **Unblock**: l'evento atteso ha luogo e il thread viene sbloccato e posto in stato ready.
- **Finish**: il thread termina e il suo contesto (registri, program counter, stack) viene deallocato.

4.2.2 Esempio di Programmazione con Thread

Invocazione di due RPC su due siti diversi, La soluzione con un unico thread: un processo composto da un thread e una CPU



Invocazione di due RPC su due siti diversi. La soluzione con due thread: un processo composto da due thread e una CPU



4.2.3 Sincronizzazione tra Thread

Tutti i thread di un processo condividono lo stesso spazio di indirizzamento e, in generale tutte le risorse allocate dal processo. Per evitare interferenze tra i thread é necessario impiegare meccanismi di sync (es. mutual exclusion negli accessi in scrittura)

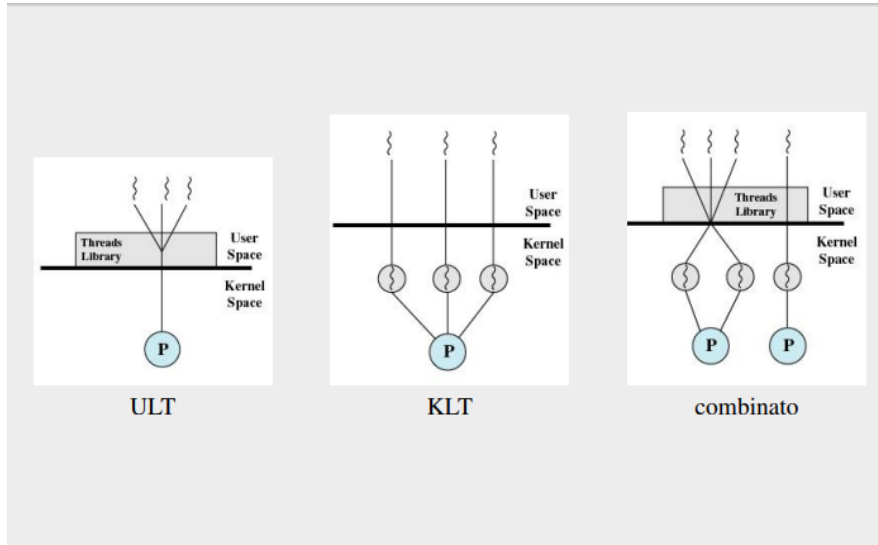
Le tecniche utilizzate sono essenzialmente le stesse applicate a livello dei processi.

4.2.4 User-level Thread e Kernel-level Thread

Ci sono due tipologie di thread:

- **User-level thread:** (ULT), tutta la gestione si svolge a livello di applicazione e il kernel ignora l'esistenza dei thread.
- **Kernel-level thread;** (KLT), tutta la gestione é fatta dal kernel (Windows).

Esistono sistemi che combinano le tipologie Ex: Solaris.



User-level Threads Una applicazione di questo tipo può essere sviluppata utilizzando opportune *librerie* che implementano le primitive per la gestione quali: creazione, terminazione, sincronizzazione, comunicazione (etc) dei thread (lvl utente).

- una applicazione inizia ad eseguire con un singolo thread che poi crea altri thread, per ognuno di essi é necessario fornire un codice da eseguire (in C una funzione).
- tutti i thread sono allocati ad un singolo processo gestito dal kernel.
- la libreria crea le strutture dati necessarie per realizzare lo scheduling dei threads (internamente al processo).

Lo scheduling è a **livello di processo**. Affiancato dallo scheduling "interno all' applicativo" supportato dalla libreria. C'è convivenza tra lo stato del processo e quello dei thread a lui collegati, ma *solo uno* di questi può essere running.

4.2.5 Vantaggi e Svantaggi di ULT

Rispetto a KLT si possono considerare questi **vantaggi**:

- lo switching tra thread non richiede l'intervento del Kernel né il passaggio dalla modalità utente a quella kernel (e viceversa).
- lo scheduling può essere specificamente progettato/implementato per la specifica applicazione indipendente dalle politiche utilizzate dal kernel nel CPU-scheduling.
- vi è maggiore portabilità, infatti basta solo implementare le librerie

Un approccio ULT porta questi maggiori **svantaggi**:

- spesso molte system call sono bloccanti (il chiamante attende il risultato della chiamata), quindi se un ULT ne invoca una tutto il processo passa in waiting (possibile soluzione: *jacketing*).
- dato che solo un processo corrisponde al gruppo di ULT di un' applicazione non si può sfruttare la presenza di multiprocessori.

Kernel-level threads La gestione avviene **interamente a livello di kernel**. Esso gestisce sia l'exec del processo che quello di tutti i thread.

- lo scheduling avviene sui thread e la competizione può essere ristretta ai thread dello stesso processo o di più processi.
- il blocco di un thread non implica il blocco dei fratelli.

- un approccio KLT permette di realizzare il kernel stesso usando i thread.

4.2.6 Svantaggi KLT

Il principale svantaggio di KLT é il maggior costo dello switch tra i threads, anche tra fratelli infatti comporta un passaggio alla modalit kernel. La tabella sottoriportata compara le performance di ULT e KLT su due operazioni:

- *Null Fork*: creazione, scheduling, exec di "null", terminazione. (Essenzialmente il tempo di creazione);
- *Signal-Wait*: signal e attesa condizione. E' il tempo impiegato dall'operazione di Sync tra due thread/processi

Operazione	ULT	KLT	Processi
Null fork	34	948	11300
Signal-wait	37	441	1840

4.2.7 Approcci combinati?

Alcuni OS combinano i due lv di thread, ad esempio alcune versioni di Solaris, in questo caso:

- I threads sono creati a lvl utente.
- Gli ULT fratelli in un processo sono **mappati** in numero inferiore o uguale ai KLT.
- Vi è massimo vantaggio qualora il sistema sia multiprocessore: più i thread fratelli possono eseguire in concorrenza tra loro che con altri di altri processi.

4.2.8 Problematiche nella gestione dei Thread

In presenza di più flussi di exec nello stesso processo si devono affrontare alcuni problemi supplementari, Ex:

- Definire il significato di una "fork()" effettuata da un thread:
 - il nuovo processo ha tutti i thread del processo padre?
 - ha un solo thread, copia del thread chiamante?
- Definire l'effetto dell'exec(), normalmente tutto il processo viene sostituito.
- Implementare la *cancellazione* di un thread: operazione compiuta da un altro thread o processo che lo interrompe (che abbia finito o meno).
Questa può avvenire in maniera **asincrona** oppure **differita**; ogni thread controlla in determinati istanti se è stato cancellato.
- Occorre stabilire come avviene la gestione dei **signal**:
 - a quale thread viene notificato?
 - quale invece lo gestisce?

5 Scheduling

5.1 Scheduling dei Processi

5.1.1 Concetti Generali

In generale risolvere un problema di scheduling consiste nel **mappare** un certo numero di **compiti** su un (diverso) numero di **esecutori** in modo da realizzare obiettivi globali del sistema rispettando **vincoli** prefissati.

I principali **eventi** che possono causare uno **switch di contesto** (e che fanno quindi entrare in gioco lo scheduler) sono:

- interruzione relativa a timer (termine del quanto di tempo)
- interruzione dovuta a dispositivi I/O
- invocazione di system call (es. exit())
- gestione di segnali

5.1.2 Compiti del Dispatcher

Lo scheduler sceglie il processo da eseguire, successivamente il dispatcher:

- effettua il context switch
- inizializza program counter alla prossima istruzione del programma da (continuare a) eseguire
- effettua il passaggio alla modalità utente

Il tempo per fare ciò viene detto latenza di dispatch.

5.1.3 Parametri Generali

- numero di processori (1+), omogeneità/disomogeneità dei processori (in tipo, velocità, ISA...)
- staticità/dinamicità insieme dei task
- on-line e off-line scheduling
- tempi di esecuzione noti a priori o sconosciuti
- presenza o meno di preemption (prelazione)
- presenza o meno di precedenza tra task
- presenza o meno di priorità
- imposizione o meno di scadenze (deadline) su inizio/terminazione task
- presenza o meno di **obiettivo** da ottimizzare (tempo di risposta, tempo di completamento, produttività...)

5.1.4 Obiettivi

Obiettivi diversi, spesso antitetici, sono ottimizzazione di:

- lunghezza dello schedule
- **tempo di risposta** massimo
- **tempo di completamento** medio
- tempo di risposta medio
- tempo di attesa
- numero minimo di CPU utilizzate
- produttività (num. task/tempo → **throughput**)
- utilizzo della CPU
- massimo ritardo rispetto a eventuali deadline

5.1.5 Criteri per CPU Scheduling

Dal punto di vista **utente**:

- minimizzare tempo risposta
- minimizzare tempo completamento
- rispettare deadline

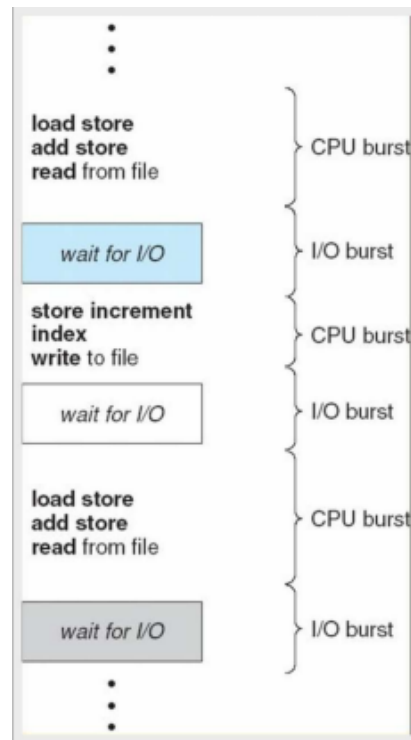
Dal punto di vista **sistema**:

- massimizzare utilizzo CPU
- garantire **fairness** (problema della **starvation**)
- massimizzare throughput
- riflettere le priorità

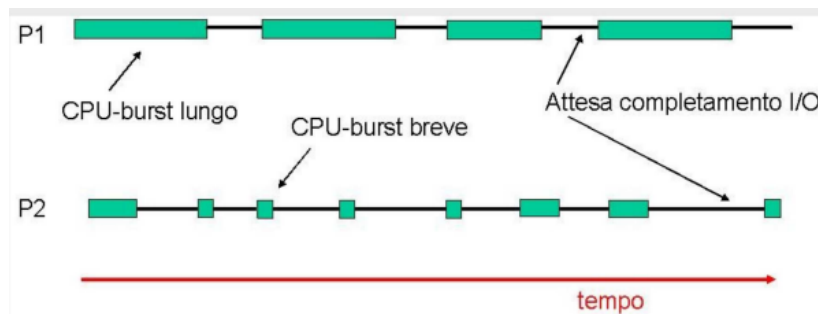
Sia scheduler che dispatcher rappresentano bottleneck per il S.O.

5.1.6 CPU-burst e I/O-burst

Il comportamento tipico di un processo consiste nell'alternare sequenze di operazioni di elaborazione (CPU-burst) a sequenze di operazioni di I/O (I/O burst), terminando con un CPU-burst.



Processi CPU-bound avranno pochi CPU-burst lunghi (P_1), mentre processi I/O-bound avranno molti CPU-burst brevi (P_2):



5.1.7 Scheduling Preemptive e Non-preemptive

Lo schema di scheduling può essere con o senza **diritto di prelazione**:

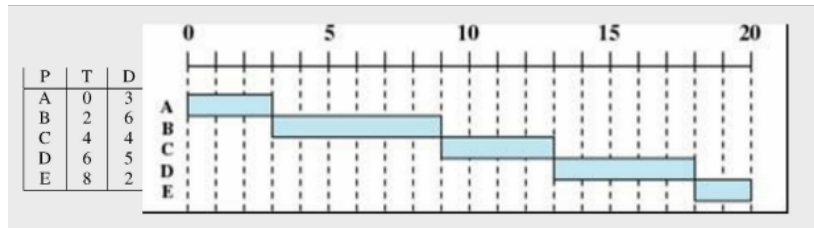
- **Nonpreemptive scheduling**: processo running continua a detenere la CPU fino a quando termina o la cede
- **Preemptive scheduling**: processo può essere interrotto (e portato in stato di ready) a seguito di un evento "esterno" (scade quanto di tempo, altro processo diventa ready...)

Nonpreemptive scheduling tipico dei sistemi batch, preemptive scheduling evita che processi "meno importanti" ostacolino quelli critici (comportando però un overhead di gestione)

5.2 Algoritmi di Scheduling Processi

5.2.1 FCFS - First Come First Served

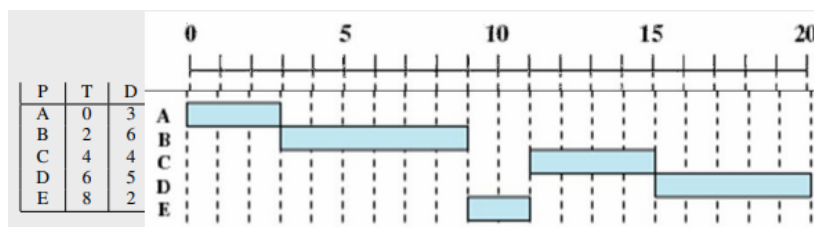
- Si seleziona il processo che è nella ready-queue da più tempo (processi serviti nell'ordine in cui entrano nella ready-queue)
- Nonpreemptive



- Vantaggi
 - Semplice da implementare, non comporta overhead di gestione
 - Non ci può essere starvation in quanto tutti i processi ready, prima o poi, ottengono uso CPU
 - Svantaggi
 - Tempo d'attesa medio molto legato all'ordine di arrivo dei burst
 - Tempo di risposta di un processo può essere molto alto anche per processi di breve durata
 - Un processo CPU-bound tende a monopolizzare il processore a discapito dei processi I/O-bound
 1. Processi I/O-bound terminano burst di I/O e si accodano in ready
 2. CPU-bound rilascia la CPU per fare I/O
 3. Tutti gli I/O-bound terminano il burst di CPU per fare I/O
 4. Il CPU-bound termina l'I/O e torna ready ("prima" degli I/O-bound) e tutto si ripete
- Si tende quindi a sottoutilizzare sia device che CPU (convoy effect)

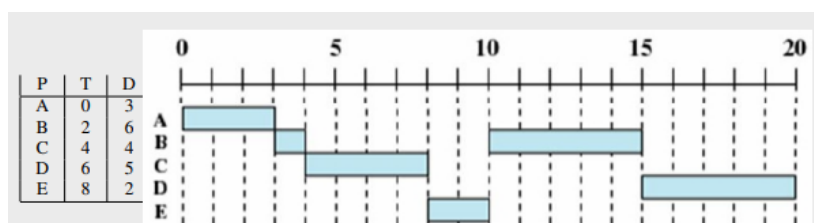
5.2.2 SJF - Shortest Job First

- Si seleziona il processo che avrà il prossimo CPU-burst più breve
- Nonpreemptive



5.2.3 SRTF - Shortest Remaining Time First

- Si seleziona il processo che avrà il prossimo CPU-burst più breve
- Se P_1 è running e in ready-queue giunge P_2 con durata inferiore al tempo restante di P_1 , allora P_2 sottrae la CPU a P_1
- Preemptive



5.2.4 Caratteristiche SJF / SRTF

- Processi con CPU-burst più breve sopravanzano gli altri
- SJF rappresenta lo scheduling ottimale: realizza il tempo di attesa medio migliore
- Può esserci starvation: i processi con CPU-burst lunghi tendono a restare molto tempo in ready-queue, non garantito ricevano l'uso della CPU in un tempo limitato

- Richiedono di prevedere la durata dei CPU-burst (non sempre possibile determinare queste durate)

5.2.5 Stima durata CPU burst

La previsione/stima della durata del prossimo CPU-burst di un processo può essere compiuta basandosi sui suoi CPU-burst del passato tramite media esponenziale:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$$

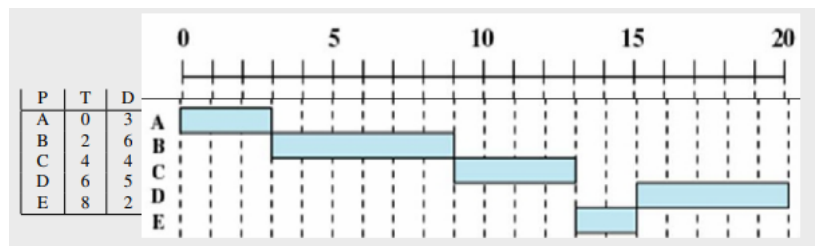
dove t_n è la durata dell'n-esimo CPU-burst cioè la storia recente, mentre τ_n rappresenta il contributo della storia remota. Sviluppando il termine n-esimo si ottiene:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\alpha t_{n-1} + (1 - \alpha)^2 \alpha t_{n-2} + \dots + (1 - \alpha)^{n+1} \tau_0$$

con τ_0 e $0 \leq \alpha \leq 1$ costanti predefinite. In particolare α permette di modulare il peso della storia recente/remota.

5.2.6 HRRF - Highest Response Ratio First

- Si seleziona il processo che ha il **Response Ratio** R maggiore, dove $R = \frac{t_a + t_s}{t_s}$ con t_a il tempo che il processo ha speso in attesa della CPU e t_s la durata prevista del suo CPU-burst
- Nonpreemptive



- Processi con CPU-burst brevi sono favoriti ma si tiene conto dell'età del processo, processi con tempo di servizio t_s lungo guadagnano priorità man mano che invecchiano
- Meccanismo di **aging**: evita la starvation
- Come SJF e SRTF richiede stime su tempo di servizio richiesto da processi

5.2.7 Scheduling con Priorità

Viene assegnata una priorità ad ogni processo e CPU viene allocata a processo ready con priorità superiore. Nonpreemptive o preemptive (in questo caso un processo ready con priorità superiore sottrae la CPU ad un processo running di priorità inferiore)

- Gestione delle priorità
 - Priorità statiche
 - * priorità assegnata ad un processo resta costante
 - * facile implementazione
 - * minimo overhead di gestione
 - * no adattamento a cambiamenti del carico di sistema
 - Priorità dinamiche
 - * reattività rispetto a mutazioni del carico di sistema
 - * permettono di “seguire” evoluzione del comportamento dei processi
 - * maggior overhead per gestione
 - * aumento dinamico (solitamente) della priorità ai processi I/O-bound
- Priorità e Starvation Starvation = processo P, pur essendo ready, non ottiene mai uso della CPU. In una politica con priorità potrebbe verificarsi se venissero continuamente creati processi con priorità superiore a quella di P. Per evitare starvation si usa una strategia di aging dei processi (essi tendono a migliorare la loro priorità con il passare del tempo)

5.2.8 Round Robin (circolare)

Come per FCFS si serve il processo che attende da più tempo

Preemptive usando timer: allo scadere del quale il processo running viene rimesso in ready-queue

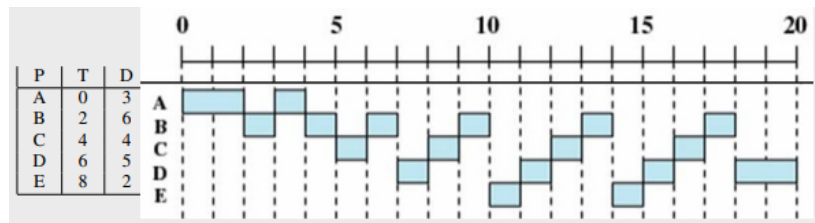


Figure 11:

Processi I/O-bound penalizzati (in generale hanno CPU-burst più brevi del quanto di tempo), burst termina prima della fine del quanto (vengono messi in waiting prima di aver sfruttato tutto tempo a disposizione)

Processi CPU-bound tendono a esaurire il quanto, quindi anche se mix tra CPU e I/O bound è adeguato in media i CPU-bound sfruttano (tendenzialmente) di più la CPU. Vantaggio: evita la starvation

5.2.9 Durata quanto di tempo

Durata di un quanto è critica per il comportamento del sistema:

- deve essere sensibilmente superiore al tempo necessario per gestire interruzioni / switch di contesto
- si rilevano buoni risultati se la durata del quanto è maggiore della durata di almeno 80% di CPU-burst
- non può essere troppo lungo → penalizza processi I/O-bound (tende ad un FCFS)

5.2.10 Scheduling con Code Multiple

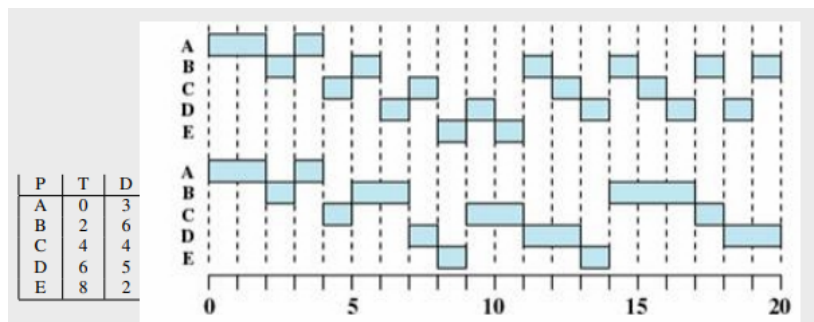
Più code di processi:

- processi partizionati in gruppi disgiunti secondo opportuni criteri (es. processi batch e processi interattivi)
- processi assegnati permanentemente alle code
- ogni coda gestita mediante algoritmo di scheduling più adatto a caratteristiche dei suoi processi
- si deve stabilire uno scheduling di code:
 - relazione di priorità tra code e strategia preemptive: processo di una coda viene interrotto appena un altro processo entra in una delle code con priorità maggiore
 - **time slicing** a livello delle code: ad ogni coda viene assegnata una percentuale di tempo totale di CPU

5.2.11 Scheduling con Code Multiple -Retroazione

Evoluzione delle code multiple:

- **Evita starvation**, permettendo ai processi di migrare tra code secondo criteri di **aging**.
- Solitamente **preemptive**.
- I processi che utilizzano a lungo la CPU vengono spostati in code con **priorità più bassa**.
- I processi **interattivi (I/O-bound)** tendono invece a ricevere **priorità più alta**.
- Un processo che resta troppo tempo in una coda a bassa priorità può essere **promosso** a una coda con priorità maggiore (per evitare starvation).
- **Overhead di gestione** viene bilanciato da una maggiore **sensibilità e reattività del sistema**.



Parametri da considerare:

- Numero code da utilizzare
- Algoritmi di scheduling per ogni coda.
- **Regole di migrazione** dei processi tra le code (es. un processo preempted può “retrocedere”, mentre l’**aging** lo fa salire di priorità).
- **Posizione iniziale** nuovi processi (es. si può scegliere di inserirli direttamente nella coda con priorità più alta).

5.2.12 Comparazione tra algoritmi

	Selection Function	Decision Mode	Throughput	Response Time	Overhead	Effect on Processes	Starvation
FCFS	$\max[w]$	Nonpreemptive	Not emphasized	May be high, especially if there is a large variance in process execution times	Minimum	Penalizes short processes; penalizes I/O bound processes	No
Round Robin	constant	Preemptive (at time quantum)	May be low if quantum is too small	Provides good response time for short processes	Minimum	Fair treatment	No
SPN	$\min[s]$	Nonpreemptive	High	Provides good response time for short processes	Can be high	Penalizes long processes	Possible
SRT	$\min[s - e]$	Preemptive (at arrival)	High	Provides good response time	Can be high	Penalizes long processes	Possible
HRRN	$\max\left(\frac{w+s}{s}\right)$	Nonpreemptive	High	Provides good response time	Can be high	Good balance	No
Feedback	(see text)	Preemptive (at time quantum)	Not emphasized	Not emphasized	Can be high	May favor I/O bound processes	Possible

5.2.13 Fair Share Scheduling

I processi vengono **partizionati in gruppi** e lo scheduling avviene mirando a **equità di trattamento rispetto ai gruppi**:

- adatto a sistemi multi utente
- adatto in presenza di applicazioni multi-thread e scheduling viene effettuato sui thread
- può trattare diversamente processi di differenti tipologie di utente

5.3 Scheduling dei Thread

5.3.1 Kernel-Thread e User-Thread

- User Threads (UT)
 - Gestiti nello spazio utente (user space).
 - **Il kernel non ne è consapevole.**
 - Più veloci da creare e gestire.
 - **Problema:** se un thread effettua una system call bloccante, blocca tutti i thread del processo

– PROCESS-CONTENTION SCOPE

- Kernel Threads (KT)
 - Gestiti dal kernel, nello spazio kernel.
 - Ogni thread è schedulato **indipendentemente**.
 - Più costosi da gestire in termini di performance.
- **SYSTEM-CONTENTION SCOPE**

5.4 Scheduling per Sistemi Multiprocessore

5.4.1 Scheduling in Sistemi Multiprocessore

In linea di principio possono essere adattate le tecniche di scheduling per i casi single-processor anche in presenza di più unità di elaborazione.

Il carico deve però essere distribuito e **bilanciato**. **Tipologie di multielaborazione**

- **Asimmetrica**: attività di sistema (scheduling, gestione, interruzioni...) affidate tutte a un unico processore **master server**
 - codice degli utenti eseguito dagli altri processori
 - **difetto**: master server è un potenziale bottleneck del sistema
- **Simmetrica**: ogni processore ha uno scheduler che attinge alla ready-queue che può essere
 - di sistema: possibili race condition se più processori scelgono stesso task (serve mutua esclusione all'interno di ready-queue → possibile bottleneck)
 - per-processor: serve un meccanismo per distribuzione/bilanciamento carico tra ready-queue

Scheduling (Multi)processori Multicore

- Processori multicore presentano più core in stesso chip fisico
- S.O. vede ogni core come un processore logico separato
- Ogni core esegue (solitamente) più thread hardware: **chip multithreading, hyper-threading**
- Presenza di multithreading a livello di core comporta doppio livello di scheduling:
 - scheduling (hardware) di thread hardware sui core della CPU multicore
 - scheduling (software) di thread/processi/task sulle CPU (lo scheduling studiato finora)

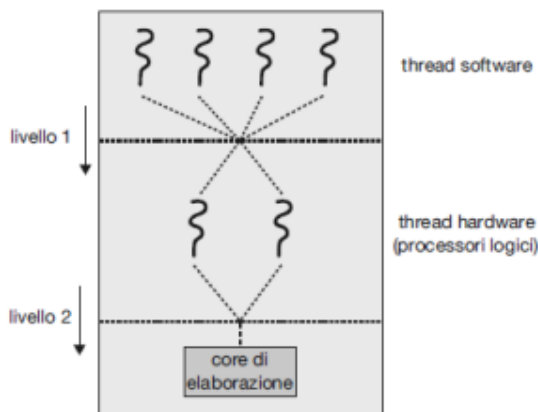


Figura 5.15 Due livelli di scheduling.

5.4.2 Bilanciamento del Carico

Con più unità di elaborazione va massimizzato distribuendo equamente il carico tra i core del sistema, si utilizzano principalmente due tecniche base (non esclusive):

- Push migration: un processo dedicato controlla periodicamente carico di lavoro assegnato a predecessori → sottrae task da CPU overloaded e li riassegna a CPU meno cariche
- Pull migration: ogni processore, quando inattivo, preleva un task da uno dei processori overloaded

5.4.3 Sistemi Eterogenei

Presenza di diversi processori con caratteristiche non omogenee:

- diverso tipo (processori grafici / dedicati a task specifici)
- diverso set di istruzioni
- diversa potenza di calcolo / consumo energetico

Utile in caso si debba dare maggiore importanza a consumo energetico (sistemi embedded...)

5.5 Scheduling Real-time

5.5.1 Sistemi Real-Time

- Utilizzati in dispositivi di largo consumo (sistemi embedded, player, robotica, domotica)
- Funzionalità devono rispettare vincoli temporali (scadenze, durate, tempi di reazione...)
- Obiettivo primario rispetto di deadline e vincoli ai tempi di reazione
- Necessitano algoritmi di scheduling specifici

Tipologie di Sistemi Real-Time:

- Real-time soft: ai task critici viene concessa precedenza (no violazioni deadline però)
- Real-time hard: task devono necessariamente rispettare deadline

Solitamente sono event-driven (all'accadere di un evento il sistema reagisce nel minor tempo possibile ed esegue il task richiesto → tempo di reazione detto latenza dell'evento)

- latenza di interruzione: tempo trascorso tra la ricezione dell'interruzione e l'inizio della procedura di gestione
- latenza di dispatch: tempo impiegato dal dispatcher per eseguire lo switch di contesto

Scheduling Sistemi Real-Time

- **Priorità:** scheduling con priorità con prelazione visto in precedenza o varianti per garantire deadline hard e gestire task periodici
- **Priorità proporzionale a frequenza:** come precedente ma con priorità assegnate staticamente in base a frequenza del task periodico
- **Earliest Deadline First:** priorità assegnate dinamicamente (prima il task che ha scadenza più vicina, non richiede periodicità)
- **A quote proporzionali:** assegnata una percentuale del tempo di CPU a ogni applicazione, a ogni richiesta di utilizzo CPU si valuta se ammetterla o meno

5.6 Scheduling in UNIX e Linux

5.6.1 Scheduling di UNIX Tradizionale

- Utilizza **code multiple con priorità dinamiche** e algoritmo **round-robin** per ogni coda.
- **Preemption** dopo 1 secondo di utilizzo continuato della CPU.
- Ogni processo ha una **priorità di base** che lo colloca in una banda specifica (es. Swapper, User processes) per ottimizzare l'uso dei dispositivi I/O.
- La priorità viene modificata in base all'utilizzo della CPU e alle preferenze dell'utente (nice), ma rimane nella stessa banda.
- Le priorità sono ricalcolate ogni secondo.

5.6.2 Scheduling in O(1) di Linux

- Con **preemption**.
- Basato su **priorità numeriche** con due gamme:
 - Priorità alta (es. 0) → quanto lungo (200 ms).
 - Priorità bassa (es. 140) → quanto breve (10 ms).

Gestione dei Task

- **Task ready**: classificati come **active** (se hanno tempo residuo) o **expired** (se hanno esaurito il quanto).
- Il task **running** diventa **expired** quando consuma il suo quanto e non può essere rieseguito finché ci sono task active.
- Due array gestiscono task active e expired. La CPU è assegnata al task active con priorità massima.
- Gli array si scambiano i ruoli quando tutti i task **active** diventano **expired**.

6 Sincronizzazione

6.1 Il problema della Sezione Critica

6.1.1 Esecuzione Concorrente

Quando più processi accedono contemporaneamente a dati condivisi, possono sorgere problemi di *incoerenza* ed *inconsistenza* dei dati.

Questo accade perchè un **context switch** può verificarsi prima che un processo abbia completato tutte le sue modifiche sui dati.

Per affrontare questa problematica, il sistema operativo deve fornire strumenti specifici, il cui obiettivo è:

- Garantire l'accesso in mutua esclusione ai dati condivisi [serializzazione degli accessi]
- Assicurare che il tempo di attesa per i processi sia ragionevole e che venga applicata *fairness*
- Evitare attese infinite [processi che si attendono a vicenda]

6.1.2 Concetti Essenziali

Sezione Critica [SC] : Una sezione di codice che legge/scrive dati condivisi

Race Condition: Una situazione in cui l'esecuzione concorrente di una sezione critica, da parte di più processi, porta a risultati *dependenti* dall'ordine in cui i processi alternano l'esecuzione delle loro istruzioni

Mutua Esclusione: Meccanismo con cui si evitano le race condition assicurando l'accesso esclusivo ai dati. [serve una autorizzazione per entrare nella sezione critica]

Deadlock: Situazione in cui dei processi sono permanentemente bloccati perchè in attesa vicendevole del rilascio di risorse

Starvation: Situazione in cui ad un processo che ha richiesto l'accesso ad una sezione critica non viene mai permesso perchè vengono servite le richieste di altri processi

Atomico: esecuzione di un'azione non interrompibile

6.1.3 Possibile soluzione al problema della SC

La possibile soluzione deve garantire:

- **Mutua Esclusione**: al più un solo processo nella SC
- **Progresso**: la decisione riguardante l'ingresso in SC deve essere completata entro un tempo limite finito e non può essere influenzata da processi che stanno eseguendo la loro sezione non critica
- **Attesa limitata**: se P_1 chiede di accedere alla sua SC, allora può esserci solo un numero finito di processi che entrano nella SC prima che P_1 possa entrare [no Starvation]

La soluzione consiste dunque nel progettare un **protocollo** che assicuri effetti globali indipendenti dall'ordine di esecuzione delle azioni dei processi.

Uno schema generale per il protocollo può essere:

- SC racchiusa tra due *porzioni di codice* dette **sezione d'ingresso** e **sezione d'uscita**
- Per accedere alla SC si esegue il codice della *sezione d'ingresso*
- Per abbandonare la SC si esegue il codice della *sezione d'uscita*

6.1.4 Soluzione di Peterson

```
boolean flag[2];
int turno;
do {
    // ... sezione non critica
    flag[i] = true;    //dichiara il suo interesse
    turno = j;        //assume che sia il turno dell'altro e gli offre l'occasione di entrare
    while (flag[j] && turno == j) {
        // ... attesa del proprio turno
        // ... sezione non critica
        flag[i] = false; // fine dell'interesse
        // ... sezione non critica
    }
} while (1);
```

Funziona:

- *turno* garantisce mutua esclusione, elimina la possibilità di deadlock ed evita che un processo monopolizzi la SC
- l'uso dei *flag* evita che un processo rimanga chiuso fuori perchè l'altro non rientri in SC
- permette ad un processo di accedere la SC più volte di seguito

Difetto: *busywaiting*

6.1.5 Soluzioni basate sugli Interrupt

Questo tipo di soluzione si basa sul disabilitare gli interrupt prima di entrare in una SC per poi riabilitarli una volta uscito [possibili **deadlock**].

I possibili difetti di questa soluzione si suddividono in base al tipo di sistema:

- **mono-processore:** *mutua esclusione* garantita ma efficienza degradata [perchè i context switch sono inibiti], atomicità SC
- **multi-processore:** *mutua esclusione* NON garantita, SC atomica ma non esclusiva

6.1.6 Soluzione con Supporto Hardware

Una soluzione a livello Hardware si può ottenere in presenza di **particolari istruzioni macchina** che:

- disciplinano gli accessi concorrenti in memoria [es. barriere di memoria]
- eseguono multiple azioni in modo unitario ed atomico sulla stessa locazione di memoria
 - esempi: test+assegnamento, read+modifica+write, swap, test+swap,...
 - l'esecuzione di queste istruzioni sono in mutua esclusione anche in sistemi multi-processore

6.1.7 Memory Barriers

sono istruzioni che regolamentano load e store della memoria (in presenza di sistemi multi-processore le modifiche effettuate da un processore potrebbero non essere immediatamente visibili agli altri processori)

```
Th_1
while (!flag) {
    memory_barrier();
}
y = x;
```

```
Th_2
x = 100;
memory_barrier();
flag = true;
}
```

Ci si assicura che Th_1 legga il valore di *flag* prima di quello di *x*. La modifica di *x* fatta da Th_2 sarà resa disponibile a Th_1 prima che Th_2 modifichi *flag*.

6.1.8 Test and Set: istruzione atomica

```
boolean TestAndSet (boolean *target) {
    boolean retval = *target;
    *target = TRUE;
    return *retval;
}
```

Difetti:

- busy waiting
- non garantisce attesa limitata

```

boolean lock;      // condivisa e inizializzata a false

do {
    // ... sezione non critica
    while (TestAndSet(&lock)) {
        // attesa del proprio turno
        // ... sezione critica [chiave false]
    }
    lock = false; //uscita dalla SC
} while(1)

```

6.1.9 Swap: istruzione atomica

```

void Swap (boolean *a, boolean *b) {
    boolean temp = *a;
    *a = *b;
    *b = temp;
}

```

Difetti:

- busy waiting
- non garantisce attesa limitata

```

boolean lock;      // condivisa e inizializzata a false
// lock==true significa porta chiusa
boolean chiave;    // privata e locale al processo

do {
    // ... sezione non critica
    chiave = true;
    while (chiave == true) {
        Swap(&lock, &chiave);
        // ... sezione critica [chiave false]
    }
    lock = false; //uscita dalla SC
} while(1)

```

6.1.10 CompareAndSwap: istruzione atomica

```

void CompareAndSwap (int *val, int test, int newval) {
    int temp = *val;
    if (*val == test) {
        *val = newval;
    }
    return temp;
}

```

Garantisce:

- mutua esclusione
- progresso
- attesa limitata

Difetti:

- busy waiting

```

int lock;          // condivisa e settata a 0 [open]
boolean waiting[N]; // array condiviso settato a false
while (true) {
    // ... sezione non critica
    waiting[i] = true;
    key = 1;
    while (waiting[i] && key == 1) { // attesa turno
        key = CompareAndSwap(&lock, 0, 1);
    }
    waiting[i] = false;
    // ... sezione critica
    j = (i + 1)%N;
    while ((j != 1) && !waiting[j]) {
        j = (j + 1)%N;
    }
    if (j == i) {
        lock = 0;
    } else {
        waiting[j] = false;
    }
    // ... sezione non critica
}

```

Variabili Atomiche Le istruzioni sopra descritte possono essere impiegate per l'implementazione di operazioni (aritmetiche, booleane) su locazioni di memoria per poterle eseguire **atomicamente**

Evitare il Busy Waiting Una soluzione ottimale dovrebbe evitare lunghe permanenze di processi in attesa della SC in CPU. [introduzione dello stato di **waiting** e, dunque del ***semaforo***].

- i processi *waiting* non occupano CPU
- tornerebbero ready solo quando hanno la possibilità di entrare nella SC

6.2 Semafori

Semaforo - Definizione: struttura dati costituita da:

- int S
- operazioni atomiche predefinite:
 - *wait(S)*
 - *signal(S)*
- operazioni di creazione ed inizializzazione

In base ai valori che può assumere S si distingue tra:

- **semaforo binario**
- **semaforo generico**

```
wait(S) {  
    while ( S <= 0 ) {  
        // busy wait  
    }  
    S--;  
}
```

```
signal(S) {  
    S++;  
  
}
```

6.2.1 Semafori di MutEx e di Sincronizzazione

Utilizzando un semaforo *binario* S si può regolamentare l'accesso ad una sezione critica intervallandola ad un **wait(S)** ed un **signal(S)**.

Se inizializzato ad '1' il semaforo si dice **MutEx** (**mutual exclusion**).

Se inizializzato a '0' il semaforo è di sincronizzazione.

6.2.2 Spinlock e Busy Waiting

Se l'operazione wait del semaforo utilizza busy waiting, il semaforo si dice *spinlock*:

- vantaggiosi se l'attesa è breve [il wait(S) non implica un *context switch*]
- può essere utile nei sistemi multi-processore [il processo che esegue la wait(S) attende una signal(S) da un processo in SC su un'altra CPU]
- con attese lunghe conviene usare **context switch**

6.2.3 Implementazione dei Semafori senza Busy Waiting

L'idea risolutiva è la seguente:

- se un processo P esegue wait(S) e $S \leq 0$, P viene inserito in una *coda di attesa associata al semaforo S* e portato in stato di *waiting*
- il controllo passa allo *short-term scheduler*
- quando viene eseguita una signal(S) **uno** dei processi in coda viene svegliato e messo ready

La struttura di un semaforo senza BW è la seguente:

```
typedef struct {  
    int valore;  
    // valore  
    struct processo *lista;  
    // processo = coda di attesa  
} semaforo
```

```
wait(semaforo *S) {  
    S → valore--  
    if (S → valore < 0) {  
        // add P to S → lista;  
        sleep();  
    }  
}
```

```
signal(semaforo *S) {  
    S → valore++;  
    if (S → valore ≤ 0) {  
        // take P from S → lista;  
        wakeup(P);  
    }  
}
```

Per l'implementazione si usano le seguenti operazioni elementari (system call fornite dall'OS):

- **sleep()**: mette in *waiting* il processo che lo ha invocato
- **wakeup(S)**: porta P in *ready*

Si può notare che:

- non è detto che *lista* venga gestita con politica FIFO
- *lista* può essere implementato inserendo dei puntatori nel PCB
- il valore del semaforo può essere negativo [indica il numero di processi in attesa]

6.2.4 Atomicità di wait e signal

Per fare in modo che queste due operazioni siano atomiche, esse vanno considerate come **sezioni critiche**. Abbiamo diverse soluzioni in base al tipo di sistema:

- **monoprocessore**
 - inibire le interruzioni durante wait(S) e signal(S)
- **multiprocessore**
 - inibire le interruzioni in *tutte* le CPU durante wait(S) e signal(S)
 - usare tecniche che sfruttano TestAndSet, Swap, ... (reintroduce busy waiting ma limitato)
 - usare spinlock (reintroduce busy waiting ma limitato)

6.3 Tipici Problemi di Sincronizzazione

6.3.1 Produttori e Consumatori con Buffer Limitato

Descrizione:

- uno o più processi **produttori** generano dei dati e li inseriscono in un buffer
- uno o più processi **consumatori** prelevano dati dal buffer

Limitazioni:

- ogni dato può essere prelevato solo una volta [chi lo preleva lo consuma]
- al buffer si **deve** accedere in mutua esclusione
- il buffer ha *capacità limitata* ad *n* posizioni
- nessun consumatore può prelevare da un buffer vuoto e nessun produttore può scrivere in un buffer pieno

Una soluzione utilizza tre semafori (mutex e due generici)

```
semaphore piene,      // numero di celle usate [iniz. 0]
vuote,               // numero di celle libere [iniz. MAX]
mutex;               // per la mutua esclusione [iniz. 1]
```

```
do {                  // PRODUTTORE
// ...
produco un dato
// ...
wait(vuote);         // c'\e spazio libero?
wait(mutex);         // entro nella SC
// ...
inserisco il dato nel buffer
// ...
signal(mutex);       // lascio la SC
signal(piene);        // segnale nuovo dato inserito
} while(1);
```

```
do {                  // CONSUMATORE
wait(piene);          // ci sono dati?
wait(mutex);          // entro nella SC
// ...
prelevo un dato dal buffer
// ...
signal(mutex);        // lascio la SC
signal(vuote);         // segnalo new entry
} while(1);
```

6.3.2 Soluzione con Priorità ai Lettori

```
int readcount = 0;    // n lettori attivi
semaphore semW = 1;
semaphore mutex = 1;
semaphore extra = 1;
```

```
void lettore() {
// ...
wait(mutex);
rc++; // readcount
if(rc == 1) {
wait(semW);
}
signal(mutex);
// ... lettura ...
wait(mutex);
rc--;
if(rc == 0) {
signal(semW);
}
signal(mutex);
// ...
}
```

```
void scrittore() {
// ...
wait(extra);
wait(semW);
// ... scrittura
signal(semW);
signal(extra);
// ...
}
```

6.3.3 Soluzione con Priorità agli Scrittori

```
semaphore W = 1;
semaphore R = 1;
semaphore mutex1 = 1;
semaphore mutex2 = 1;
semaphore mutex3 = 1;
int countR = 0;
int countW = 0;
```

```
void lettore() {
    wait(mutex3);
    wait(R);
    wait(mutex1);
    countR++;
    if (countR == 1) {
        wait(W);
    }
    signal(mutex1);
    signal(R);
    signal(mutex3);
    // ... lettura ...
    wait(mutex1);
    countR--;
    if (countR == 0) {
        signal(W);
    }
    signal(mutex1);
}
```

```
void scrittore() {
    wait(mutex2);
    countW++;
    if (countW == 1) {
        wait(R);
    }
    signal(mutex2);
    wait(W);
    // ... scrittura ...
    signal(W);
    wait(mutex2);
    countW--;
    if (countW == 0) {
        signal(R);
    }
    signal(mutex2);
}
```

Osservazioni Le due soluzioni proposte presentano difetti: pur garantendo l'accesso simultaneo ai lettori ed esclusivo agli scrittori, non escludono la starvation.

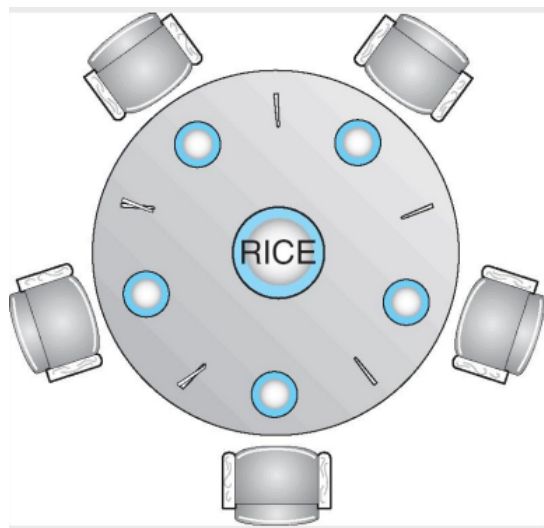
In particolare:

- nella soluzione con priorità ai **lettori** si può avere *starvation* degli scrittori
- nella soluzione con priorità agli **scrittori** si può avere *starvation* dei lettori

6.3.4 Problema dei Cinque Filosofi

Il problema di sincronizzazione può essere formulato così:

- $N = 5$ filosofi e bacchette, $R = 1$ ciotole di riso
- se un filosofo pensa, non interagisce con altri
- se un filosofo ha fame prende 2 bacchette e inizia a mangiare
 - il filosofo può prendere solo le due bacchette ai suoi lati [dx e sx]
 - il filosofo può prendere solo una bacchetta alla volta
 - al filosofo servono entrambe per poter mangiare
- quando un filosofo termina di mangiare appoggia le bacchette



6.3.5 Soluzione Cinque Filosofi con Semafori

Un tentativo di soluzione che utilizza i semafori prevede che vi sia un vettore 'sem[N]' di N semafori mutex (uno per ogni bacchetta)

```
do {
    wait(sem[i]);          // bacchetta sx
    wait(sem[(i + 1) % N]) // bacchetta dx
    // ... mangia ...
    signal(sem[i]);
    signal(sem[(i + 1) % N]) // bacchetta dx
    // ... pensa ...
} while (1);
```

Difetti:

- **non** è deadlock free [si verifica nel caso tutti i filosofi prendessero contemporaneamente una bacchetta]
- **non** è starvation free [non è garantito che tutti riescano a mangiare in un tempo finito]

Rimedi:

- aggiungere un controllo sulla bacchetta dx prima di prelevarla, se non è libera rilascio la sx [dl free ma st possibile]
- stesso controllo ma con un ritardo casuale prima di riprovare a prendere le bacchette [dl free, st meno probabile]
- introdurre un mutex che racchiuda la SC dalla prima *wait()* all'ultimo *signal()* [dl free ma parallelismo nullo, st free se si ha un fair mutex (es. coda FIFO)]
- similmente alla soluzione precedente si può usare un semaforo (inizializzato a 4) si ammette che al massimo 4 filosofi si siedano al tavolo
- soluzione "asimmetrica": un filosofo dispari prende prima la bacchetta sx, un filosofo pari prende prima la bacchetta dx
- soluzione senza asimmetrie e che massimizza il parallelismo:
 - distinguere tre possibili stati per filosofo:
 - * *THINKING*
 - * *HUNGRY*
 - * *EATING*
 - ogni filosofo che voglia mangiare deve controllare lo stato dei vicini prima di prendere le bacchette

Implementazione:

```
RIGHT(i) = ((i + 1) % N)
LEFT(i) = ((i + N - 1) % N)
stato[N] // vettore di stati
semaphore mutex // init:1
semaphore sem[N]; // uno per filosofo (init:0)
```

```
void test(int i) {
    if (stato[i] == HUNGRY &&
        stato[LEFT(i)] != EATING &&
        stato[RIGHT(i)] != EATING) {
        stato[i] = EATING;
        signal(sem[i]);
    }
}
```

```
void filosofo(int process) {
    while(1) {
        // ... pensa ...
        get_sticks(process);
        // ... mangia ...
        put_sticks(process);
    }
}
```

```
void get_sticks(int i) {
    wait(mutex);
    stato[i] = HUNGRY;
    test(i);
    signal(mutex);
    wait(sem[i]);
}
```

```
void put_sticks(int i) {
    wait(mutex);
    stato[i] = THINKING;
    test(LEFT(i));
    test(RIGHT(i));
    signal(mutex);
}
```

6.3.6 Osservazioni Finali sui Semafori

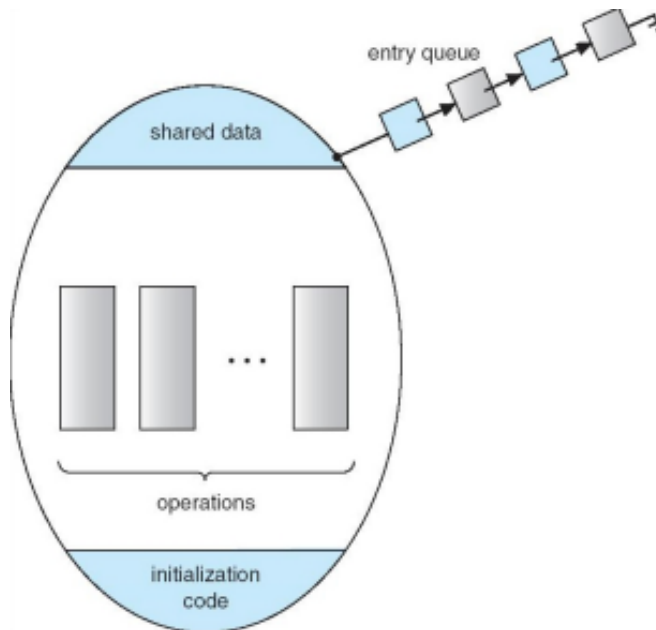
I semafori sono uno strumento semplice per risolvere problemi di sincronizzazione e mutua esclusione, **MA**:

- richiede l'inserimento di *wait(S)* e *signal(S)* in vari punti del codice dei processi
- può essere complicato capire se al runtime si possa verificare *deadlock* o *starvation*
- è sensibile ai piccoli errori

6.4 Monitor

Costrutto di sincronizzazione di alto livello che permette una condivisione sicura di dati fra processi concorrenti [dato astratto che fornisce funzionalità di mutua esclusione]:

- permette di gestire/definire una collezione di dati privati e le procedure per accedervi
- i processi possono invocare solo le procedure (non a dati/variabili interne al monitor)
- **UN** solo processo alla volta può eseguire codice del monitor
- il programmatore raccoglie in un monitor i dati condivisi e tutte le sezioni critiche [assicura *mutua esclusione*]
- è implementato dal compilatore tramite i costrutti per mutua esclusione di basso livello [semafori e shared memory]
- garantisce la mutua esclusione grazie alla *entry queue* che con opportuno scheduling permette di evitare anche la starvation



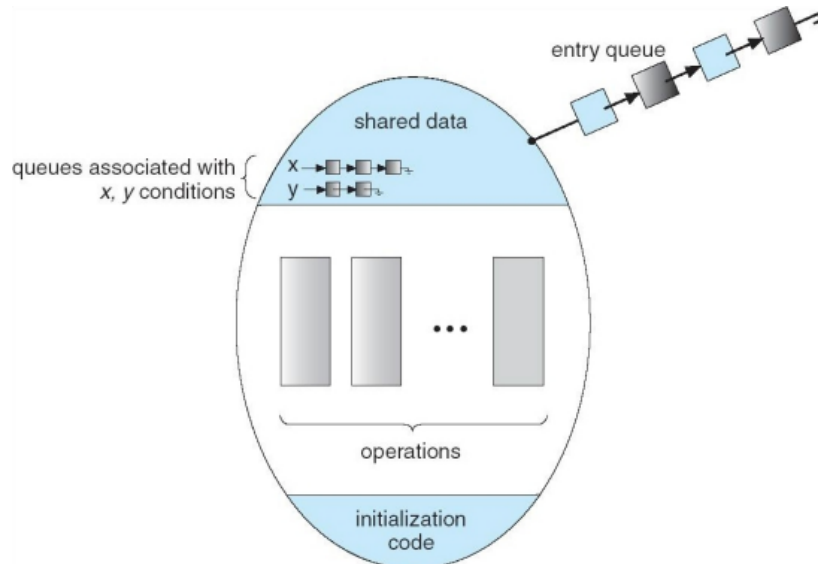
```
monitor example  
integer i;  
condition c;  
  
procedure producer( );  
.  
.  
.  
end;  
  
procedure consumer( );  
.  
.  
.  
end;  
end monitor;
```

6.4.1 Sincronizzazione nei Monitor: Condition Variables

Per permettere la *sincronizzazione* tra processi nel monitor si utilizzano le **condition variables** [locali al monitor]:

condition *x*, *y*;

- permettono ad un processo di attendere il verificarsi di un evento
- su una condition *x* si possono utilizzare due operazioni:
 - ***x.wait()***: il processo chiamante viene sospeso e posto in una coda associata ad *x* fino a che un altro processo non invoca la *signal*
 - ***x.signal()***: attiva esattamente uno tra i processi sospesi in attesa sulla condition *x* (se la coda è vuota non fa nulla)



6.4.2 Risveglio del Processo

Nel caso vi sia P_2 in attesa nella coda di x quando P_1 esegue una *signal()* si possono implementare diversi tipi di "risposta":

- **signal&exit:** P_1 esce e il controllo passa a P_2
- **signal&wait:** P_1 cede il controllo a P_2 e si mette in attesa che P_2 rilasci il monitor o vada in attesa su una condition
- **signal&run:**
 - P_1 continua ad essere attivo fino a che non rilascia il monitor oppure vada in attesa su una condition
 - P_2 non sarà più in coda sulla condition x ma potrà in futuro prendere il controllo del monitor (es. rimesso in coda)

6.4.3 Monitor che invocano altri monitor

Cosa accade se una procedura definita in un monitor M_1 invoca una procedura definita in un altro monitor M_2 ?

- Mantenendo la mutua esclusione su M_1 c'è il pericolo che si verifichi un deadlock tra monitor
- Rilasciando la mutua esclusione su M_1 c'è il pericolo che si perda la consistenza dei dati condivisi a causa di altri processi subentranti in M_1

Soluzione: consiste nel vietare le chiamate nidificate di monitor

6.4.4 Esempi di uso dei Monitor - 5 Filosofi

```
monitor filosofi {
    enum { THINKING, HUNGRY, EATING } stato[5];    // creo array che accetta quei tre valori possibili
    condition self[5];                             // una condition per processo

    void put_sticks (int i) {
        stato[i] = HUNGRY;
        test(i);
        if (stato[i] != EATING) {
            self[i].wait();
        }
    }

    void put_sticks (int i) {
        stato[i] = THINKING;
        test ((i + 4) % 5);    // da una possibilit' a ...
        test ((i + 1) % 5);    // ... ai due vicini
    }

    void test (int i) {
        if ( (stato[(i + 4) % 5]) != EATING &&
            (stato[i] == HUNGRY) &&
            (stato[(i + 1) % 5] != EATING) ) {
            stato[i] = EATING;    // se pu' o passa a EATING
            self[i].signal();
        }
    }
}
```

```

initialization_code() {
    for(int i = 0; i < 5; i++) {
        stato[i] == THINKING;
    }
}

} // end monitor, soluzione deadlock free ma non evita la starvation

```

6.4.5 Esempi di uso dei Monitor - Produttori_Consumatori

```

monitor ProduttoriConsumatori {
    condition full, empty;
    int count;

    void inserisci(item x) {
        if (count == N) {
            full.wait();
            // ... inserisci x nel buffer ...
        }
        count++;
        if (count == 1) {
            empty.signal();
        }
    }

    item preleva() {
        item x;
        if (count == 0) {
            empty.wait();
            // ... rimuovi x dal buffer ...
        }
        count--;
        if (count == N - 1) {
            full.signal();
        }
        return(x);
    }
} // end monitor

```

```

// Uso del Monitor

Produttore() {
    item I;
    while(true) {
        // ... genera un item I ...
        ProduttoriConsumatori.inserisci(I);
    }
}

Consumatore() {
    item I;
    while(true) {
        I = ProduttoriConsumatori.preleva();
        // ... usa item I ...
    }
}

```

6.4.6 Esempi di uso dei Monitor - Lettori_Scrittori

```

monitor LettoriScrittori {
    condition read, write;
    int countR = 0;
    boolean writing = false;

    void beginRead() {
        countR++;
        while (writing) {
            read.wait();
        }
    }

    void endRead() {
        countR--;
        if (countR == 0) {
            write.signal();
        }
    }

    void beginWrite() {
        while ((countR > 0) || (writing)) {
            write.wait();
        }
        writing = true;
    }

    void endWrite() {
        int i;
        writing = false;
        if (countR > 0) {
            for (i = 0; i < countR, i++) {
                read.signal();
            }
        } else {
            write.signal();
        }
    }
} // end monitor

```

```

// Uso del Monitor:

Lettore() {
    while(true) {
        LettoriScrittori.beginRead();
        // ... legge ...
        LettoriScrittori.endRead();
    }
}

Scrittore() {
    while(true) {
        LettoriScrittori.beginWrite();
        // ... scrive ...
        LettoriScrittori.endWrite();
    }
}

```

6.5 IPC di UNIXLinux

- pipe: (pipe, read, write)

- **segnali:** (kill, sigaction, ...)
- **semafori e vettori di semafori:** (semget, semop, semctl)
- **shared memory e memory mapping:** (shmget, shmat, shmdt, shmctl, mmap)
- **mailbox:** (msgget, msgctl, msgrcv, msgsnd)
- **gestione IPC:** (ipcs, ipcrm)
- **socket:** (socket, accept, bind, connect, read, write, ...)
- **mutex e condition variables nei pthread** [pthread = thread di POSIX]

7 Deadlock

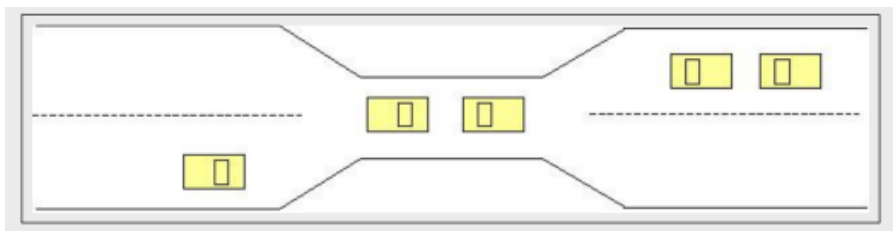
7.1 Il problema del Deadlock

7.1.1 Esempio ponte

Supponiamo che bi sia una strada con un ponte a senso unico alternato:

- Ci sarebbe spazio per una macchina alla volta (spazio dell'auto = risorsa).
- Le auto possono procedere in ambo le direzioni (auto = processi/utenti).
- Non vi sono semafori stradali, ovvero non esiste un protocollo da seguire per competere alla risorsa.

Ciò che ne risulterà sarà:



Ergo: nessun processo può procedere perché non detiene *l'uso esclusivo* delle **risorse** necessarie.

7.1.2 Varie tipologie di risorse

Ci possono essere risorse **logiche**, quali buffer, files, directory, semafori. O risorse **fisiche** quali RAM, disco, stampante, tastiera.

Una risorsa può essere **preemptable**, il cui stato può essere *ripristinato* e può essere sottratta "senza danni" al processo che la usa. Oppure non **preemptable** se la sua sottrazione causa la perdita di lavoro fatto.

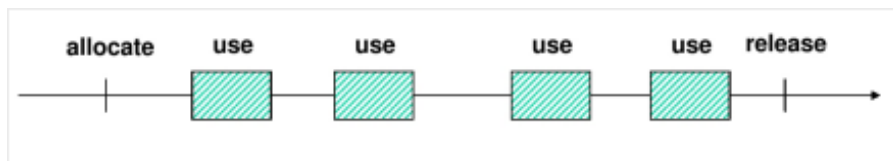
Assumeremo che $R_1, R_2, \dots, R_m$ siano i differenti **tipi** di risorsa disponibili:

(Ex. CPU, memoria, dispositivi...) e che per ogni R_i esistano $W_i \geq 1$ **istanze** della risorsa.

7.1.3 Protocollo d'uso delle risorse

Per poter utilizzare una risorsa il processo deve seguire un **protocollo d'uso**:

- **richiedere** la risorsa;
- **usare** la risorsa (anche più volte);
- **rilasciare** la risorsa;



Se non sono disponibili istanze della risorsa allora il processo può:

- attendere la disponibilità;
- attendere ma solo per un certo tempo (attesa limitata);

- fallire/terminare;
- ...

7.1.4 Il Deadlock

Definizione: Un insieme di processi si trova in **deadlock** se ogni processo dell'insieme è in attesa di un evento che può essere provocato *solo* da un altro processo dello stesso insieme.

Di solito l'evento atteso consiste nel rilascio di una risorsa.

NB. Vi sono delle condizioni **necessarie** affinché si possa verificare un deadlock!

7.1.5 Caratterizzazione delle Situazioni Critiche

Di seguito le quattro condizioni **NECESSARIE** al verificarsi del deadlock:

1. **Mutua esclusione:** devono essere coinvolte risorse che possono essere utilizzate solo da un processo alla volta;
2. **Possesso ed attesa:** un processo che detiene (almeno) una risorsa deve essere in attesa per ottenerne un'altra necessaria per procedere;
3. **Assenza di preemption:** non deve essere possibile la sottrazione di risorse ai processi che le hanno ottenute (fino a che non sono loro stessi a rilasciarle);
4. **Attesa circolare:** esiste un insieme di processi $P_0, \dots, P_n$ tale che $i=1, \dots, n$ ogni P_i attende una risorsa detenuta da P_{i-1} e P_0 attende una risorsa detenuta da P_n .

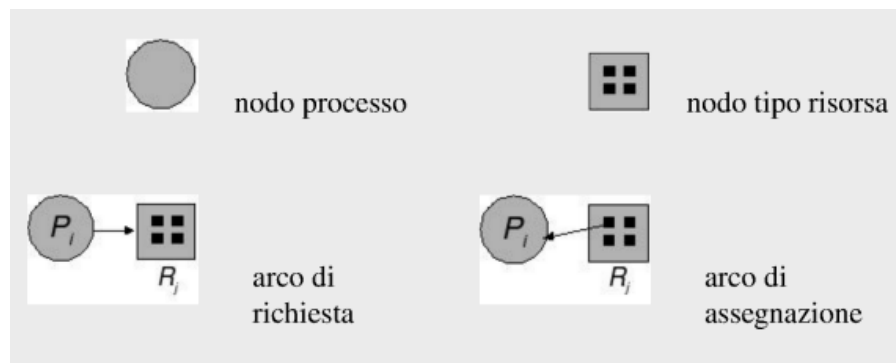
Queste condizioni sono **necessarie** ma non indipendenti.

7.1.6 Grafo di Allocazione delle Risorse

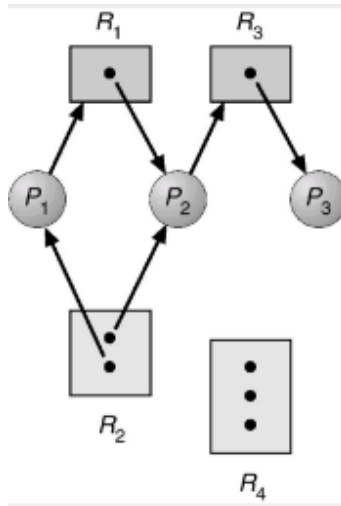
Le relazioni tra processi e risorse sono rappresentati tramite il *grafo di allocazione delle risorse* $G(V,E)$:

- **Vertici V:** i vertici sono di due tipi:
 1. *processo* P_i
 2. *tipo di risorsa* R_i
- **Archivi E:** gli archi connettono un tipo di risorsa e un processo e sono di due tipi:
 1. un arco $P_i \rightarrow R_j$ indica che il processo P_i è in attesa di ottenere una istanza della risorsa di tipo R_j (*arco di richiesta*);
 2. un arco $R_j \rightarrow P_i$ indica che una istanza della risorsa di tipo R_j è assegnata al processo P_i (*arco di assegnazione*);

Rappresentazione grafica:



Esempio: Si considerino tre processi e quattro tipi di risorsa: il tipo R_2 è disponibile in due istanze, mentre R_4 è disponibile in tre istanze:



P_2 detiene una sola istanza di R_2 e una di R_1 mentre è in attesa di una istanza (l'unica) di R_3

7.1.7 Proprietà del Grafo

Si può verificare che:

- se il grafo **NON** contiene cicli allora **NON** ci può essere **deadlock**;
- se invece vi sono cicli **POTREBBE** verificarsi un deadlock: dipendentemente dal numero di istanze di risorse esistenti;

In particolare:

- se per i nodi risorsa del ciclo vi sono istanze singole allora si ha deadlock (in tal caso ogni processo nel ciclo è in deadlock);
- altrimenti, se vi sono più istanze di risorse dello stesso tipo la presenza del ciclo è una condizione solo *necessaria* ma **non sufficiente** per il verificarsi del deadlock

Ergo: **grafico ciclico** e **attesa circolare** non sono concetti equivalenti!

7.1.8 Immagini di esempio grafo ciclico con e senza deadlock

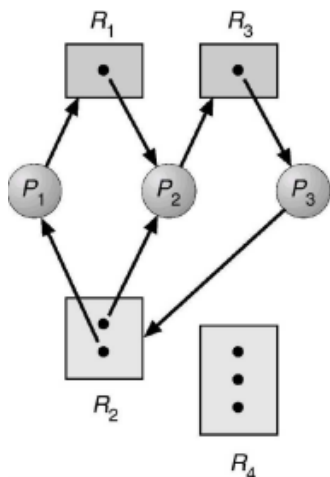


Figure 12: Vi sono due cicli e in questo caso anche **Deadlock**

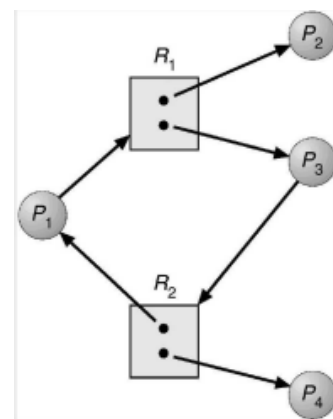
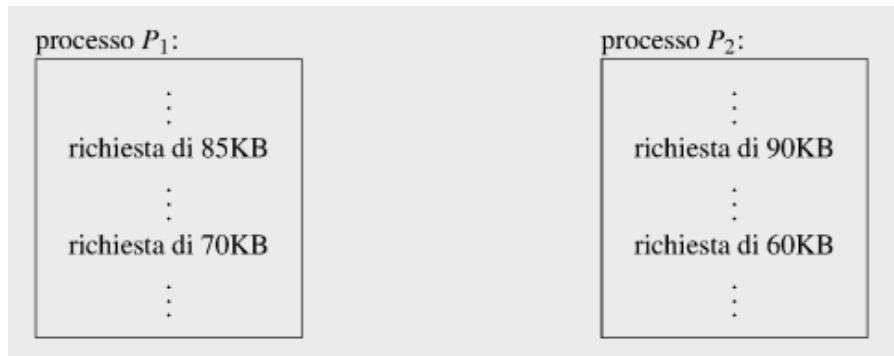


Figure 13: C'è un ciclo ma in questo caso non c'è **Deadlock**

Osservazione La successione e la temporizzazione della particolare sequenza di richieste/uso/rilascio delle risorse è **importante!!** Supponiamo che l'OS disponga di 200KB di memoria e consideriamo i due processi:



L'occorrenza del deadlock **dipende dalla velocità relativa** dei due processi.

- Dato che l'OS effettua lo scheduling dei processi, esso può operare in modo da influenzare la sequenza di richiesta e allocazione delle risorse (**solo *PARTICOLARI* sequenze portano al Deadlock**);
- L'OS potrebbe simulare per ogni sequenza la successione di eventi, costruendo un grafo di allocazione, grazie a questo può scegliere la successione più sicura

Il corretto svolgimento di questo compito non è tuttavia immediato, impone ad esempio di conoscere in anticipo tutto ciò che un processo deve fare durante la exec.

center

7.2 Gestione del Deadlock

I principali approcci al problema del deadlock mirano a:

1. prevenire il verificarsi del deadlock, **deadlock prevention** (*prevenzione statica*);
2. evitare il verificarsi del deadlock, **deadlock avoidance** (*prevenzione dinamica*);
3. rilevare e risolvere le situazioni di deadlock, **deadlock detection & recovery**;

Prevention e **avoidance** assicurano che il deadlock non si verifichi, mentre **detection & recovery** effettuano un'azione di ripristino qualora rilevi la presenza di deadlock.

7.2.1 The Ostrich Algorithm

Una possibilità ulteriore è l'**Ostrich Algorithm**:

Molti OS tra cui UNIX e Windows si limitano ad *ignorare il problema* e "confidare" che non si verifichi. Questa strategia ha senso se:

- le risorse sono "abbondanti" e il deadlock non è frequente;
- il costo per prevenire/rilevare sarebbe troppo elevato;

Nel malaugurato caso in cui si verificasse deadlock? **REBOOT!!**

7.2.2 Prevenzione del Deadlock

Questa tecnica risolve preventivamente il problema evitando che si verifichi *almeno una* delle condizioni necessarie al deadlock.

Purtroppo non sempre gli approcci sono applicabili. Ex:

Evitare Mutua esclusione: Si deve fare una distinzione tra risorse **non condivisibili** e risorse **condivisibili**. Si può così evitare la mutua esclusione imponendo che un processo NON aspetti mai per ottenere una risorsa condivisibile.

- Tuttavia molte risorse non sono condivisibili, quindi questo approccio spesso *non* è risolutivo;
- Alcune risorse (ex. stampanti) sono **spooled**: solo il (printer) **daemon** vi accede;
- NON tutte le risorse si possono gestire con lo spooling;

Evitare Possesso e attesa: Per evitare la condizione di possesso + attesa si deve imporre che un processo che detiene risorse non ne possa richiedere altre, quindi:

- Imporre al processo di richiedere tutte le risorse che serviranno, **prima** di eseguire una qualsiasi altra system call (ex. all'inizio della exec);
- Oppure premettere al processo di richiedere risorse **solo se non ne possiede** (ex. dopo che ha rilasciato quelle allocate);

Non è una soluzione ottimale perché porta ad un basso utilizzo delle risorse: tendendo verso la detenzione per più tempo del necessario. E soprattutto può comportare starvation: un processo che usa numerose risorse le deve chiedere contemporaneamente "sperando" che siano tutte libere nello stesso istante prima o poi.

Evitare Assenza di preemption: Si deve assicurare che non vi sia assenza di prelazione su risorse già acquisite:

- Se alla richiesta di un processo la risorsa non è disponibile allora tutte le risorse che detiene vengono forzatamente rimosse. (così la sua attesa non causa ad altri processi di attendere);
- Il processo verrà riavviato solo quando tutte le risorse (sottratte + richiesta) tornano disponibili

Un'altra possibile soluzione: se un processo P richiede una risorsa detenuta da Q questa viene sottratta a Q e allocata a P .

NB. Entrambi gli approcci si possono usare solo in presenza di risorse preemptable, non con quelle non-preemptable!!

Evitare Attesa circolare: Questa condizione si può evitare

- Stabilendo un ordinamento su tutti i tipi di risorsa esistenti;
- Imponendo che un processo richieda le risorse secondo una numerazione crescente;

In alternativa si impone che P non possa richiedere risorse di ordine n se possiede qualche risorsa di ordine $m > n$

Questa soluzione comporta però:

- la responsabilità del programmatore a sviluppare l'applicazione in modo che richieda le risorse nel modo corretto;
- la tendenza a rallentare l'exec dei processi e sottoutilizzare le risorse;

7.2.3 Evitare il Deadlock

Al contrario della prevenzione statica le tecniche di prevenzione **dinamica** comportano diverse decisioni:

- si presuppone di avere delle informazioni supplementari, a priori, su processi e risorse richieste;
- in base a queste info si usano degli algoritmi per individuare le situazioni pericolose e che potrebbero *evolvere in un deadlock* e decidere se concedere o meno le risorse;

7.2.4 L'idea di Base

Assumendo di conoscere a priori il massimo numero di richieste di ogni tipo di risorsa che potranno verificarsi: ogni processo deve dichiarare preventivamente tutto quello che **al massimo** userà.

Si introduce la nozione di **stato di allocazione delle risorse del sistema**, che contempla:

- #risorse disponibili;
- #risorse allocate;
- massime richieste da parte di ogni processo;

Così facendo si caratterizzano gli stati **safe** e prima di concedere una risorsa si verifica che ciò mantenga il sistema in uno stato safe

7.2.5 Stato Safe

Definizione: Uno stato è detto **safe** se e solo se esiste almeno una *sequenza safe di exec* di tutti i processi. Altrimenti lo stato è **unsafe**.

Def Sequenza Safe: Una sequenza $[P_1, \dots, P_n]$ di exec dei processi è **safe** se per ogni i le risorse che il processo P_i può ancora richiedere non sono superiori alla somma delle risorse disponibili e di quelle detenute dai processi P_j per $j < i$.

Se le richieste di P_i non possono essere soddisfatte allora P_i attenderà fino alla terminazione di tutti i $P_j (\forall j < i)$. Quando tutti i $P_j (\forall j < i)$ sono terminati allora P_i potrà ottenere le risorse richieste.

7.2.6 Stati Safe, Unsafe e Deadlock

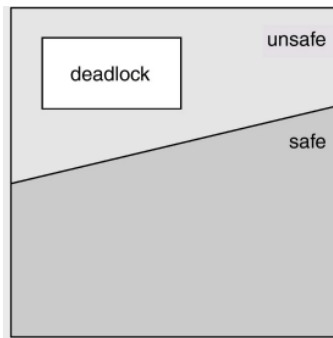


Figure 14: Se il sistema è in uno stato safe allora non c'è deadlock, viceversa non ci fosse lo stato safe **potrebbe** verificarsi deadlock.

7.2.7 Algoritmi per la Deadlock-Avoidance

Lo scopo di questi è di far evolvere il sistema garantendo lo stato safe; due sono le tecniche base:

1. **Grafo di allocazione delle risorse**, in caso di istanze *singole* delle risorse;
2. **Algoritmo del banchiere**, in caso di istanze multiple;

7.2.8 Alg Grafo assegnazione risorse

Viene sfruttato il grafo precedentemente definito, con l'aggiunta dell'**arco di reclamo o claim edge**:

- dati nodo P_1 e R_1 un arco di reclamo $P_1 - - > R_j$ indica che il processo P *potrebbe* richiedere R ;
- nel momento della richiesta l'arco di reclamo si converte in richiesta $P_i \rightarrow R_j$;
- in caso di riascio invece l'assegnamento torna reclamo;
- ogni processo *DEVE* dichiarare all'inizio tutti gli archi di reclamo nel grafo;

Avvenuta una richiesta questa può essere soddisfatta solo se la conversione dell'arco di reclamo in assegnamento NON introduce cicli nel grafo. Ex:

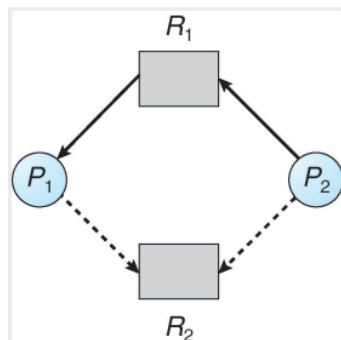


Figure 15: P_1, P_2 potrebbero richiedere R_2

Una richiesta da P_1 può essere subito eseguita ma per servire P_2 bisogna attendere il deallocaimento della risorsa detenuta da P_1 . Per mantenere il **Safe Status** R_2 deve essere allocata prima a P_1 e poi a P_2 . Ex. concessione R_2 prima a P_2 :

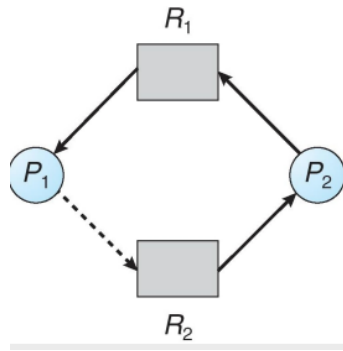


Figure 16: In questa situazione le seq di exec possibili sono $[P_1, P_2]$ e $[P_2, P_1]$

Entrambe le situazioni sarebbero **NON SAFE!!**

7.2.9 Algoritmo del banchiere

Il **Banker's algorithm** è la generalizzazione della tecnica di deadlock-avoidance al caso di tipi di risorse con istanze multiple

- anche in questo caso i processi devono dichiarare inizialmente le risorse *al massimo* richieste;
- all'atto della richiesta si può verificare una attesa prima di ottenere la risorsa;
- una volta ottenute le risorse, queste devono essere rilasciate entro tempo limitato;

I punti salienti sono:

1. Definizione delle strutture dati;
2. Fase di verifica della safety;
3. Protocollo per la richiesta/concessione delle risorse;

Strutture dati: Sia n il numero di processi e m il numero di tipi di risorse, definire quattro vettori/matrici:

- **available[m]**: l'elemento **available** indica il numero di istanze della risorsa disponibili;
- **max[n,m]**: è la *matrice di reclamo*, l'elemento **max[i,j]** indica il massimo numero di risorse di tipo j che il processo i può richiedere;
- **allocation[n,m]**: l'elemento **allocation[i,j]** è il numero di risorse j che il processo i detiene;
- **need[n,m]**: l'elemento **need[i,j]** indica il massimo numero di risorse di tipo j che il processo i potrebbe ancora richiedere per completare l'exec;

NB. *need* è ottenibile come differenza tra *max* e *allocation*!

Verifica Safety: Questo è un test della **safety di uno stato**: determina, se esiste, una sequenza safe. Si definiscono **work[m]** e **finish[n]** come due vettori:

$$work := available - finish[i] := false (\forall i)$$

1. trova i tale che

- $finish[i] == false$;
- $need_i \leq work$

se tale i non esiste va al passo 3;

2. pone $work := work + allocation_i$ e $finish[i] := true$ (back to step 1)

3. se $finish[i] == true$ per ogni i allora stato = **safe**;

La successione degli indici i trovati al passo 1 è la **seq safe**.

Richiesta Risorse Per ogni processo P_i si impiega un vettore $request_i[m]$, $request[j] = k$ indica che P sta richiedendo k istanze di R. Se P operadelle richieste allora:

1. if($request_i \leq need_i$) {continue()} else{exit 1};
2. if($request_i \leq available$) {continue()} else {wait(P)} che si liberino le risorse;
3. simulazione assegnamento:

$$\begin{aligned} available &:= available - request_i \\ allocation_i &:= allocation_i + request_i \\ need_i &:= need_i - request_i \end{aligned}$$

Ora detemina se lo stato ottenuto é **safe**.

- se safe concede le risorse a P;
- se non safe P attende e si ripristina lo stato precedente;

7.2.10 Esempio Alg Banchiere

Supponendo vi siano 5 processi ($P_0, \dots, P_4$) 3 tipi di risorse: A con 10 istanze, B con 5 e C con 7. A T_0 abbiamo:

Processo	allocation			max			available			need		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	3	3	2	7	4	3
P_1	2	0	0	3	2	2				1	2	2
P_2	3	0	2	9	0	2				6	0	0
P_3	2	1	1	2	2	2				0	1	1
P_4	0	0	2	4	3	3				4	3	1

Figure 17: $[P_1, P_3, P_4, P_2, P_0]$

Usando l'algoritmo di safeness si trova che lo stato é safe con la seq sopracitata.

7.2.11 Rilevare il Deadlock

L'approccio basato sulla rilevazione ed eliminazione delle situazioni di deadlck é così strutturato:

- il sistema è lasciato libero di evolvere e può raggiungere uno stato non sicuro;
- utilizzando algoritmi appositamente progetati per individuare la presenza di situazioni di deadlock;
- qualora si rilevi la presenza di deadlock si attuano delle procedure per eliminarlo e tornare a un **Safe State**;

Le due tecniche principali riguardano:

- il caso con solo tipi di risorse con **istanza singola**
- il caso generale di tipi di risorse con **istanze multiple**

Risorse a Singola Istanza Si utilizza una versione semplificata del grafo di allocazione, quello di **attesa**:

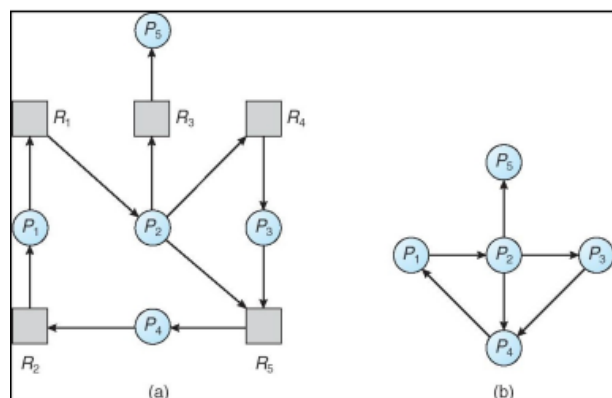


Figure 18: Grafo di *allocazione* (a) — Grafo di *attesa* (b)

Uso del grafo di attesa:

- gli archi si eliminano quando le risorse vengono rilasciate;
- la presenza di un ciclo nel grafo indica un deadlock;
- l'OS attiva **periodicamente** un algoritmo di rilevamento dei cicli nel grafo;
- se vi sono n processi tale algoritmo richiede un numero di operazioni dell'ordine di n^2 ;

Risorse a Multipla Istanza In questo caso si devono gestire informazioni sullo stato delle varie istanze dei diversi tipi di risorse:

- **available[m]**: l'elemento $available[j]$ indica il numero di istanze della risorsa di tipo j disponibili;
- **allocation[n,m]**: l'elemento $allocation[i,j]$ indica il numero di risorse di tipo j che il processo i detiene;
- **request [n,m]**: l'elemento $request[i,j]$ indica il numero di istanze della risorsa j che il processo i richiede;

7.2.12 Algoritmo di Rilevamento

L'idea base prevede di:

- allocoare le risorse ai processi assumendo che
 1. non ne chiedono altre in futuro;
 2. quelle richieste siano sufficienti a terminare l'exec e vengano rilasciate concluso l'utilizzo;
- nel caso queste assunzioni vengano falsificate e si **verifichi Deadlock** si attueranno delle procedure di eliminazione;

Similmente all'algoritmo del banchiere si usano due vettori: $work[m]$ e $finish[m]$ inizializzati come segue:

$work := available$

$\forall i$, se $allocation[i] \neq 0$ allora $finish[i] := false$, altrimenti $finish[i] := true$

1. Trova un processo i tale che $finish[i] == false$ e $request[i] \leq work$ ¹. if !exist $\rightarrow$ punto 3;
2. Pone $work := work + allocation[i]$ e $finish[i] := true$ ²; $\rightarrow$ punto 1;
3. Se $finish[i] == false$, per qualche i , allora c'è deadlock e questo coinvolge il processo i .

NB. L'algoritmo necessita di ordine di $m * n^2$ operazioni!

7.2.13 Esempio Alg Rilevamento

Supponendo ci siano 5 processi ($P_0, \dots, P_4$), 3 tipi di risorse: A(7 istanze), B(2 istanze) e C(6 istanze). A T_0 abbiamo:

Processo	allocation			request			available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	0	0	0	0	0	0
P_1	2	0	0	2	0	2			
P_2	3	0	3	0	0	0			
P_3	2	1	1	1	0	0			
P_4	0	0	2	0	0	2			

Il sistema non presenta un deadlock, infatti eseguendo l'algoritmo secondo la sequenza $[P_0, P_2, P_3, P_1, P_4]$ si determina che $finish[i] == true, \forall i$.

Si supponga ora che il processo P_2 richieda un'altra istanza di tipo C. La soluzione diventa:

Processo	allocation			request			available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	0	0	0	0	0	0
P_1	2	0	0	2	0	2			
P_2	3	0	3	0	0	1			
P_3	2	1	1	1	0	0			
P_4	0	0	2	0	0	2			

Ora il sistema presenta deadlock composto da: P_1, P_2, P_3, P_4 perchè le risorse detenute da P_0 non bastano a sbloccare la situazione.

7.2.14 Quando attivare l'algoritmo di rilevamento?

Bisogna considerare:

- quanto spesso in media si verifica un deadlock;
- quanti sono in media i processi coinvolti;
- quanto costa eseguire l'algoritmo;

Alcune possibilità sono:

- ad ogni richiesta;
- ogni volta che una richiesta non può essere soddisfatta;
- iterativamente, ad intervalli di tempi prestabiliti;
- quando la percentuale di utilizzo della CPU scende sotto una soglia minima;

E' altresì importante considerare che: attivare l'algoritmo **troppo spesso** causa un eccessivo overhead, e viceversa attivarlo **troppo raramente** porta a rilevare situazioni di deadlock (eventualmente multipli) troppo tardi: vengono coinvolti troppi processi e diviene molto difficile capire chi ha portato al problema.

7.2.15 Tecniche di ripristino

Ve ne sono principalmente due:

Terminazione di uno o più processi
Revoca delle risorse concesse e detenute da uno o più processi

Terminazione di processi: Si può agire in due modi

1. **Terminare tutti i processi** coinvolti nel deadlock: semplice ma costoso;
2. **Terminare un processo alla volta** fino a che persiste la situazione di deadlock: indubbiamente più articolato perchè cicla il controllo deadlock su ogni passaggio.

Nel decidere quale processo terminare si possono considerare le seguenti:

- priorità e classe del processo;
- mole di lavoro già eseguito o da eseguire;
- quante risorse detiene e quante ne richiederà per terminare;

In generale la terminazione è un'opzione da evitare, causa overhead di gestione e perdita di lavoro. Può inoltre portare a perdita di consistenza di risorse modificate dal processo.

Prelazione sulle risorse: In questo caso si seleziona un processo **vittima**, a cui revocare la o le risorse, si stabilisce una politica decisionale sulle risorse da sottrarre. Fatto ciò si effettua un **rollback** del processo selected, per riportarlo all'ultimo save state consistente (in alternativa lo si riavvia) e si pone un limite al numero id volte che lo stesso processo può essere selezionato come vittima.

7.2.16 Approccio Combinato per Gestione Deadlock

Un approccio combinatorio visto in precedenza:

- Partizionare i tipi di risorse in gruppo e stabilire un ordinamento. Ex:
 1. memoria secondaria;
 2. risorse per processi (I/O, files...);
 3. memoria principale;
- Impiegare la tecnica di prevenzione per evitare l'attesa ciclica tra gruppi;
- Intermanete da ogni gruppo, impiegare la piùadatta tra le varie tecniche "anti-deadlock";

8 Memoria Centrale

8.1 Generalità e Gestione

8.1.1 Gestione della memoria

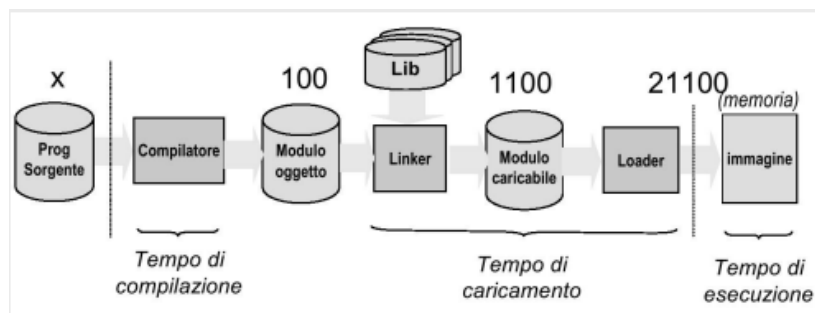
Problematiche da affrontare:

- **Allocazione** di job/processi in memoria
- **Protezione** dello spazio di indirizzamento
- **Condivisione** dello spazio di indirizzamento
- Gestione dello **swapping**

Vincolo di funzionamento : per essere eseguito un programma deve essere caricato in memoria e trasformato in processo. La CPU preleva istruzioni e operandi dalla memoria, esegue, e alla fine la memoria viene rilasciata.

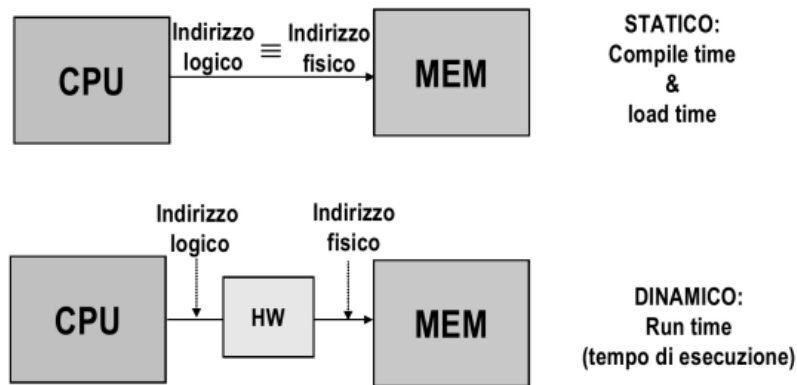
8.1.2 Indirizzi di memoria e binding

- Trasformazione del programma in processo implica la distinzione tra **spazio degli indirizzi logici** e **spazio degli indirizzi fisici**.
- Il **binding** degli indirizzi può avvenire:
 - Al compile time (*statico*):
 - * se la posizione della MC ove andrà collocato il programma deve essere nota a priori [si può generare codice **assoluto**]
 - * per ricollocare il programma si deve ricompilare
 - Al load time (*statico*):
 - * si genera del codice **rilocabile** tramite compilazione
 - * indirizzi **relativi** [all'inizio del programma]
 - * per cambiare collocazione bisogna ricaricare
 - Al run time (*dinamico*):
 - * binding posticipato [durante l'esecuzione si può spostare il processo]
 - * è richiesto supporto **hardware** adeguato perchè sia efficiente

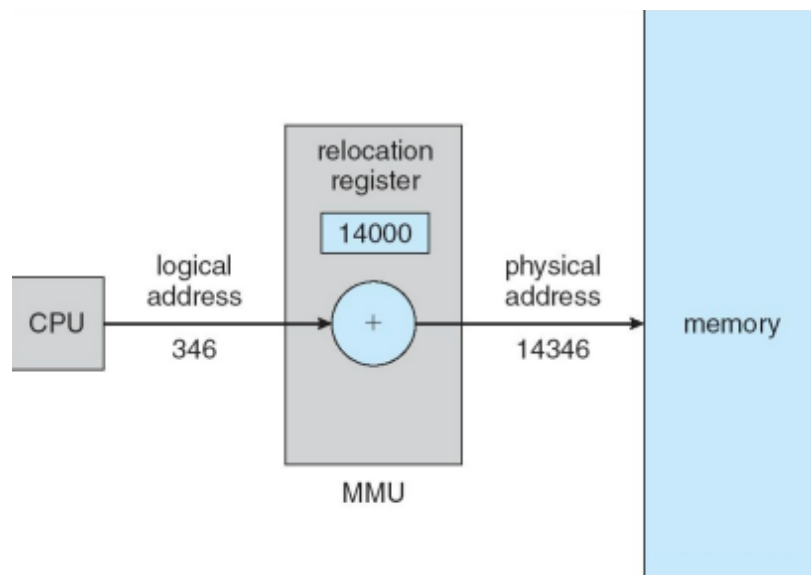


Il binding di dati/istruzioni a indirizzi assoluti può avvenire in diversi momenti:

- Al tempo di compilazione
- Al tempo di caricamento
- Al tempo di esecuzione (richiede MMU)



MMU (Memory Management Unit): Il registro di rilocazione (base) viene aggiunto a ogni indirizzo logico generato dal processo:



8.1.3 Caricamento e collegamento

Caricamento:

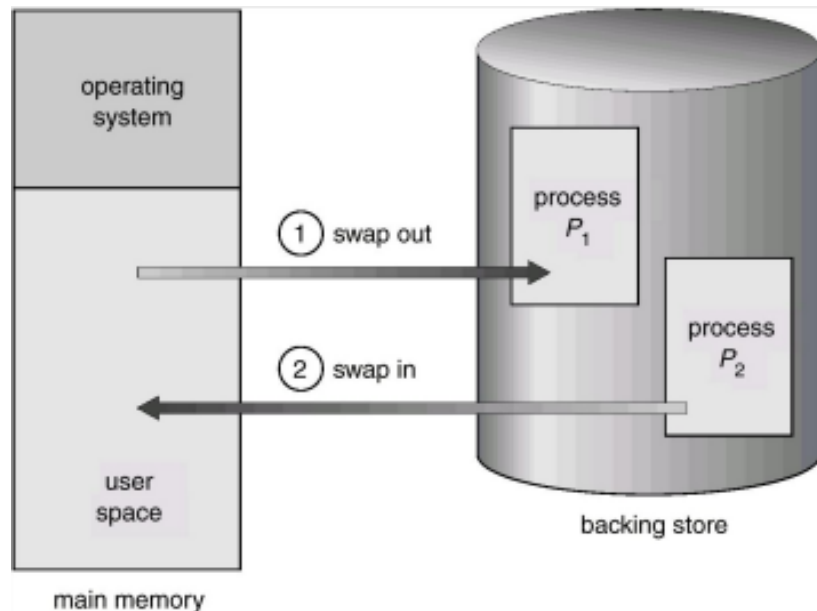
- **Caricamento statico:** tutto il codice in memoria prima dell'esecuzione
- **Caricamento dinamico:** i moduli di codice vengono caricati solo al primo utilizzo, quindi le parti di codice non vengono caricate "risparmiando" memoria [es. utile per error handling]

Collegamento:

- **Collegamento statico:** tutte le dipendenze sono risolte prima dell'avvio, l'immagine del processo contiene tutto il codice delle eventuali librerie utilizzate.
- **Collegamento dinamico:** solitamente per le librerie, il codice viene inserito solo se richiesto ma contiene dei riferimenti chiamati *stub* (indica dove trovare il codice della libreria alla prima occasione per poi venir sostituito dalla sua locazione in memoria)

8.1.4 Swapping

Tecnica che permette di rimuovere un processo dalla memoria principale e spostarlo in memoria secondaria (*swap out*), per poi ricaricarlo (*swap in*).



- Il tempo di swapping dipende dalla quantità di dati trasferiti.
- ha senso in presenza di multiprogrammazione (in caso contrario si dovrebbe attivare ad ogni Context Switch).
- necessario supporto al binding dinamico per poter rilocalizzare i processi swapped-in
- Nei sistemi moderni lo swapping avviene spesso solo su porzioni del processo (pagine).
- Nei dispositivi mobili lo swapping spesso non viene usato perché degraderebbe la memoria flash.

8.1.5 Considerazioni

- In multiprogrammazione non si può fissare a priori la posizione dei processi in memoria.
- Lo swapping richiede rilocalizzazione dinamica.

Le principali tecniche di gestione della memoria principale, nel caso il programma sia **interamente** caricato in MC, sono:

- Allocazione contigua
- Paginazione
- Segmentazione
- Segmentazione paginata

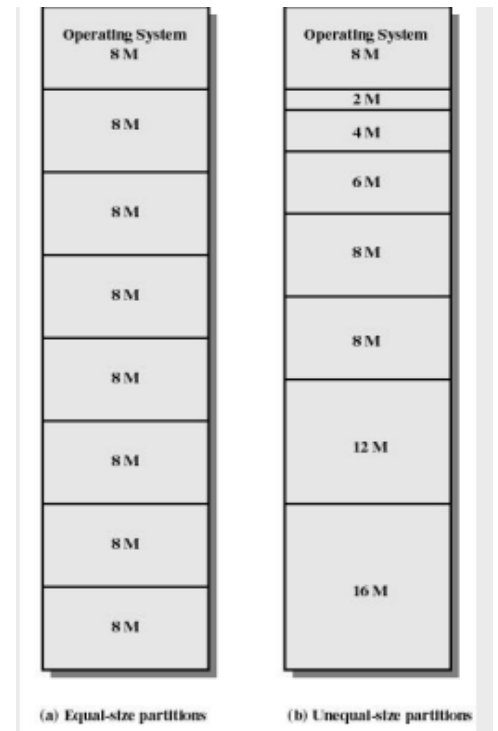
8.2 Allocazione Contigua

8.2.1 Concetti generali

- I processi sono allocati in memoria in posizioni contigue.
- La memoria è suddivisa in due zone: una per il S.O. e una per i processi utente.
- Due principali varianti:
 - Allocazione contigua con partizioni fisse
 - Allocazione contigua con partizioni variabili

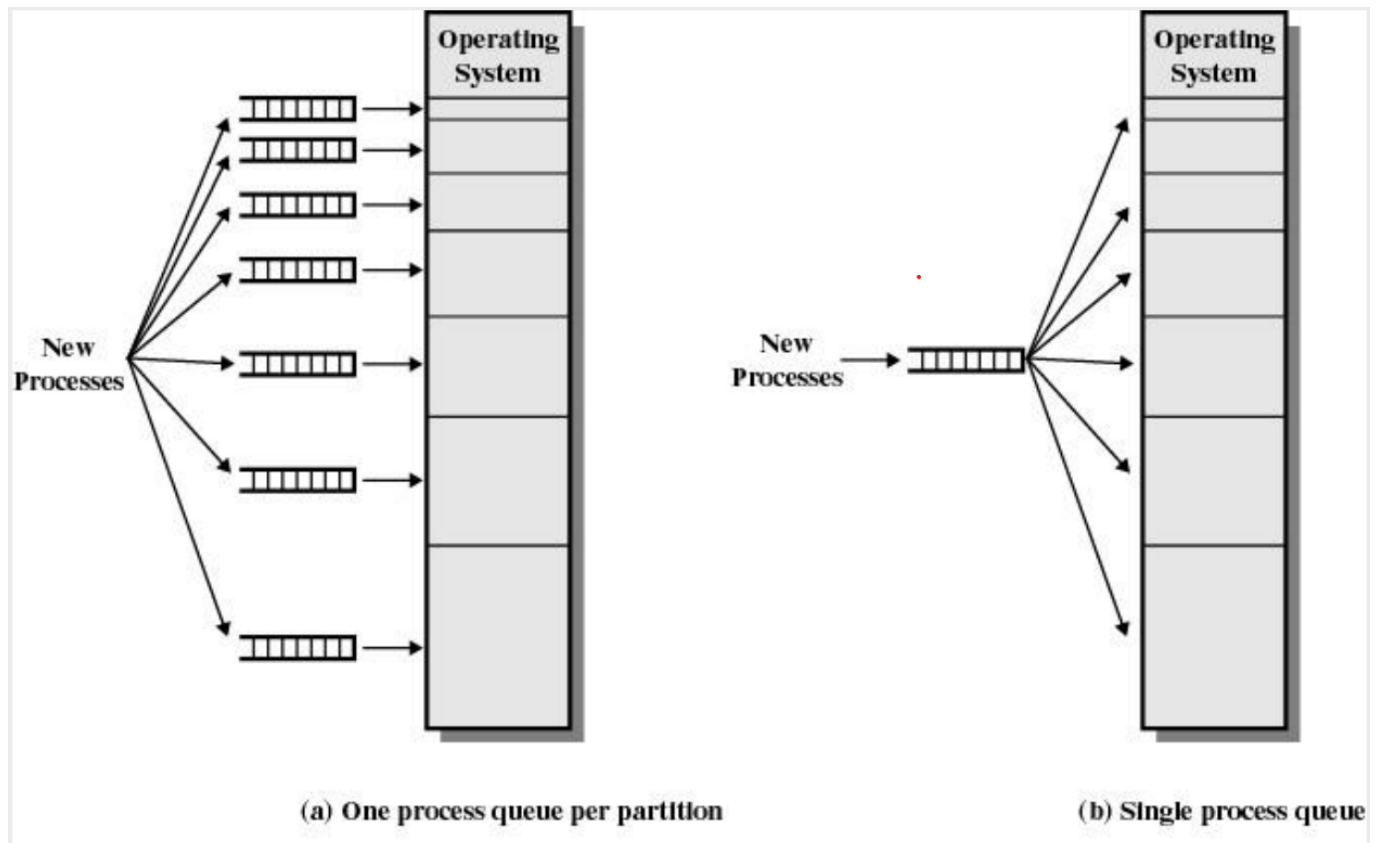
8.2.2 Partizioni fisse

- Memoria suddivisa in partizioni di dimensioni prefissate (non necessariamente uguali).
- La scelta del numero di partizioni e delle dimensioni è unica.
- Ogni processo viene caricato in una partizione in grado di contenerlo.

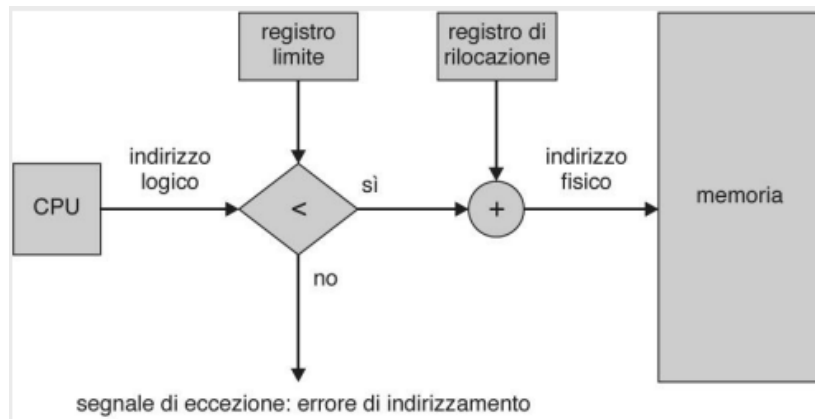


Allocazione della memoria

- **una coda per ogni partizione:** ogni processo creato viene assegnato alla coda relativa alla partizione più piccola tra quelle che possono contenerlo [manca di flessibilità → ci possono essere processi in coda e partizioni vuote non utilizzate]
- **una coda singola:** il processo viene allocato nella partizione libera più piccola sufficiente a contenerlo [se no si fa swap-out].



Supporto per rilocalizzazione MMU con registri base e limite:



Valutazione

- **Vantaggi:** implementazione semplice, basso overhead [insieme delle risorse (tempo, spazio, ecc.) che vengono consumate per gestire e coordinare le attività e non per eseguire direttamente le applicazioni dell'utente].
- **Svantaggi:** grado di multiprogrammazione fissato, generazione di frammentazione interna [se la dimensione della partizione è superiore alla dimensione del processo che vi si carica] ed esterna [se vi sono partizioni non utilizzate perchè non soddisfano le esigenze dei processi in coda] .

8.2.3 Partizioni variabili

Si vorrebbe partizionare la memoria in parti di dimensioni corrispondenti alla dimensione dei processi che vi saranno caricati [eliminando così il problema della frammentazione intera], per ottenere questo risultato non va fissato a priori numero e dimensioni delle partizioni.

- Ogni processo viene caricato in un buco (hole [sezione di memoria libera]) di memoria sufficientemente grande.
- Il S.O. mantiene la lista dei buchi disponibili.

Politiche di allocazione

- **First-Fit:** si occupa il primo buco sufficientemente grande.
- **Best-Fit:** si occupa il buco più piccolo sufficiente.
- **Worst-Fit:** si occupa il buco più grande.

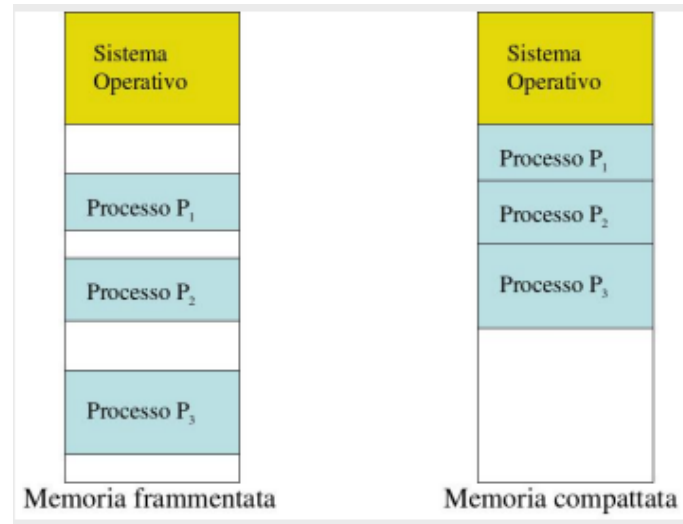
Supporto per rilocalizzazione MMU come per le partizioni fisse [registri base e limite].



Valutazione

- **Vantaggi:** non esiste frammentazione interna, grado di multiprogrammazione non prefissato.
- **Svantaggi:** frammentazione esterna.

Compattazione Soluzione per la frammentazione esterna: rilocalizzare i processi e compattare lo spazio libero per ottenere un buco unico [possibile solo con rilocalizzazione dinamica].



8.2.4 Buddy System

- Compromesso tra partizioni fisse e variabili.
- Sia 2^m = dimensione del più piccolo blocco che si vuole allocare, 2^M = tutta la memoria.
 - memoria vista come insieme di liste di blocchi di dimensione 2^k con $m \leq k \leq M$
 - la memoria disponibile è quindi un insieme di blocchi di dimensione 2^i ($\forall i$)
 - all'inizio la lista di blocchi di dimensione 2^M contiene un solo blocco (memoria intera) mentre tutte le altre sono vuote
 - quando giunge una richiesta di dimensione s si cerca un blocco di dimensione sufficiente ma che sia una potenza di 2, in questo modo:
 - * se $2^{M-1} < s \leq 2^M$ si alloca l'intera memoria, altrimenti
 - * il blocco di dimensione 2^M viene diviso in due blocchi di dimensione 2^{M-1} e si verifica ricorsivamente il primo punto proposto
- quando un processo rilascia il suo blocco questo viene inserito nell'opportuna lista di blocchi liberi
- se due blocchi adiacenti da 2^i si liberano, si possono fondere ottenendo uno da 2^{i+1}

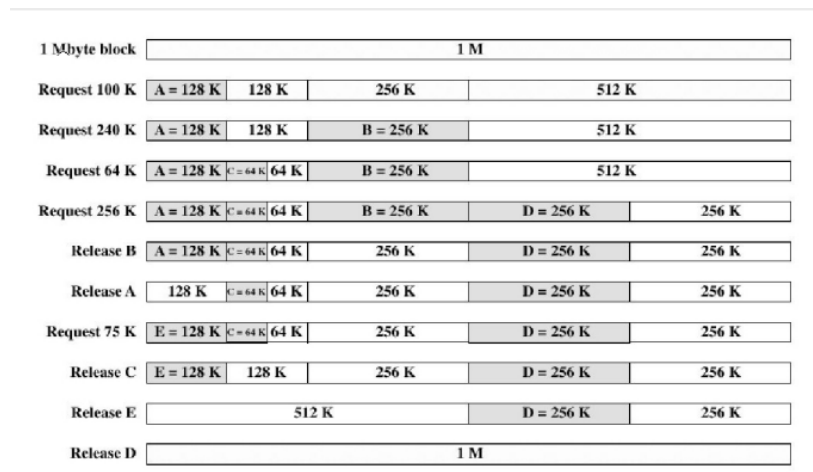


Figure 19: Esempio di operazioni su Buddy System

Valutazione

- **Vantaggi:** Compattazione semplice.
- **Svantaggi:** Presenza di frammentazione interna [i processi non hanno sempre dimensione 2^i].

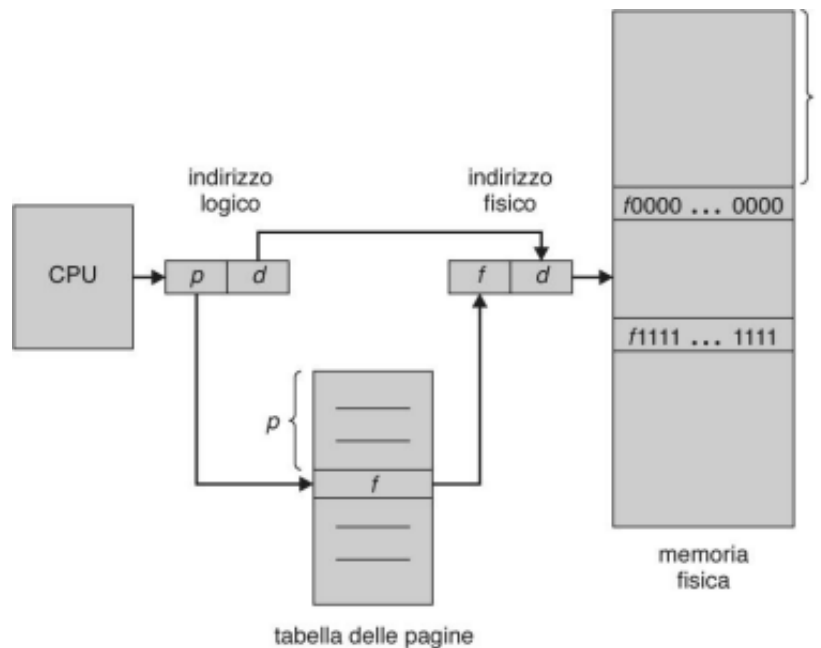
8.3 Paginazione

8.3.1 Generalità

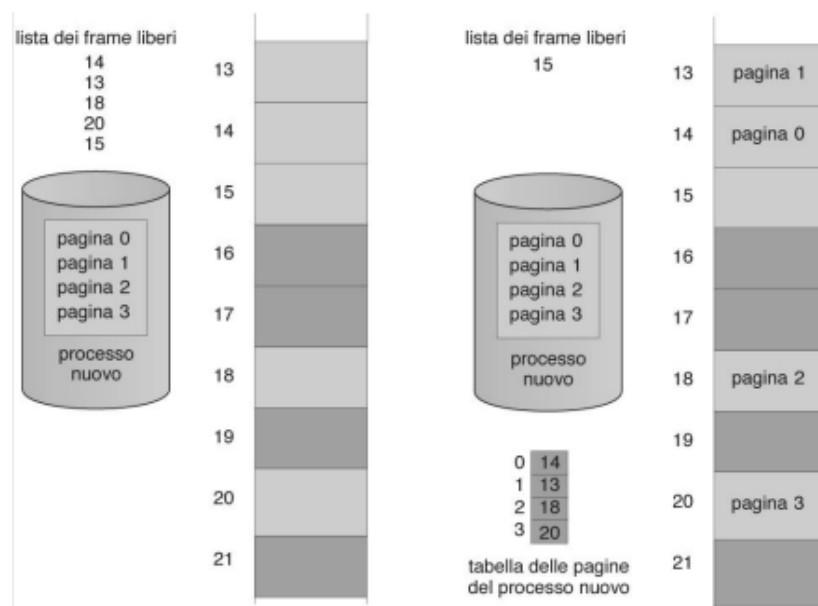
- Elimina la frammentazione esterna (idea = permettere che un processo *non sia allocato in modo contiguo*).
- Memoria fisica organizzata in **frame**, memoria logica in **pagine** della stessa dimensione [sia frame che pagine sono blocchi di dimensione fissa].
- Il S.O. tiene traccia dei frame liberi.
- per caricare un processo che occupa n pagine bisogna trovare n frame liberi
- La traduzione degli indirizzi si basa su una page table per processo.
- Frammentazione interna solo nell'ultima pagina di ogni processo.

8.3.2 Architettura

- L'indirizzo logico è diviso in numero di pagina [p - indice nella page table per determinare il numero di frame in memoria fisica] e offset [d - combinato con il numero di frame determina l'indirizzo fisico].



- Si tiene traccia dei frame liberi tramite una **tabella dei frame**.



8.3.3 Gestione dei processi

- Ogni processo ha una page table, il cui puntatore viene inserito nel PCB.

- Il context switch è più oneroso (serve gestire informazioni sulla paginazione).
- Lo swapping influenza le tabelle dei frame.

8.3.4 Page Table

- Può essere realizzata con registri dedicati [efficiente ma non adatta per tabelle grandi, switch di contesto più complesso], oppure in memoria [un solo registro da gestire nei context switch, *due* accessi alla memoria per accedere in memoria (difetto)].
- Il **page-table base register (PTBR)** punta alla tabella in memoria.
- Accesso rapido con **TLB** (Translation look-aside Buffer) [memoria associativa key-value].

Page Table con TLB

- La TLB contiene solo **parte** della page table.
- Per tradurre un indirizzo si accede alla TLB.
- In caso di TLB-miss si accede alla tabella in memoria.
- La gestione di quale parte della TLB sia da duplicare è fatta con tecniche utilizzate per la gestione delle cache (es. FIFO, LRU, ...)
- la percentuale di TLB-hit si dice **hit-ratio**
- Durante il context switch la TLB va svuotata.

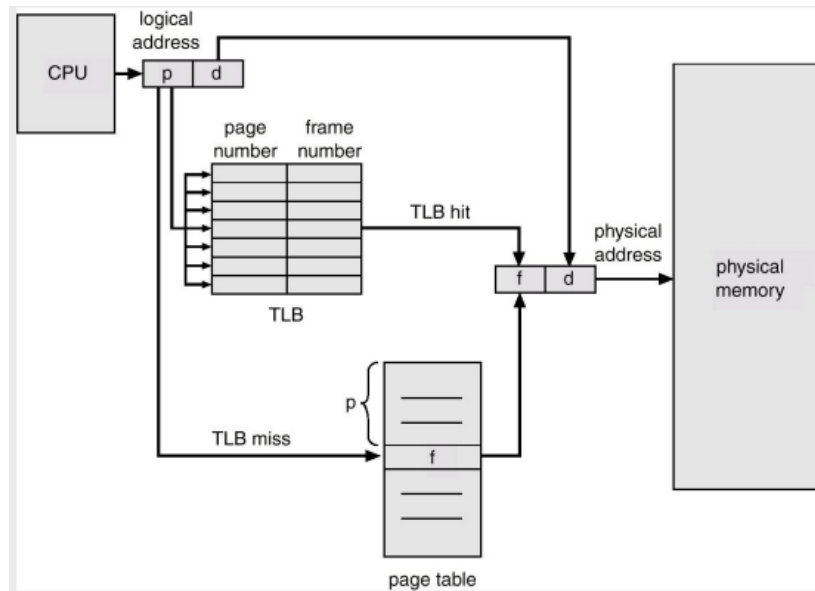
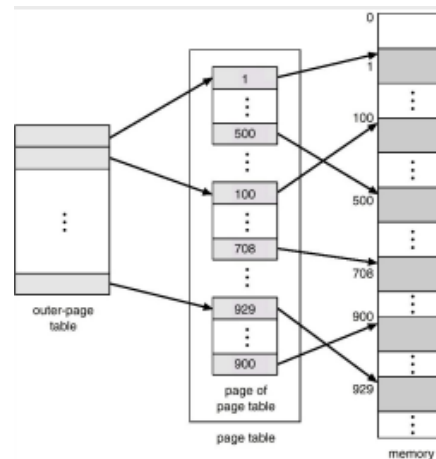
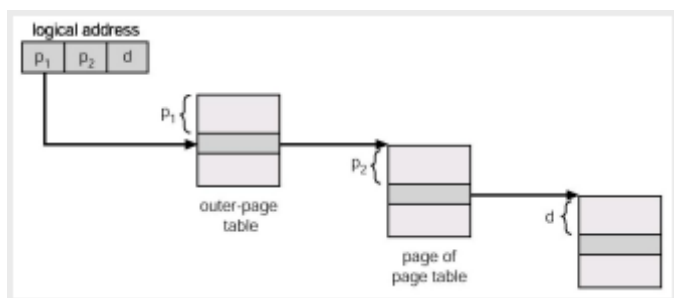


Tabella delle Pagine e Protezione

- Ad ogni elemento della Page Table si associa un *bit di validità* per determinare quali pagine sono nello spazio logico del processo
- Nella Page Table potrebbero esserci anche bit per indicare che un frame può essere oggetto di *scrittura, lettura, esecuzione, ...* da parte di un processo

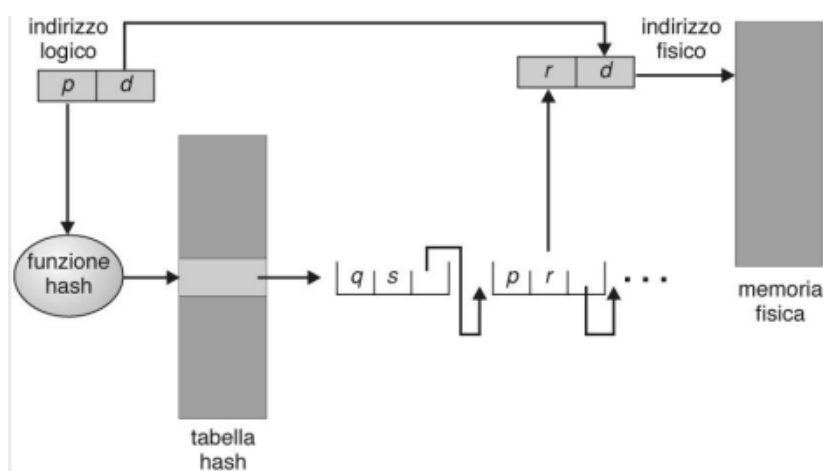
Struttura della Page Table Utili nel caso la tabella sia di dimensioni elevate

- **Paginazione gerarchica:** la page table stessa è paginata (più livelli), il seguente esempio è un'organizzazione a 2 livelli dove il numero di pagina p è suddiviso in p_1 e p_2 .



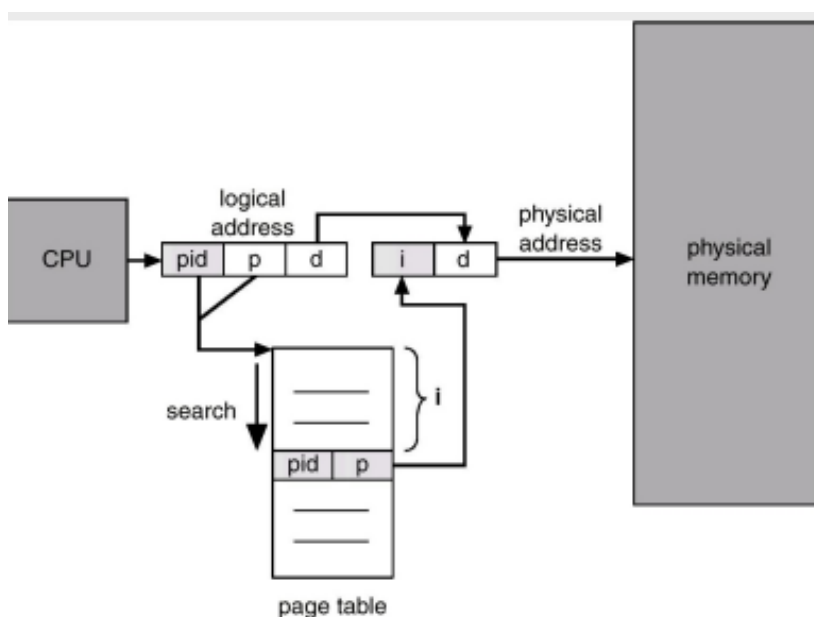
Problema: L'aumento dei livelli nelle tabelle delle pagine multilivello comporta più accessi per la traduzione degli indirizzi, rendendo l'hashing una soluzione interessante per ridurre il numero di accessi necessari.

- **Hashing:** soluzione per indirizzamenti molto grandi, rispetto alla precedente diminuisce il numero di accessi.



Problema: La Page Table di un processo deve contenere una entry per ogni possibile pagina logica, anche se molte di queste pagine non sono effettivamente allocate: di conseguenza, la tabella può risultare molto grande, richiedendo molta memoria anche quando il processo utilizza solo una parte delle sue pagine potenziali.

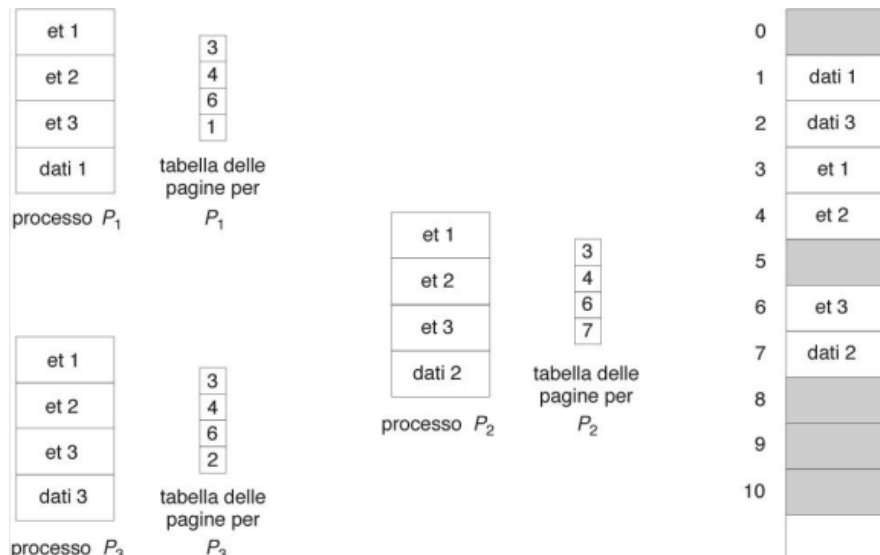
- **Page Table invertita:** tabella che memorizza solo le informazioni sulle pagine usate dai processi, unica per tutto il sistema [una entry per ogni frame fisico i . La entry contiene l'indirizzo logico della pagina p caricata in quel frame e l'indicazione del processo possessore pid .



Problema: Minor memoria occupata ma Maggior tempo di traduzione, bisogna cercare la coppia (pid, p) nella tabella che è ordinata rispetto agli indirizzi fisici *i*. Si può rimediare implementando la *inverted page table con hashing*

Pagine condivise Le tecniche viste ci permettono di mappare pagine logiche in pagine fisiche o di mappare *diverse* pagine logiche alla *stessa* pagina fisica

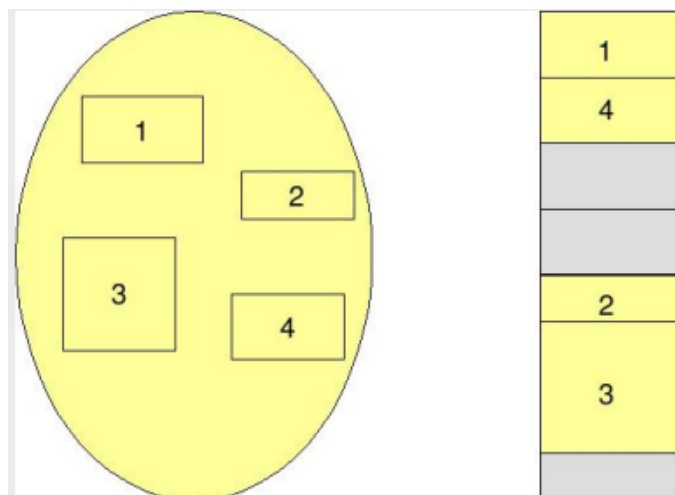
- condivisione di dati e codice (evitando replicazioni) permessa
- Il codice condiviso non deve essere in grado di auto-modificarsi [si dice *reentrant*]



8.4 Segmentazione

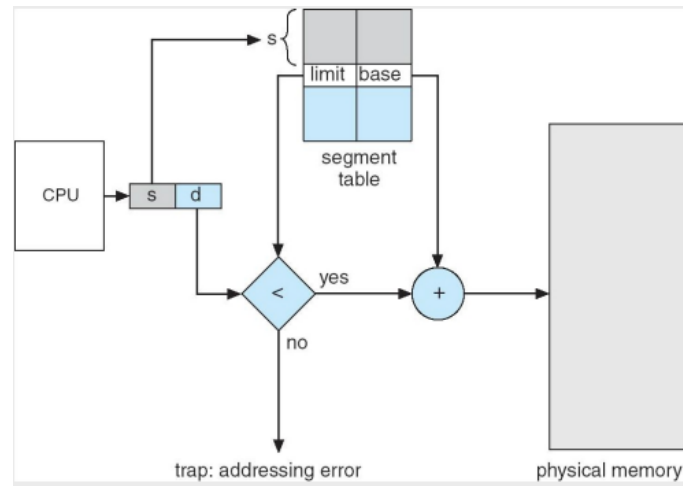
8.4.1 Generalità

- La memoria è vista come un insieme di segmenti con **nome**, **dimensione** e **funzione** (main, procedure, dati, stack...).
- Indirizzo specificato dal nome del segmento (es. id numerico) e un offset al suo interno.
- il compiler genera i segmenti opportuni durante la compilazione del programma [ma si possono ancora aggiungere tramite per esempio il *linker* mentre il *loader* gli fornisce dei nomi].



8.4.2 Architettura per Segmentazione

- la traduzione degli indirizzi avviene tramite **segment table** che per ogni segmento contiene base (indirizzo fisico di un segmento) e limite (lunghezza del segmento).
- La segment table è mantenuta in registri o allocata in memoria (con STBR [*segment-table base register*] e STLRL [*segment-table length register*]).



8.4.3 Considerazioni

- Rilocalizzazione dinamica a livello di segmento.
- Possibilità di condivisione di segmenti (es. librerie).
- Problema di frammentazione esterna (i segmenti possono avere dimensioni diverse).
- Protezione tramite bit di validità e privilegi e modalità di accesso [read, write, ...] per ogni segmento nella segment table.
- La segmentazione è visibile all'utente/programmatore ma paginazione trasparente all'utente.

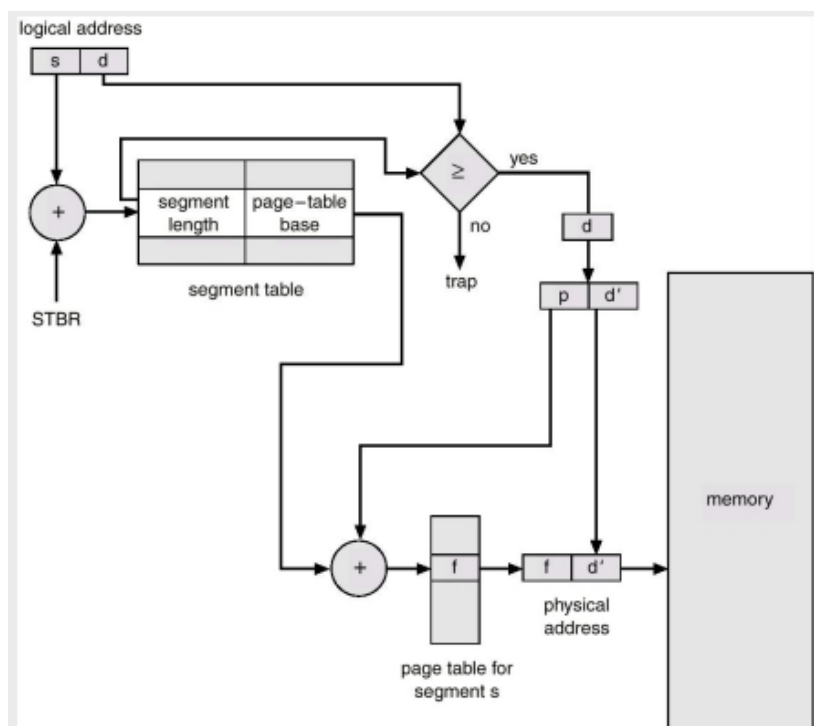
8.5 Segmentazione Paginata

8.5.1 Generalità

Combinazione tra segmentazione e paginazione:

- Lo spazio logico è segmentato ed ogni segmento è paginato [suddiviso in pagine della dimensione del frame fisico].
- Per ogni processo si gestisce **una** segment table e **certo numero di** page table per ogni segmento.
- Si evita la frammentazione esterna, resta solo quella interna nell'ultima pagina di ogni segmento (ma comunque minore).

8.5.2 Architettura



9 Memoria Vituale

9.1 Motivazioni

Una caratteristica comune a tutte le tecniche di gestione della memoria é che l'intero programma deve **essere caricato in memoria** per essere eseguito. In generale questo non sarebbe necessario:

L'idea è di caricare in memoria solo le pagine di programma che sono **necessarie all'exec**.

Conseguenze:

- La dimensione dello spazio logico degli indirizzi viene svincolata da quello fisico e può essere maggiore;
- Aumenta il grado di multiprogrammazione avendo più processi mantenuti (anche se *parzialmente*) in memoria;

9.1.1 Memoria Virtuale

Il meccanismo principale è lo **swapping applicato alle singole pagine** invece che a interi processi.

Definizione: La Mem. Virtuale realizza la separazione o **visualizzazione** della memoria logica da quella fisica:

$$\text{Memoria Virtuale} = \text{RAM} + \text{Mem Secondaria(Disco)}$$

9.2 Paginazione su Richiesta

L'idea di base è che una pagina venga caricata in memoria solo quando è necessaria per l'exec. Questo funziona perchè si sfrutta il principio della **località dei riferimenti**

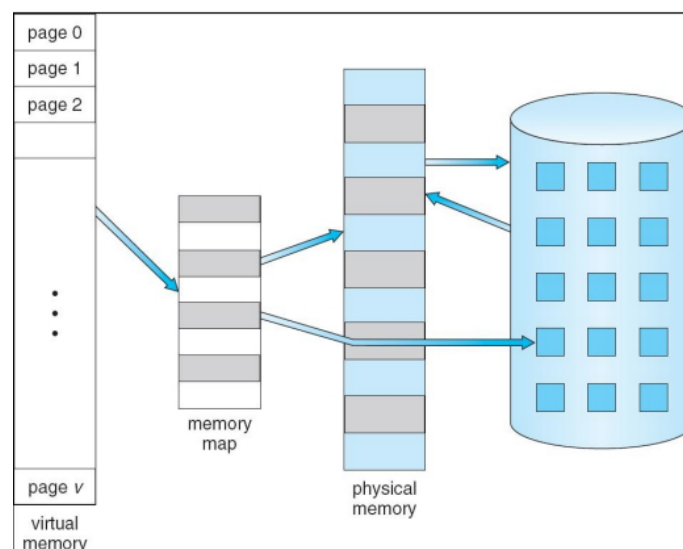
Vantaggi:

- un processo occupa meno memoria, quindi se ne possono mantenere in exec un maggior numero;
- meno operazioni di I/O quando è necessario caricare/scaricare un processo

NB. è *necessario* associare ad ogni pagina logica uno **stato**:

in memoria o non in memoria

9.2.1 Schema di Funzionamento



9.2.2 Tabella delle Pagine

In ogni entry della page table deve esserci un **bit di validità** che indica se la pagina è presente in memoria centrale (1) o secondaria con accesso negato (0).

Inizialmente tutti i bit di validità sono 0, se durante la traduzione degli indirizzi logici in fisici si trova il bit 0 si ha un **page fault**: pagina non caricata o indirizzo illegale.

9.2.3 Esempio

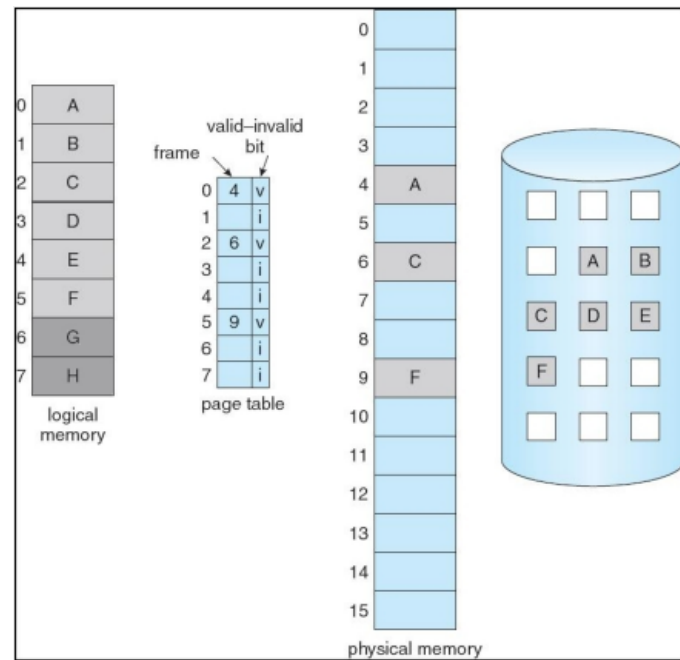


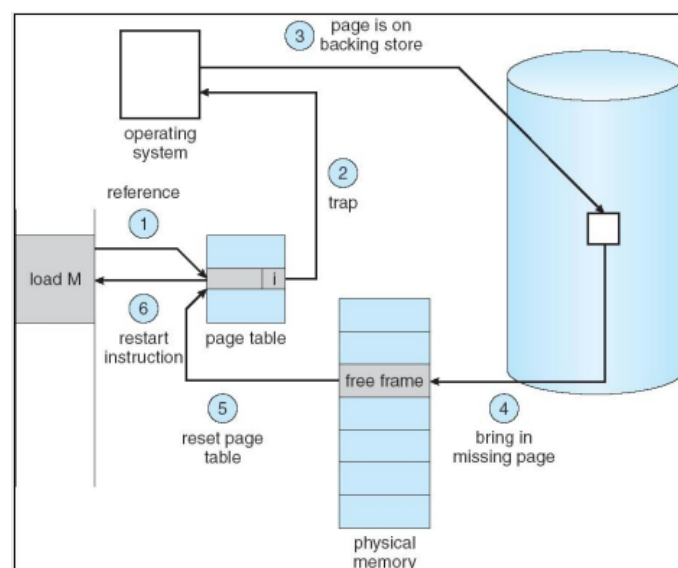
Figure 20: I riferimenti pg. 6 o 7 sono *illeciti*. – I riferimenti pg. 3 o 4 generano *page fault*

9.2.4 Gestione del Page Fault

Un page fault causa un'interruzione e il controllo passa all'OS che:

1. Accede ad una tabella interna associata al processo per determinare la causa del Fault:
 - if(illegal_access){abort / exit1} il processo viene teminato;
 - elseif(page !in memory) {attiva procedura di caricamento};
2. Cerca un frame vuoto e trasferisce la tabella dal disco al frame;
3. Aggiorna le tabelle: la page table del processo, la tabella interna associata al processo, il valid bit...
4. **Riavvia l'istruzione che ha generato il fault;**

Demand paging puro: Si ha quando il primo accesso in memoria di un programma risulta essere sempre un page fault. Esempio in cui l'OS trova un frame libero:



Problema: Il riavvio dell'istruzione generatrice del page fault potrebbe non essere banale.

Ex. Un page fault occorre durante un'istruzione di spostamento di un blocco di memoria, si supponga che l'origine e la destinazione siano "overlapping".

Come si porcede per il riavvio?

- alcuni OS operano un **ripristino del contenuto della memoria** prima di gestire il fault;
- altri iniziano l'exec di ogni istruzione **solo se sono presenti tutte le pagine necessarie**;

9.2.5 Supporto alla Paginazione su Richiesta

Questo necessita di adeguato supporto hardware:

1. page table (supporto alla paginazione);
2. parte della memoria secondaria dedicata alle pagine non caricate: **swap area**;
3. possibilità di riavviare le istruzioni in fault;

9.3 Rimpiazzamento Pagine

Se non ci sono frame liberi cosa accade? bisogna attivare il **rimpiazzamento delle pagine**, ovvero la ricerca di un frame occupato da una pagina (detta **vittima**) e scambiare suddetta con quella richiesta.

Importante notare che:

- solo se la pagina vittima è stata modificata bisogna aggiornarla anche su disco;
- è meglio selezionare una vittima tra le pagine non necessarie;
- si deve evitare di scegliere una vittima che abbia alta probabilità di essere successivamente richiesta;
- serve un algoritmo per scegliere le vittime, che massimizzi le performace minimizzando il numero di fault;

9.3.1 Prestazioni della Paginazione su Richiesta

La paginazione su richiesta influenza il tempo di accesso effettivo alla memoria: **Effective Access Time / EAT**. Se si definisce il *tasso di page fault* p come l'aprobabilità che si verifichi un fault si ha subito:

$$EAT = (1 - p) * tm + p * tg$$

con tm tempo di accesso alla memoria e tg tempo di gestione del fault.

9.3.2 Esempio

Si pongano questi valori:

- $tm = 100ns$
- $tg = 1ms = 1000000ns$

si ottiene:

$$EAT = 100 * (1 + 9999 * p)ns$$

Se si volesse peggiorare al massimo del 10% si rispetti il vincolo:

$$110 > 100 * (1 + 9999 * p)$$

Ovvero mantenere $p < 10^{-4}$ Questo porta a permettere un page frame al più ogni 10^4 accessi alla memoria principale ==> è **FONDAMENTALE** scegliere bene le page vittima.

In assenza di frame liberi sono necessari due accessi alla mem secondaria:

1. Swap-out victim page;
2. Swap-in load page;

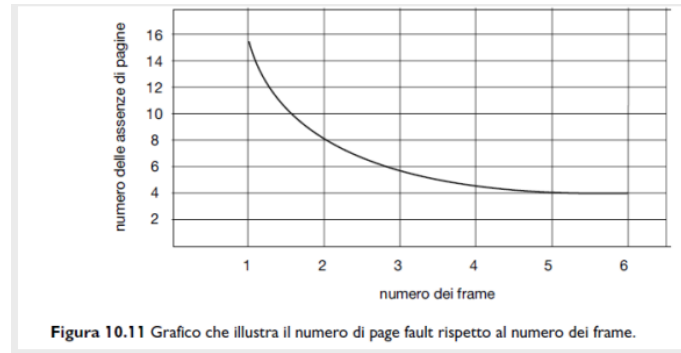
Questo determina un doppio trasferimento: doppio tempo di fault (rispetto a quanto servirebbe ci fosse un frame libero). Una possibile ottimizzazione sarebbe usare un *flag* nella page table: **bit di modifica / dirty bit**: indica se ci sono modifiche alla pagina(write only). Così si evita di mettere su disco le vittime con dirty bit = 0.

9.3.3 Problematiche

1. **Rimpiazzamento delle pagine**: se si deve rimpiazzare una pagina, come scelgo la vittima?
2. **Allocazione dei frame**: quanti frame assegnare ad ogni processo?

9.3.4 Frequenza di page Fault

Notare che la frequenza di fault é in genere inversamente proporzionale al numero di frame a disposizione



9.4 Algoritmi di Rimpiazzamento Pagine

L'obiettivo è ottenere il minor numero di fault, gli algoritmi possono essere confrontati tramite il loro operato rispetto alle sequenze di accessi in memoria. Alcuni di questi accessi generano fault; invece di considerare gli indirizzi di memoria è meglio ragionare sulle pagine di memoria che contengono tali indirizzi.

Per esempio se la sequenza di indirizzi fosse:

1050 7076 1100 1040 2095 2210 2058 3111

con una dimensione delle pagine da 1KB, allora gli accessi sarebbero:

pagine: 1 6 1 1 2 2 2 3

gli accessi consecutivi, tuttavia, generano al massimo un fault quindi la sequenza da considerare:

1 6 1 2 3

Passando in rassegna i vari algoritmi di rimpiazzamento confrontati con la **sequenza di riferimenti pagine** si ottiene:

- Sequenza:

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

- assumendo **3 frame** a disposizione;
- I vari algoritmi si caratterizzano definendo il criterio di:

selezione della vittima

9.4.1 FIFO

Si seleziona la pagina presente da **più tempo** in memoria, questo algoritmo è:

- semplice da implementare;
- è *cieco*, ovvero non considera l'effettivo effetto delle pagine;
- tende ad aumentare la frequenza di page fault;
- soffre della **Anomalia di Belady**



Figure 21: In questo esempio si ottengono **15 page fault**

9.4.2 Anomalia di Belady

Considerando FIFO applicato sulla seguente sequenza con 3 frame:

♦ Reference string											
1	2	3	4	1	2	5	1	2	3	4	5
3 frame 9 page fault											
1	1	1	4	4	4	5	5	5	5	5	5
	2	2	2	1	1	1	1	1	3	3	3
		3	3	3	2	2	2	2	2	4	4

Nel tentativo di diminuire la frequenza di fault si aumentano a 4 i page frame:

4 frame 10 page fault											
1	1	1	1	1	1	5	5	5	5	4	4
	2	2	2	2	2	2	1	1	1	1	5
		3	3	3	3	3	3	2	2	2	2
			4	4	4	4	4	4	3	3	3

9.4.3 Algoritmo ottimale

Viene selezionata la pagina che sarà usata **più tardi** in futuro, questo garantisce il numero *minimo* di fault. Però è complicato da implementare e bisogna conoscere in anticipo la sequenza di riferimenti.

Ex.

successione dei riferimenti											
7	0	1	2	0	3	0	4	2	3	0	3
2	1	2	0	3	2	1	2	0	1	7	0
1	7	0	1	1	3	3	2	1	2	0	1
frame delle pagine											
7	7	7	2	2	2	2	2	2	2	7	7
	0	0	0	0	4	0	0	0	0	0	0
		1	1	3	3	3	3	1	1	1	1

Figura 10.14 Algoritmo ottimale di sostituzione delle pagine.

Si ottengono 9 fault.

9.4.4 LRU: Least Recently Used

Viene selezionata la pagina che **non è** usata da più tempo. Approssima l'algoritmo ottimale sfruttando il passato per vedere il futuro, anch'esso difficile da implementare dovendo tenere traccia dei tempi di accesso alle pagine e necessitando di supporto hardware.

successione dei riferimenti											
7	0	1	2	0	3	0	4	2	3	0	3
2	1	2	0	3	2	1	2	0	1	7	0
1	7	0	1	1	3	3	2	1	2	0	1
frame delle pagine											
7	7	7	2	2	4	4	4	0	1	1	1
	0	0	0	0	0	0	3	3	3	0	0
		1	1	3	3	2	2	2	2	2	7

Figura 10.15 Algoritmo di sostituzione delle pagine LRU.

Si ottengono 12 fault.

9.4.5 Implementazione LRU

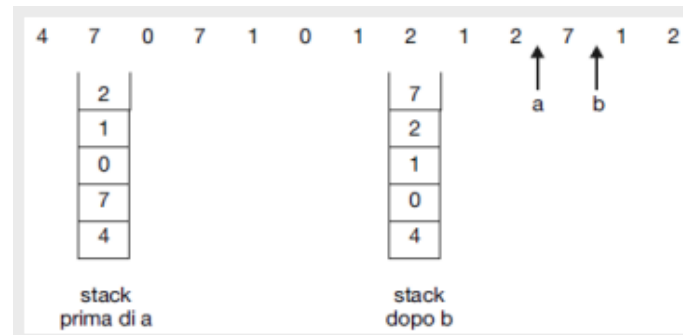
Come si può trovare la pagina usata meno di recente?

Tramite counter:

- counter associato ad ogni pagina con un timestamp ad ogni referenza;
- cercare la pagina con il timestamp più vecchio;

Tramite stack

- ad ogni riferimento si inserisce il numero di pagina sullo stack;
- e si elimina l'eventuale altra occorrenza nello stack;
- selezionare la pagina in fondo allo stack;
- *deve esser fatto a livello HW*;



9.4.6 Approssimazioni LRU

Uso reference bit: un bit associato ad ogni pagina (init = 0), ad ogni referenza si pone = 1, si seleziona una delle pag con bit = 0.

più reference bit: si usano k bit per pagina, ad intervalli regolari i reference vengono salvati. K bit interpretati come un valore intero, si seleziona la pagina con il valore inferiore.

NUR - Not Used Recently: si usano i reference bit R e dirty bit M, ad ogni referenza si pone R a 1 e ad ogni modifica M a 1. Periodicamente si riazzerano gli R, e viene considerata MR come un binario $C \in \{0, 1, 2, 3, \}$ e selezionata la pag con val(C) inferiore.

9.4.7 Tecniche basate su conteggio

LFU - Last Frequently Used: ad ogni riferimento si incrementa un counter e viene selezionata la pagina con il counter minore.

MFU - Most Frequently Used: opposto di LFU, motivato dal fatto che una pagina appena caricata ha un counter basso.

9.4.8 Algoritmo di seconda Chance

E' una variante di FIFO in cui si tiene conto del reference bit. L'intento é evitare di selezionare una pagina anche se "vecchia", quindi si considerano le aigne in coda circolare in base a tempo di caricamento. Pagina riferita = ref bit = 1, la vittima è la più vecchia.

- se bit = 0 si seleziona;
- se 1 si concede una seconda chance, azzerando il bit e passando alla pagina seguente;

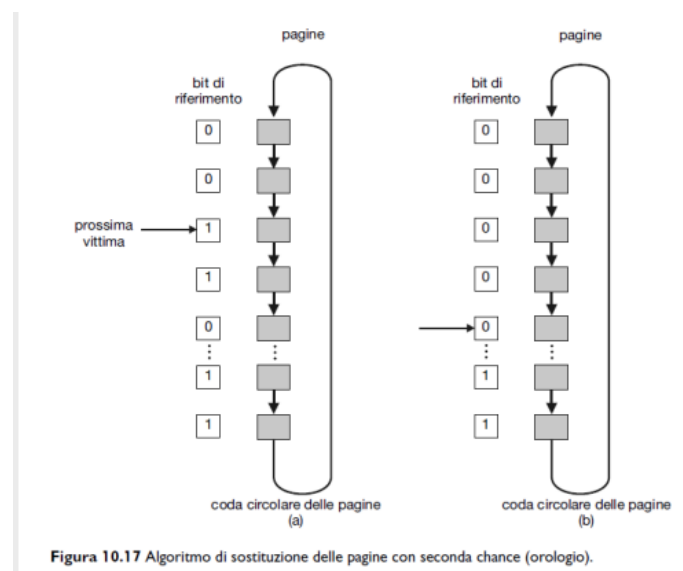


Figura 10.17 Algoritmo di sostituzione delle pagine con seconda chance (orologio).

9.5 Allocazione del Frame

Supponendo di caricare (parzialmente) in memoria N processi e poter usare M frame:

- si deve decidere quanti frame assegnare ad ogni processo, ci potrebbero essere vincoli sul numero minimo di frame;
- l'istruzione che ha causato il page fault deve essere riavviata, quindi tutte le pagine che usa devono essere presenti;
- il numero di frame deve essere sufficiente ad eseguire l'istruzione che prevede il maggior numero di riferimenti possibile:
 - per esempio, se il linguaggio macchina prevede indirizzamento indiretto, allora servono almeno tre pagine o frame: istruzione, dato riferito, altro dato;
 - altro esempio, si consideri un'istruzione che fa copy/paste, da memoria a memoria: l'istruzione potrebbe essere a cavallo di due pagine, sorgente e destinazione potrebbero essere indirizzamenti indiretti: servono 6 frame.

9.5.1 Schemi di allocazione

Supponendo di dover allocare M frame, suddividendoli tra N processi ecco due semplici strategie: **Uniforme**: è una allocazione in cui a ciascun processo si concedono M/N frame.

Proporzionale: si considera la dimensione D_i di ogni processo P_i , a questo processo si concede una frazione del N frame proporzionale alla sua dimensione: $M \cdot \frac{D_i}{\sum_j D_j}$.

9.5.2 Contesto di rimpiazzamento

Si supponga che un processo P generi un fault, e sia necessario selezionare una vittima:

contesto locale: la pagina vittima viene scelta tra le pagine di P

contesto globale: la pagina vittima viene scelta tra le pagine di tutti i processi

9.5.3 Thrashing

Si consideri un sistema in cui il numero di frame allocati ai processi è troppo basso, il numero di fault tenderà quindi ad aumentare. Avendo pochi frame il sistema tenderà a selezionare le vittime che molto probabilmente verranno riferite nei successivi accessi. Così facendo i processi spendono più tempo nell'attesa e gestione dei fault, quindi gli stati waiting sono maggiori dei ready. Questo porta anche a sottoutilizzare la CPU e l'OS per controbilanciare aumenta il grado di multiprogrammazione, peggiorando la situazione: + processi = + fault.

In una situazione simile il sistema crolla in performance e viene chiamata **thrashing**:

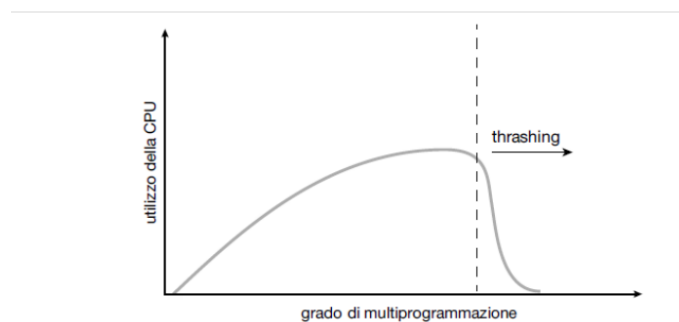


Figura 10.20 Thrashing.

9.5.4 Località dei riferimenti

NB. Nella situazione precedentemente descritta, per evitare il thrashing sarebbe stato necessario diminuire il grado di multiprogrammazione, usando ad esempio lo swapping al fine di assicurare i frame necessari ai processi presenti in memoria. Per definire il numero corretto di frame per processo si considera un **modello di località** dell'exec.

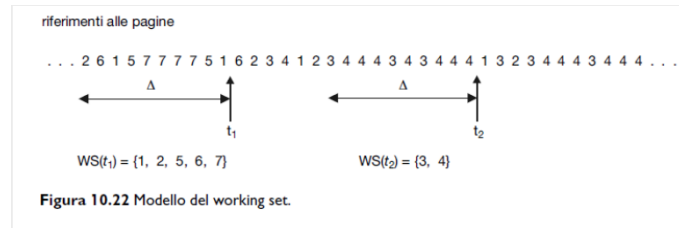
- **Località Temporale**: locazioni riferite recentemente hanno buona probabilità di essere riferite nuovamente;
- **Località Spaziale**: se si è riferita una locazione è probabile che si riferiranno anche le locazioni ad essa vicine;

9.5.5 Modello del Working set

La tecnica del **working set** mira a determinare la dimensione della località attuale del processo, $\forall P_i$ processo, si definisce una *finestra di working set* considerando gli ultimi Δ riferimenti che P_i ha effettuato. Così si definisce il **working set** WS_i come l'insieme delle pagine riferite negli ultimi Δ accessi; la dimensione del set stima il numero di frame necessari a P_i .

Se l'OS rileva che il numero di frame a disposizione non soddisfa la somma del working set size ($\sum_i WSS_i$) allora si deve diminuire il grado di multiprogrammazione.

Ex.



con $\Delta = 10$ si ha:

1. all'istante t_1 un $WS(t_1) = \{1, 2, 5, 6, 7\}$ e quindi $WSS(t_1) = 5$;
2. all'istante t_2 un $WS(t_2) = \{3, 4\}$ e quindi $WSS(t_2) = 2$;

La scelta di Δ è importante:

- se troppo piccolo non si cattura bene la località;
- se troppo grande si unisce località differenti concedendo più frame del necessario al processo;

La valutazione al runtime dei WS e delle WSS può comportare inefficienza, un modo per approssimare il WS prevede di usare i reference bit:

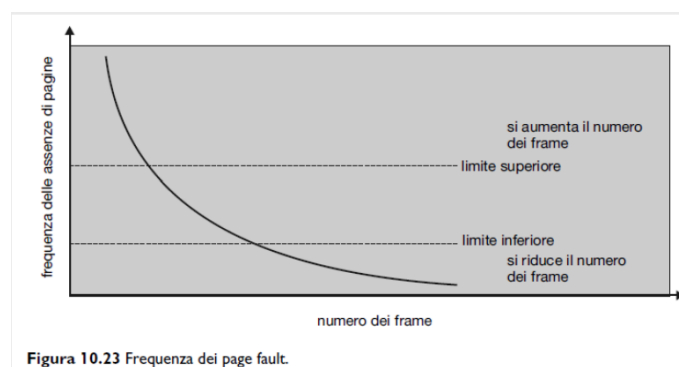
- a intervalli regolari si salvano i reference bit delle pagine e si azzerano;
- le pagine che hanno almeno un bit a 1 tra quelli salvati negli ultimi k intervalli compongono un'approssimazione del WS;

9.5.6 Page fault Frequency

Una tecnica alternativa è basata sul monitoraggio della **frequenza di fault**:

fissando una soglia minima alla frequenza, porta a indicare (se la soglia non viene superata) che si può sottrarre frame al processo.

Fissandone una massima porta a indicare che il processo è sofferenza e si devono concedere più frame. Una frequenza in mezzo alle soglie è ovviamente accettabile.



9.5.7 Compressione della Memoria

Ci sono situazioni in cui non è possibile effettuare né swapping né replace delle pagine, ciò è il caso degli OS per dispositivi mobili come Android e IOS. In queste situazioni si sostituisce la paginazione virtuale con la **compressione della memoria**.

Quando è necessario liberare della memoria si selezionano k frame e si salvano in formato compresso in uno dei frame liberi che viene messo nella lista dei frame compressi, così si liberano $k - 1$ frame. Se uno dei frame viene riferito si attua decompressione, l'efficienza della tecnica dipende dalla efficienza degli algoritmi di compressione.

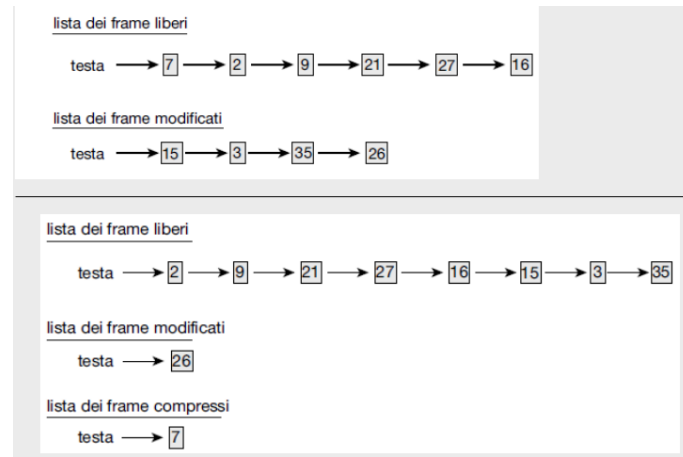


Figure 22: Esempio di Compressione

10 Disk Scheduling

10.1 Struttura di un disco

Un disco si compone di un settore: (unità + info che leggi/scrivi)

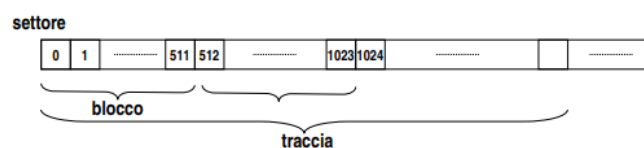
- dimensione: variabile tra 32 byte e 4KB (di solito 512 byte);
- raggruppati dal punto di vista logico in cluster (motivi di efficienza):
 - dimensione dipendente dalla grandezza del disco e dall'OS (da 2KB a 32KB);
 - Ex. HDD 60G = cluster da 4KB su WinXP e Linux;
- un file occupa sempre almeno un cluster;

Per accedere ad un settore bisogna specificare la superficie, la traccia e il settore corrispondente.

Vista logica: un vettore unidimensionale di *blocchi logici*. Blocco = cluster = unità minima di trasferimento.

Questo vettore viene mappato sequenzialmente su settori del disco:

- Settore 0 = primo settore prima traccia del cilindro;
- la numerazione procede gerarchicamente per settori, tracce, cilindri



10.2 Tempo di accesso a disco

Si suddivide in:

1. Seek time: tempo necessario a spostare la testina sulla traccia;
2. Latency time: posizionare il settore desiderato sotto la testina (dipende dalla velocità di rotazione);
3. Transfer time: tempo del settore per passare sotto la testina (actually READING!);

Tempo di accesso = Seek time + Latency + Transfer Time

Valori tipici:

- SeekTime: 5-10ms
- Latency: in media = $0.5 * (60 / \text{velocità di rotazione}) = 0.5 / \text{vel.rot.in secondi}$
- Rotation: 5400rpm, 7200rpm
- Transfer time: dimensione blocco/ velocità di trasferimento

Esempio pratico:

- Transfer time: 40Mb/s
- RPM = 10000 RPM (166rps)
- Rotazione media: 1/2 traccia
- Dim blocco: 512 Byte
- T_{seek} : 5ms

$$T_{accesso} = 5 \text{ ms} + 0.5/166 + 0.5\text{KB} / 40\text{MB} = 5 \text{ ms} + 3 \text{ ms} + 0,0000125 = 8.0000125.$$

10.3 Disk Scheduling

Dato il grande numero di processi nel sistema che accedono a disco é necessario definire uno scheduler.
 Funzione di costo = rifusione dello spostamento della testina (minor seektime).
 Banda = n° byte trasferiti / tempo.

10.4 Algoritmi di Disk Scheduling

Al solito si cerca un buon compromesso tra costo ed efficacia, si analizzano i seguenti:

- FCFS
- SSTF
- SCAN, C-SCAN
- C-LOOK
- LIFO

Per semplicità verranno tutti sottoposti alla seguente sequenza di esempio:

- 98, 183, 37, 122, 14, 124, 65, 67
- Inizialmente la testina su traccia 53
- Intervallo dei valori = [0, 199]

10.4.1 FCFS

Le richieste vengono processate nell'ordine di arrivo:

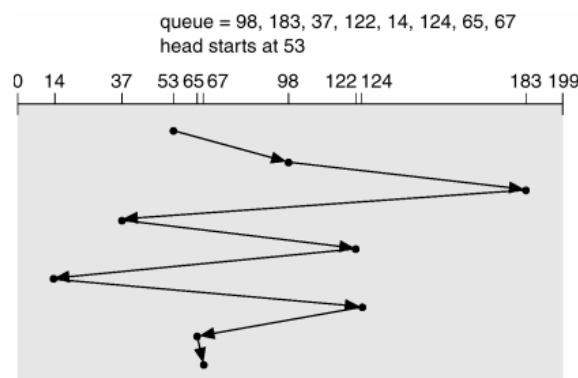


Figure 23: Spostamento testina **640 Tracce**

10.4.2 SSTF

E' la scelta più efficiente, si seleziona la richiesta con il minimo spostamento rispetto alla posizione attuale della testina. L'acronimo sta per *Shortest Seek Time First* (simile a SJF), questa soluzione migliora FCFS ma non é ancora la scelta ottimale, perchè c'è ancora la possibilità di *starvation*

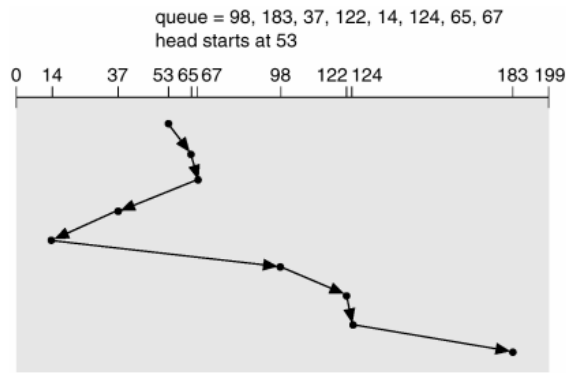


Figure 24: Spostamento testina **236 Tracce**

10.4.3 SCAN

Se si osserva la natura dinamica delle richieste si trova "SCAN" (Algoritmo dell'ascensore) in cui:

- La testina parte da un'estremità del disco;
- Si sposta verso l'antipodo, scrivendo le richieste correnti;
- Arrivata all'altro estremo riparte in direzione opposta scrivendo le nuove richieste

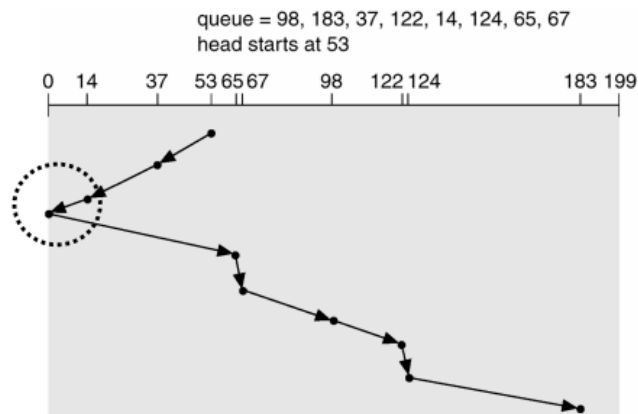


Figure 25: Spostamento testina **236 Tracce** (direzione decrescente)

10.4.4 SCAN variante 1

Una variante é il C-SCAN (circolare), funziona come SCAN ma all'arrivo ad una estremità la testina riparte immediatamente da 0 senza servire altre richieste.

Ha un tempo di attesa più uniforme rispetto a SCAN, alla fine di una scansione ci saranno più richieste all'altro estremo:

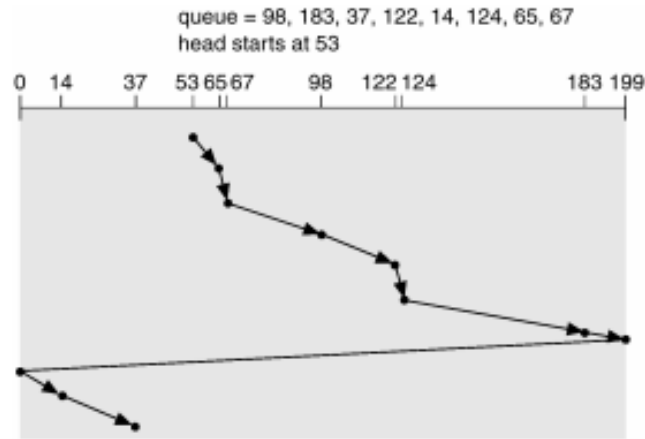


Figure 26: Spostamento testina **382 Tracce**

10.4.5 SCAN variante 2

Seconda variante é il LOOK/C-LOOK, la testina in questo caso non raggiunge l'estremità del disco, ma cambia direzione(LOOK) o riparte dalla prima traccia(C-LOOK) non appena le richieste in quella direzione terminano:

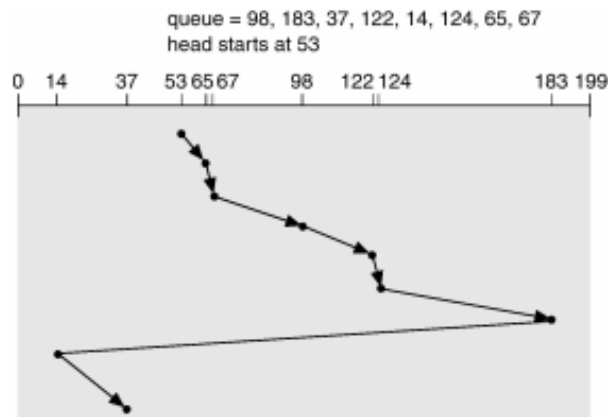


Figure 27: Spostamento testina **322 Tracce**

10.4.6 SCAN variante 3

Terza e ultima variante presa in considerazione é il N-step-SCAN, impiegata per evitare che la testina rimanga sempre nella stessa zona.

La coda delle richieste viene infatti partizionata in più di dimensione N. Al processo della coda gli accessi in arrivo riempiono le altre code (riempita una non si possono modificare le richieste interne).

Le code piene vengono servite nello SCAN successivo:

- N grande = degenera in SCAN;
- N = 1 degenera in FCFS;

(Piccola variazione é il FSCAN, simile con solo due code.)

10.4.7 LIFO

In alcuni casi è più utile schedare gli accessi in base all'ordine inverso di arrivo (Last In First Out), è utile con accessi con elevata località(Ex. molti accessi "vicini"). Ma c'è possibilità di starvation.

11 Il File System (FS)

Il **Sottosistema per la gestione dei file** è la parte più visibile di un OS. Questo fornisce meccanismi per registrazione, accesso e protezione di dati/programmi; visualizza i dispositivi di memorizzazione come HDD, SSD, USB ecc... E astrae

la complessità di utilizzo dei diversi mezzi di memorizzazione secondaria fornendo una visione logica e metodi di accesso uniformi. La struttura logica con cui l'OS gestisce i file prende il nome di FS, l'organizzazione gerarchica con cui l'OS gestisce l'archiviazione dei file (HFS - Hierarchical File System) non ha alcun legame con la posizione fisica dei file in mass memory.

Il **File system** è responsabile della gestione dei file in mass memory:

- struttura i dati in files;
- li organizza in dir;
- fornisce a User un insieme di funzioni di alto livello per operare su questi, mascherando le reali operazioni di basso livello (Malloc);

Il FS, garantisce una gestione dei file indipendente dalle caratteristiche fisiche dei dispositivi che costituiscono la MassMem.

11.1 File

Si definisce FILE un insieme di informazioni correlate e registrate nella memoria secondaria, a cui è stato assegnato un nome.

Dal punto di vista USER:

- è la più piccola parte di memoria secondaria indirizzabile logicamente;
- i dati possono essere scritti nella memoria secondaria soltanto all'interno di un file;

Dal punto di vista OS: i file vengono mappati su dispositivi fisici di memorizzazione non volatili.

Tipi di files:

- **Dati:** numerici, alfanumerici, binari;
- **Programmi:** Codice sorgente, codice oggetto, codice eseguibile;

11.1.1 Struttura dei file

Se non è presente il file è una semplice sequenza di byte.

Sequenza di Record:

- Linee;
- Record a lunghezza fissa;
- Record a lunghezza variabile

Struttura complessa:

- Documento formattato;
- File eseguibile rilocabile;

La struttura viene decisa o dall'OS o dall'applicativo che crea il file, in generale un file è formato da una sequenza di bit, byte, righe o record, il cui significato è definito dal creatore del file stesso.

Il **tipo** del file è ciò che solitamente ne definisce la struttura, tutti gli OS devono prevedere un tipo di file: **.exe**. Alcuni Sistemi prevedono la gestione diretta di diverse strutture file e mettono a disposizione operazioni specifiche

11.1.2 Attributi dei file

I seguenti:

- **Nome:** identificativo del file per lo user;
- **Identificativo:** etichetta unica (numero) che identifica il file all'interno del FS;
- **Tipo:** necessario per i sistemi che supportano tipi differenti;
- **Localizzazione:** puntatore al dispositivo e alla posizione del file sul dispositivo;
- **Protezione:** parametri di controllo per l'accesso in lettura, scrittura ed exec;
- **Ora, data e ID utente:** dati necessari alla sicurezza del sistema e per il controllo d'uso;

Le info sui file sono conservate nella struttura di **Directory** che risiede sulla memoria secondaria.

11.2 Operazioni sui file

Un file è un **tipo di dato astratto** su cui sono definite le operazioni di:

- Creazione;
- Lettura / Scrittura;
- Posizionamento nel file (**file seek**);
- Cancellazione;
- Troncamento;
- Impostazione degli attributi;

Creazione: riempimento del file nel FS, rilascia lo spazio allocato al file e elimina il corrispondente elemento della dir.

Troncamento: cancellazione del contenuto del file, ma parte degli attributi restano immutati.

impostazione degli attributi: riempimento ed aggiornamento del relativo elemento di dir.

Le operazioni su file richiedono solitamente una ricerca dell'elemento associato al file all'interno della struttura delle dir. L'OS mantiene in mem centrale una tabella contenente info su tutti i file aperti: **tabella dei file aperti**

Una volta richiesta un'operazione su file si ricercano le info relative tramite un indice nella tabella sopracitata, quando il file non è più in uso deve essere chiuso e l'OS rimuove l'elemento relativo nella tabella.

Apertura e chiusura file:

- $\text{open}(F_i)$: ricerca nella dir su disco l'elemento F e ne copia il contenuto nella tabella file aperti restituendo un puntatore all'elemento tabulato;
- $\text{close}(F_i)$: copia il contenuto dell'elemento F residente in memoria principale nella sua dir su disco e lo rimuove;

11.2.1 File aperti

Nei sistemi multiuser di cono due lvl di tabelle:

Tabella di Sistema e Tabella associata a processo

Sistema:

- riferimenti a tutti i file aperti nel sistema;
- posizione e dimensione del file nel disco;
- date di ultimo accesso/ultima modifica;
- counter di aperture;

Processo:

- riferimenti a tutti i file aperti dal processo;
- puntatore alla posizione corrente nel file;
- diritti di accesso e info di contabilizzazione;
- gli elementi della tabella dei file associata al processo puntano al relativo elemento contenuto nella tabella di sistema;

Da notare bene che:

Puntatore alla posizione corrente nel file: manda all'ultima locazione dove è stata realizzata un'operazione di R/W per ogni processo che ha aperto il file.

Contatore di aperture: torna il numero di processi che hanno aperto il file, per rimuovere i dati dalla tabella qualora venga chiuso il file.

Locazione del file su disco: cache delle info di accesso ai dati permanenti.

Diritti di accesso: controllati dall'OS per permettere o negare le operazioni di I/O richieste.

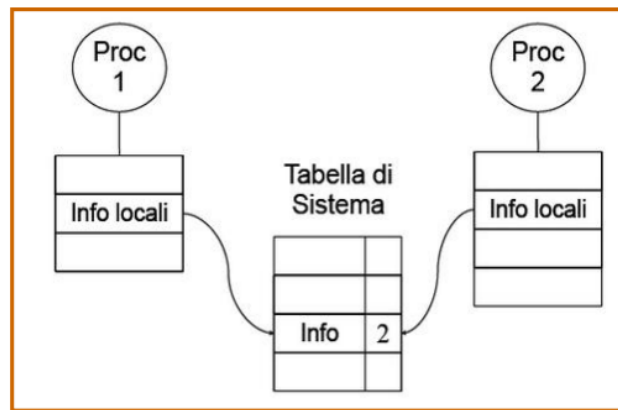


Figure 28: Una entry dalla tab di sistema può essere rimossa quando il counter = 0

11.2.2 Lock dei file aperti

Questa feature è garantita da qualche OS e FS, offre una mediazione per l'accesso condiviso a file. Obbligatorio o Consigliato:

- **Obbligatorio:** l'accesso a file viene negato se il lock è già stato acquisito da un altro processo (Windows);
- **Consigliato:** i processi trovano che lo stato di un file è bloccato e decidono su cosa fare (UNIX);

Se il lock è obbligatorio l'OS assicura integrità dei dati soggetti a lock. Se invece è consigliato il programmatore deve garantire la corretta acquisizione e cessione di lock.

11.2.3 Tipi di file

Il tipo di file è un attributo che ne indica la struttura logica e permette di interpretarne correttamente il contenuto, alcuni OS gestiscono diversi tipi di file. Conoscendo il tipo si possono evitare errori comuni quali stampare un file.exe.

Un OS che riconosce il tipo del file lo può manipolare in modo "mirato". Tre sono le tecniche di gestione dei tipi di file:

- **Meccanismo delle estensioni:** il tipo è indicato da suffisso del nome;
- **Attributo "tipo" associato al file nella dir=** MAC OS;
- **Magic number:** il tipo è rappresentato da un valore posto all'inizio del file (UNIX).

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

Esempi:

- In **MS-DOS** possono essere eseguiti solo: .COM, .EXE, .BAT;
- Su **Apple** ogni file possiede un attributo di creazione indicativo al programma che lo ha creato;
- **UNIX** memorizza un *magic number* per indicare il tipo di file, usa le estensioni solo come suggerimento.

11.2.4 Supporto alla struttura dei file

Il tipo di un file e la corrispondente struttura logica sono riconosciuti e gestiti in modi diversi dai vari OS. Se questi gestiscono molti formati il codice di sistema è più ingombrante, ci potrebbe essere incompatibilità tra programmi con

file di formato differente ma hanno una gestione efficiente per i formati supportati. Altrimenti il codice sistema è più snello ma i formati devono essere gestiti dal programmatore.

Esempio: Unix attua una scelta minimale, i file vengono considerati semplici stringhe di byte e solo i quelli eseguibili hanno un formato predefinito dall'OS.

11.2.5 Struttura interna dei file

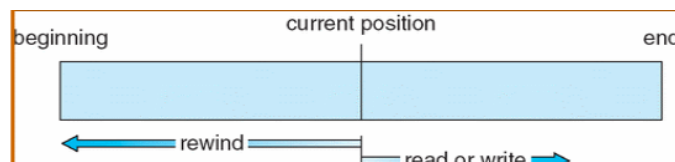
La dimensione dei **blocchi** del disco, detti anche *record fisici* è fissa, occorre risolvere il problema della corrispondenza tra record logici e fisici (*packing*)



Figure 29: Parte dell'ultimo blocco fisico contenente il file rimane inutilizzata: frammentazione interna

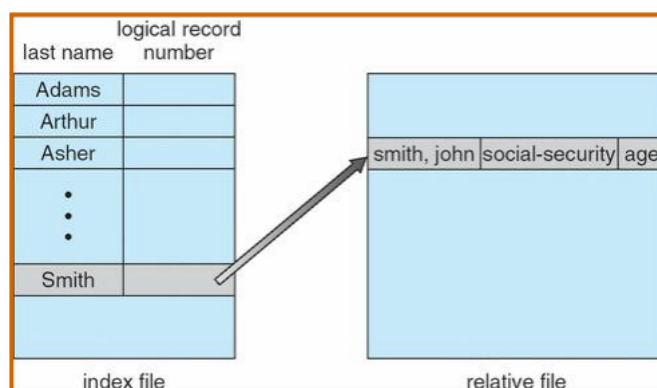
11.2.6 Modalità di accesso

L'accesso **sequenziale** è un modello di accesso a file che si ispira al nastro. Si può accedere solo al record - blocco successivo:



L'accesso **diretto**: è un modello di accesso a file ispirato al disco, si può accedere ad ogni blocco - record tramite il numero di blocco relativo

L'accesso **indicizzato** può essere realizzato sulla base del metodo ad accesso diretto, implica la costruzione di un file indice per l'accesso. Questo indice contiene puntatori ai blocchi, per reperirne un elemento occorre cercare il puntatore realizzato interno all'indice.



11.3 Directory

Si può interpretare directory come una tabella che ha un elemento per ogni nome di file, questo elemento permette di reperire gli attributi del file e le info per accedervi al contenuto. Come i file anche le directory risiedono su disco

11.3.1 Info e operazioni sulle Directory

Informazioni:

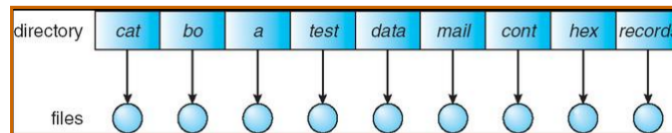
- Nome, Tipo, Indirizzo;
- Lunghezza attuale e massima;
- Data ultimo accesso e ultima modifica;
- ID proprietario e info di protezione;

Operazioni:

- Ricerca di un file;
- Creazione / Cancellazione di un file;
- Elenco dei contenuti di una dir;
- Ridenominazione di file;
- Attraversamento del file system;

11.3.2 Directory a un livello

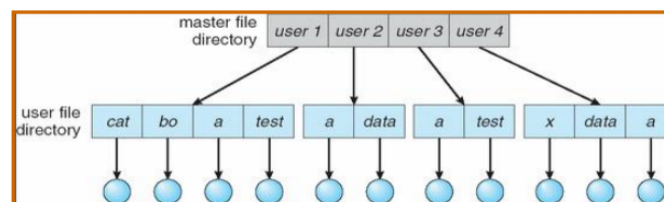
Una directory unica per tutti gli utenti



- Tutti i file di tutti gli utenti in un'unica directory
- Problemi di naming (va scelto un nome diverso per ogni file)
- Nessun raggruppamento logico
- Problemi accentuati con numero grande di file

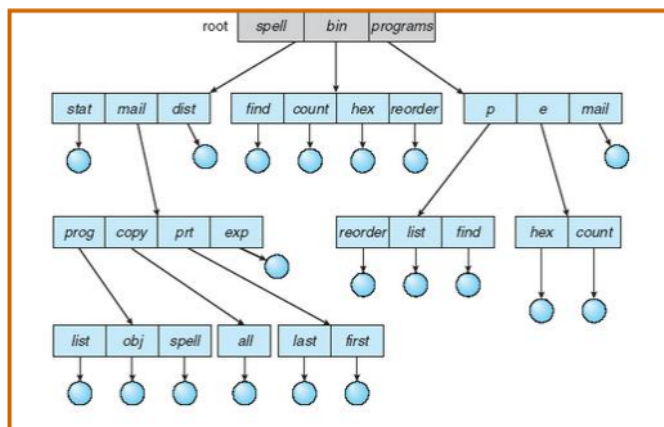
11.3.3 Directory a due livelli

Directory separate per ciascun utente



- Ammessi nomi uguali per file di utenti diversi
- Ricerca più efficiente
- Nessuna capacità di raggruppamento logico (solo in base a proprietari) Permette a un utente di individuare file di altri utenti (pathname: utente/file)
- Ricerca dei FS: percorso di ricerca
- Quando si apre sessione di lavoro di ricerca nella Master File Directory l'ID utente, ammesso poi al sistema all'interno della propria User File Directory
- Ogni riferimento a file da parte dell'utente viene interpretato da OS come esclusivamente correlato a file presenti nella relativa UFD
- Per riferirsi a file di altri utenti (se l'accesso è autorizzato) ogni utente deve utilizzare pathname completo dei file (nome utente, nome file)
- FS vengono raccolti in opportune directory raggiungibili da tutti gli utenti

11.3.4 Directory con struttura ad Albero

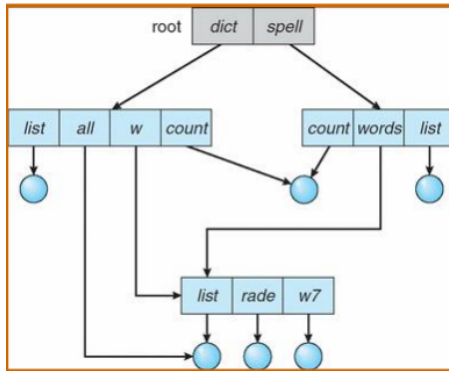


- Ricerca efficiente
- Capacità di raggruppamento logico

- Concetto di pwd (directory corrente)
- Pathname **assoluto** o **relativo**
- Creazione di nuovo file o cancellazione di un file esistente viene effettuata nella pwd
- Creazione di nuova directory viene interpretata come creazione di sottodirectory della pwd
- Cancellazione di una directory ammessa solo se directory è vuota oppure cancella anche tutto il suo contenuto

11.3.5 Directory a grafo aciclico

Vi è presenza di file e sottodir condivisi, l'**Aliasing** è un concetto fondamentale: più nomi diversi possono venir utilizzati per identificare lo stesso oggetto.

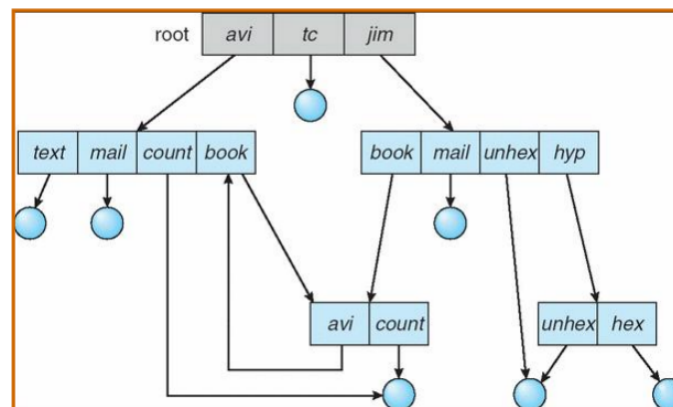


- La condivisione può essere implementata per mezzo di **link** o per duplicazione dell'elemento di dir;
- Il problema è che se si cancella la W/list il puntatore diventa dangling;

• Soluzioni:

- Usare puntatori all'indietro che consentano reperimento e cancellazione di tutti i link a file cancellato
- Conservazione del file fino a che non esistano più link (lista dei riferimenti a file, numero di riferimenti: quando contatore è 0 il file può essere cancellato)
- Vi sono **"nuovi oggetti"** contenuti nelle directory e necessitano di opportune operazioni:
 - Creare/cancellare un **link**, cioè un nome diverso (per un file già esistente)
 - **Risolvere un link** = seguire il puntatore per reperire il file

11.3.6 Directory a grafo generico



Evitare cicli o ammettere cicli?

- Mantenere grafo aciclico garantisce semplicità degli algoritmi necessari per attraversarlo
- Proibire cicli significa ammettere link a file ma non a directory
- Permessi link a directory, ma ad ogni aggiunta di link si deve verificare assenza di cicli
- Altrimenti di devono gestire i cicli → **Garbage collection**: per implementare cancellazione di parti non più riferite si attraversa FS marcando i file accessibili e in un secondo passaggio si rimuovono i file non marcati

11.4 Montaggio di un FS

- Un FS deve essere **montato** prima di poter essere acceduto da processi di un sistema
- Un FS *unmounted* può essere montato ad un **mount point** prescelto
- Alcuni OS richiedono FS di tipo prefissato, altri ne supportano diversi e sondano strutture del dispositivo/i per determinare tipo del/i file system presente/i (o richiedono all'utente di specificare il tipo)

11.4.1 Procedura di montaggio

1. Si fornisce all' OS nome del dispositivo da montare e locazione che dovrà assumere nella struttura del FS (solitamente punto di montaggio è una directory vuota cui sarà agganciato il FS da montare)
2. Una volta montato il FS risulta accessibile a programmi utente in modo trasparente e diventa parte integrante del grafo delle directory
3. Directory su cui viene montato un FS può non essere vuota: nel momento in cui si effettua il montaggio i dati ivi contenuti non risultano visibili fino all'unmount

Esempio: Si effettua montaggio su */users* del FS in figura (b) identificato (per esempio) da */device/usr* → le home degli utenti *bill* e *fred* diventano inaccessibili

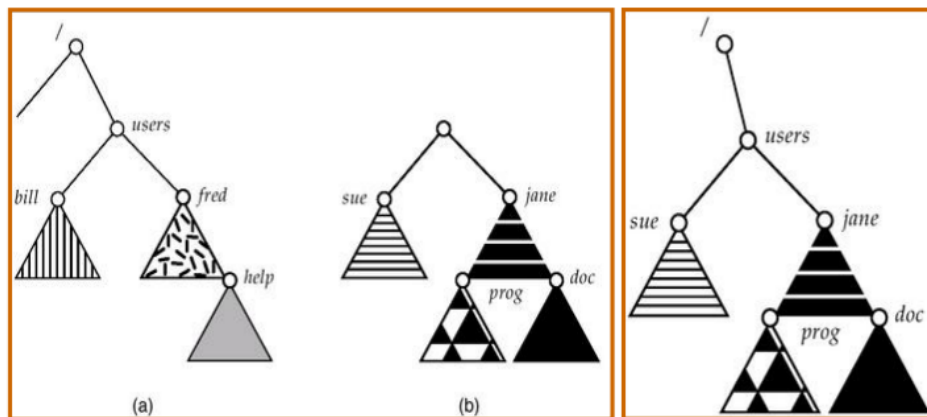


Figure 30: File System esistente – File System unmounted – */users* è il mount point

11.4.2 Considerazioni

- Sistemi Mac e Windows rilevano tutti i dispositivi all'avvio della macchina e montano automaticamente tutti i FS in essi contenuti
- Nei sistemi UNIX-like i FS devono essere montati esplicitamente:
 - Un file di conf. del sistema contiene una lista di dispositivi e relativi punti di montaggio da montare automaticamente all'avvio; mount può essere effettuato anche durante sessione di lavoro:
\$ mount /dev/usr /users
\$ unmount /dev/usr
 - in Linux: */etc/fstab* è il file di configurazione di sistema per descrizione statica dei dispositivi di memoria collegati al pc il cui mount deve essere effettuato all'avvio

11.5 File Sharing

- Il file sharing permette la collaborazione tra utenti/processi e riduce le risorse richieste per raggiungere un obiettivo di calcolo
- Sono necessari meccanismi di protezione per garantire il controllo dell'accesso ai file mediato dall'OS
- Nei sistemi distribuiti la condivisione avviene attraverso la rete
- **NFS** (Network File System) permette di realizzare la condivisione di file in ambienti distribuiti

11.5.1 File Sharing multiutente

Modello che utilizza i concetti di owner e di group di user "affini" all'owner:

- l'owner è lo user che possiede il file e che può cambiare gli attributi del file (o dir)
- un group definisce il sottoinsieme di user autorizzati a condividere l'accesso a file
- gli identificatori del group (**GID**) e del proprietario (**UID**) di un file (o dir) sono memorizzati come attributi del file (nel relativo file dir)

11.6 File System remoti

Uso della rete per ottenere accesso a file residenti su sistemi remoti, si può realizzare in diversi modi:

- **Trasferimento**
- **DFS** (File system distribuito), che permette la visibilità e l'accesso (automatico) dal calcolatore locale ad una remote dir
- accesso tramite **browser** (semi-automatico) **FTP**, anonimo, non è esplicito ma è *wrapped*, nell'operazione di download

Nel **modello client-server**, il server mette a disposizione risorse (sotto forma di dir o file) ai client che ne fanno richiesta.

- **Modello multi-a-molti**: un server può gestire richieste provenienti da più client, il client può accedere a più server
- Problemi di **autenticazione**:
 - Protocolli insicuri che possono condurre a spoofing
 - Sicurezza ottenuta mediante autenticazione reciproca di client e server tramite key di cifratura

Possibili problemi: compatibilità fra client e server (relativamente all'algoritmo di cifratura) e scambio sicuro delle chiavi.

NFS è il protocollo client-server standard di UNIX:

- gli **UID** devono coincidere nel client e nel server
- dopo il montaggio del file system remoto, le richieste di accesso a file vengono inviate al server attraverso la rete
- il server applica i normali **controlli di accesso** e, qualora producano un risultato positivo, restituisce un *file handle* al client, che lo usa per eseguire le successive operazioni su file

CIFS (Common Internet File System) equivalente Windows del protocollo NFS

Le system call locali vengono tradotte in chiamate (per gli stessi servizi) su file system remoti.

11.7 Malfunzionamenti

- Nei file system locali possono verificarsi malfunzionamenti dovuti a:
 - problemi hardware dei disks/devices
 - alterazioni dei **metadati** (informazioni per il reperimento dei file nelle dir)
 - problemi ai controller dei dischi
 - problemi alle connessioni hardware
 - errori umani
 - ...
- Nei remote file system i malfunzionamenti possono avvenire anche per cadute di rete o dei remote server sui quali sono presenti i file
- Per il ripristino da malfunzionamenti è necessario mantenere delle **informazioni di stato** sia sui client che sui server (per poter far riprendere entrambi)
- I protocolli attualmente più diffusi non mantengono informazioni di stato (es. NFS che trasferisce tutte le informazioni nella singola richiesta con alta tolleranza ai guasti ma scarsa sicurezza)

11.8 Semantica della coerenza

Semantica che specifica quando le modifiche apportate da un utente ai dati contenuti in un file possano essere osservate da altri utenti, è direttamente correlata agli algoritmi di sincronizzazione fra processi, ma risulta meno critica per via delle lunghe latenze e basse velocità di trasferimento dei dischi e della rete.

La semantica UNIX impone:

- le scritture in un file aperto da un utente siano immediatamente visibili agli altri utenti che hanno aperto lo stesso file
- la condivisione del puntatore alla posizione corrente del file (il file ha una singola immagine e tutti gli accessi si alternano, intercalandosi, a prescindere dalla loro origine)
- la contesa per l'immagine fisica del file determina il differimento dei processi utente

11.8.1 Protezione nel File System

- Per salvaguardare le informazioni si devono gestire le eventualità di:
 - danni fisici (**affidabilità**) assicurata dalla *ridondanza*
 - accessi impropri (**protezione**) ottenuta mediante *controllo degli accessi*
- Il proprietario del file deve poter controllare:
 - quali tipi di accesso sono leciti sul file
 - chi può accedere al file
- I vari tipi di accesso a file sono:
 - Read
 - Write
 - Esecuzione
 - Append
 - Cancellazione
 - Elencazione degli attributi
- Il controllo degli accessi è attuabile usando:
 - Liste di controllo accessi [al file si associano gli user e i tipi di accesso autorizzati per ogni user]
 - Liste di controllo accessi per classi di utenti [es. owner, group, others]
 - Associazione di una password a ciascun file

11.9 Caso UNIX

11.9.1 Review del caso di UNIX

File e dir possono appartenere a più dir, costruendo una struttura a *grafo generale*.

Varie dir ospitano programmi di sistema, per esempio:

- **/dev** per i device driver
- **/bin** per il codice eseguibile
- **/usr/include** per le librerie di sistemi
- **/etc** per i file di configurazione

A ciascun utente è associata una dir chiamata *home directory*, solitamente subdir di **/usr** o di **/home** e lì andranno i file creati dallo user.

11.9.2 UNIX

- Ciascun file è identificato da un **pathname assoluto** che include l'intero cammino dalla root al file stesso
- Ciascun pathname è unico (file e subdir → nomi distinti)
- Uno user che interagisce con il file system ha una propria **pwd** (present working directory) [non serve specificare il path fino alla pwd - **relative pathname**]

- Il simbolo "." fa riferimento alla pwd, mentre ".." fa riferimento alla dir contenente la pwd

11.9.3 Unix: Operazioni su file e directory

Comandi per la manipolazione di file e dir:

- **chmod** - stabilisce i parametri di protezione del file
- **chown** - cambia il proprietario di un file
- **cmp** - confronta byte a byte due file
- **cp** - copia un file in un altro (situato in un'altra dir o nella stessa ma con nome diverso)
- **find** - trova file per nome o per altre caratteristiche
- **grep** - ricerca un file attraverso una stringa contenuta in esso
- **ln** - crea un link ["ln oldfile newlink" crea un nuovo elemento della pwd, newlink, relativo ai contenuti del file oldfile]
- **more** - visualizza il contenuto di un file (su terminale per default)
- **mv** - muove un file (serve anche per rinominarlo)
- **rm** - cancella un file
- **mkdir** - crea una dir
- **cd** - cambia dir (contesto)
- **ls** - elenca i file e le subdir contenute nella dir specificata
- **pwd** - mostra il pathname della pwd
- **rmdir** - rimuove una dir
- **mount** - monta un file system
- **man** - permette di consultare il manuale

11.9.4 Unix: links

- Un **hardlink** è un'etichetta o un nome assegnato ad un file
- Si possono avere nomi distinti che identificano lo stesso file
- I comandi eseguiti relativamente ad uno qualsiasi dei nomi di un file operano sugli stessi "contenuti"
- Qualsiasi cambiamento effettuato su oldfile riguarda newlink (e viceversa)
- il comando rm può essere usato anche per rimuovere un hardlink (dopo la rimozione del link il contenuto del file viene mantenuto fino a che c'è almeno un nome che si riferisce a quel file)
- Un link simbolico o **softlink** è un tipo speciale di file che punta ad un altro file
- Diversamente agli hardlink non contiene i dati relativi al target file, semplicemente punta ad un diverso elemento di una dir
- In seguito alla cancellazione, i softlink diventano inutilizzabili (**dangling**, mentre gli hardlink preservano (finché ne esiste almeno uno) il contenuto del file)
- Per creare un link simbolico si usa: "ln -s filename mylink"

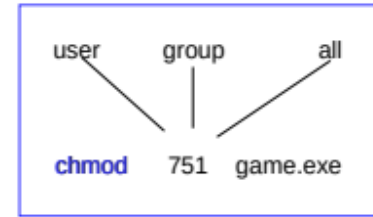
11.9.5 Protezione in UNIX

Alla creazione di un file, l'utente specifica la **protezione** da applicarci, indica quindi quali operazioni possono essere svolte e da chi. In UNIX:

- Tipo di accesso: read, write, exec, append, delete, attribute list;
- Metodo di accesso: read, write, exec;
- Tre classi User: user, group all;

esempio:

a) accesso del proprietario	RWX 7	⇒	1 1 1
b) accesso del gruppo	RWX 5	⇒	1 0 1
c) accesso pubblico	RWX 1	⇒	0 0 1



Il file può essere R/W/E dal proprietario, R/E da utenti del gruppo, E da tutti gli altri user di sistema.

Come per i file, a ciascuna dir sono associati tre campi (user, group, all) ciascuno composto da tre bit rwx. Un utente può elencare il contenuto di una dir solo se il bit r è inserito nel campo appropriato, modificarla se w è impostato e accedervi se x è settato.

12 Realizzazione del File System (FS)

12.1 Struttura del File System

il file (unità di memorizzazione logica di informazioni correlate), è arricchito da un insieme di attributi, anch'essi in memoria permanente.

Il F. System risiede in mem secondaria, tra i dispositivi di memorizzazione(Dischi), questi hanno due caratteristiche fondamentali:

- si possono riscrivere localmente;
- Permettono l'accesso diretto (o sequenziale) a qualsiasi loro sezione, quindi a qualsiasi file;

Le operazioni di IO su disco avvengono in modo granulare in blocco: ogni blocco è composto da più settori usualmente di dimensione 4KB.

File System per l'utente: progettato per definire la visione utente del FS, significa definire gli attributi, struttura delle directory e le varie operazioni.

File System per l'OS: comporta l'implementazione degli algoritmi e strutture dati che mettono in corrispondenza il FS logico con i dispositivi di memorizzazione dati.

Il FS é strutturato in livelli (stratificato):

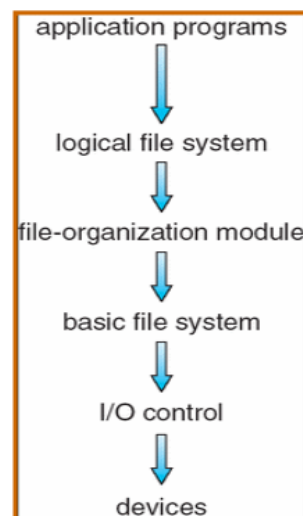


Figure 31: I livelli "bassi" possono essere comuni a vari FS

12.1.1 I/O control

Composto da:

- **Driver del dispositivo:** Traduce comandi di altro livello a sequenze di basso lvl, che controllano HW di I/O nel compiere un' operazione specifica in una locazione data.
- **Gestore delle interrupt;**

12.1.2 Basic File System

Invia comandi al driver del dispositivo per R/W **blocchi fisici** su disco, ogni blocco è identificato da un numero di unità, di cilindro, di superficie e di settore.

Gestisce anche i **buffer** dei dispositivi e le cache per i metadati del FS.

12.1.3 File-Organization module

Traduce gli indirizzi logici in fisici in base al **tipo di allocazione** dei file impiegato e contiene il modulo per la gestione dello **spazio libero**.

12.1.4 Logical File System

Gestisce i **metadati**, tutte le strutture del FS eccetto i dati veri e propri memorizzati nei files e le strutture delle directory.

Mantiene i **file control block** che contengono le info su file e fa controlli di protezione e sicurezza.

12.1.5 File Control Block

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

12.1.6 Tipi di File System

Esistono tipi diversi di FS, ed è comune che i vari OS ne supportino più di uno.

Ex:

- UNIX: **UFS** basato su File System Berkeley Fast (FFS);
- Windows: **FAT, FAT32, FAT64, NTFS**;
- Linux: **ext3, ext4**(extended file system);

Un'altro esempio è il GoogleFS, progettato per soddisfare specifiche esigenze di memorizzazione e recupero dati:

- GFS conserva i dati raccolti da Google e permette efficiente accesso in R_only da parte di numerosi client;
- Dati immagazzinati in maniera permanente in file di grosse dimensioni circa 100 GB;
- Rare operazioni di cancellazione, sovrascrittura e compressione;

12.1.7 Strutture dati del File System

Di seguito le **strutture dati** del FS residenti su **disco**

- **Blocco di controllo di avviamento:** contiene info per il boot dell'OS (*boot block* per UFS, *partition boot sector* per Windows);
- **Blocco di controllo del volume:** contiene dati sulla partizione come il tumero totale dei blocchi e la loro dimensione, un counter dei blocchi liberi e relativi puntatori, etc ... (*superblock* per UFS, *master file table* per NTFS);

- **Struttura della directory:** usata per organizzare i file (in UFS comprende i nomi dei files ed i numeri di **incode** associati);
- **Blocchi di controllo dei file:** FCB (inode in UFS)

Di seguito invece le **Strutture dati** del FS residenti in **memoria**

- **Tabella di montaggio:** contiene le info relative a ciascun valore montato;
- **Cache della struttura delle directory:** contiene info relative a tutte le dir i cui processi hanno avuto accesso di recente;
- **Tabella file aperti:** contiene una copia dell'FCB per ciascun file aperto nel sistema;
- **Tabella dei file aperti per processo:** contiene un puntatore all'elemento corrispondente nella tabella generale, più info di accesso specifiche del processo;
- **Buffer:** usati durante R/W dei blocchi da/a disco;

12.2 Realizzazione Directory

- **Lista lineare** di nomi di files con puntatori a blocchi dei dati: è semplice da implementare (tramite array o liste concatenate), ma inefficiente dal punto di vista del tempo di ricerca. Una soluzione può essere la **Lista ordinata**, che migliora il tempo di ricerca ma l'ordinamento deve essere mantenuto a fronte di ogni modifica.
- **Tabella Hash** combina una lista lineare con una struttura hash, grazie all'hashing ottiene la entry nella lista relativa a un nome di file, migliora il tempo di ricerca nella dir e inserimento/cancellazione costano $O(1)$ se non si verificano collisioni [situazione in cui due nomi di file generano lo stesso indirizzo hash in tabella].

12.3 Metodi di allocazione dei files

La natura ad accesso diretto dei dischi garantisce la flessibilità di implementare i file, l'obiettivo è allocare lo spazio disco ai file in modo da avere minimo spreco di memoria e massima velocità di accesso.

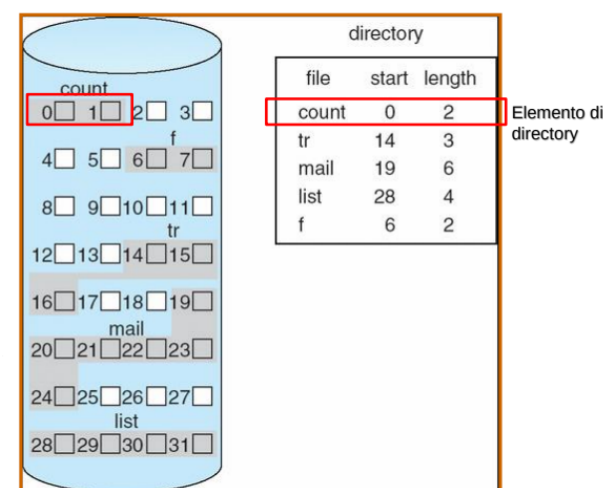
Il **metodo di allocazione** dello spazio su disco descrive come i blocchi fisici del disco vengono allocati ai file:

Allocazione: **contigua, concatenata, indicizzata**

12.3.1 Allocazione contigua

Con questa modalità il file occupa blocchi contigui su disco, per accedervi occorre conoscere solo la locazione iniziale (# blocco iniziale) e la lunghezza del file (numero blocchi totali).

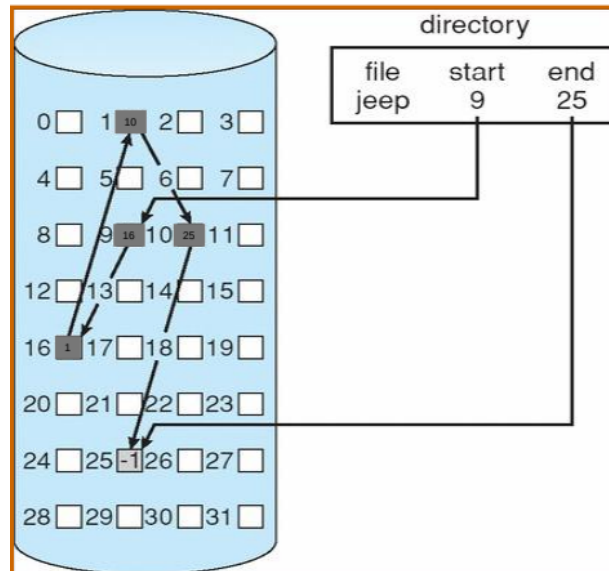
Vi è accesso casuale e diretto, la frammentazione è esterna, ha lo stesso problema dell'allocazione di memoria centrale con l'utilizzo delle stesse tecniche: First fit, best fit, compattazione, ...



12.3.2 Allocazione concatenata

Questo metodo memorizza ogni file tramite una lista doncatenata di blocchi su disco, che possono essere allocati ovunque, il cui accesso richiede l'indirizzo del blocco iniziale.

Non c'è spreco di spazio, ma non è possibile l'accesso casuale bisogna sempre percorrere tutta la lista.

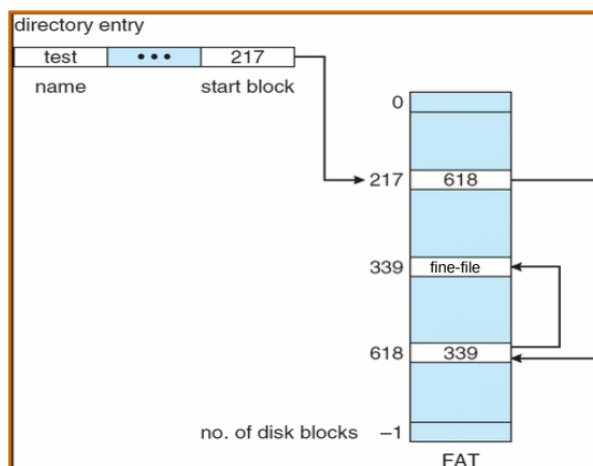


12.3.3 File-Allocation Table (FAT)

Si tratta di una variante del metodo di allocazione concatenata in MS-DOS e OS/2, per contenerla si riserva una sezione del disco, ha un elemento per ogni blocco del disco ed è indicizzata dal numero di blocco. L'elemento Directory contiene il numero del primo blocco del file.

A sua volta l'elemento della FAT indicizzato da quel blocco contiene il numero del blocco successivo nel file, e l'ultimo blocco ha come elemento un valore speciale di fine file.

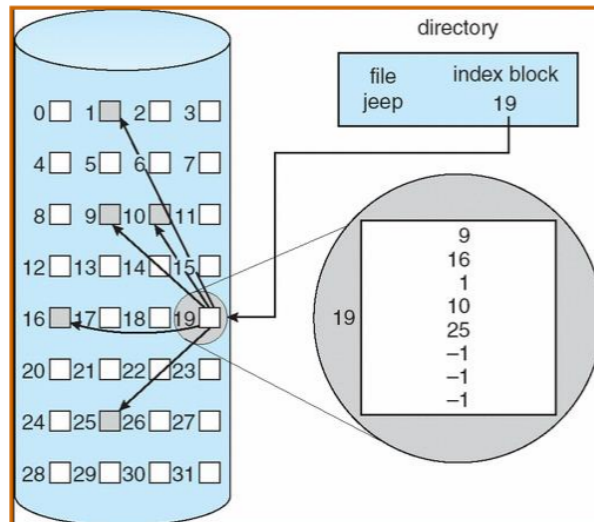
Ogni blocco inutilizzato ha valore 0, questo metodo facilita l'accesso casuale.



12.3.4 Allocazione indicizzata

Memorizza tutti i puntatori a blocchi in un **blocco indice**, anche qui è possibile l'accesso casuale, l'iesimo elemento dell'indice individua l'iesimo blocco del file.

Permette accesso dinamico senza frammentazione esterna e prevede **Overhead** anche per i file piccoli.



Prevede uno **schema concatenato**: ogni file ha almeno un blocco indice, all'aumentare della dimensione del file si collegano più blocchi indice, non avendo così un limite alla dimensione. Ogni blocco indice contiene l'insieme degli indirizzi ai primi N blocchi del file, in aggiunta ad un puntatore all'indice successivo. Possiede un **Indice a più livelli** (come le page table) e in UNIX utilizza uno *schema combinato*.

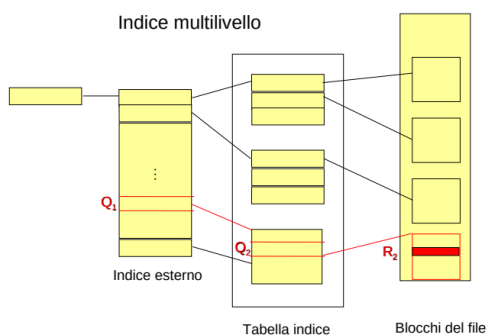


Figure 32: Indice Multilivello

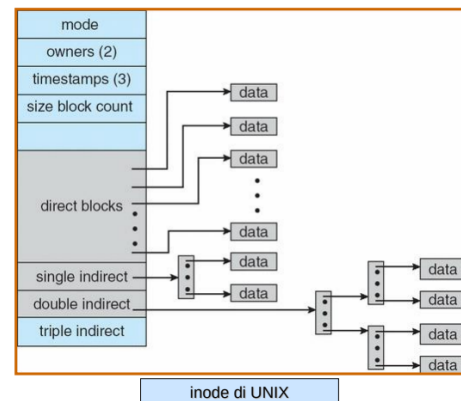


Figure 33: Schema combinato UNIX (4KB per blocco)

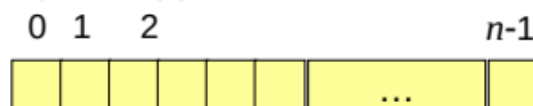
12.4 Gestione dello spazio libero

Per poter tener traccia dello spazio libero su un disco il sistema conserva una **lista dello spazio libero**.

Per caricare un file occorre cercare nella lista lo spazio necessario ed allocarlo al nuovo file, e aggiornare la lista. Qualora si cancellasse un file bisogna aggiungere alla lista i blocchi disco precedentemente assegnati al file eliminato.

Questa lista può non essere realizzata come una lista.

Vettore Bitmap se ci sono n blocchi:



$\text{bit}[i] = 1 \Rightarrow$ l'i-esimo blocco è libero

$\text{bit}[i] = 0 \Rightarrow$ l'i-esimo blocco è occupato

Viene eseguita la ricerca del blocco libero scorrendo il vettore alla ricerca del primo byte $\neq 0$. Questa tecnica possiede buone prestazioni se il vettore è in mem centrale, ed è vantaggiosa in caso di allocazione contigua dei files.

Lista concatenata , collega tutti i blocchi liberi mediante un puntatore e si mantiene il puntatore alla testa della lista in mem centrale, ogni blocco contiene il puntatore al successivo (basso spreco di spazio). L'organizzazione in lista concatenata non é generalmente efficiente, per scorrere tutta la lista occorre leggere ogni blocco e questo porta a tempi alti di I/O. Tuttavia solitamente l'OS ha bisogno di un blocco solo libero da allocare ad un file quindi si usa il primo blocco della lista.

Nella FAT, il conteggio dei blocchi liberi è incluso nella struttura dati per l'allocazione e non richiede quindi un metodo di gestione separato.

Grouping: variante della lista concatenata, sul primo blocco sono memorizzati gli indirizzi di n blocchi liberi, di questi i primi n-1 sono effettivamente liberi invece l'nesimo contiene indirizzi di altri n blocchi, e via così...

Conteggio: si usa una lista di gruppi di blocchi contigui, mantenendo una lista con un indirizzo di un blocco del disco e un counter che indica il numero di blocchi liberi (contigui) da cui è seguito.

12.4.1 Ripristino

Poichè file e dir sono mantenuti sia in RAM (parzialmente) che su disco, è necessario assicurarsi che malfunzionamenti del sistema non comportino la perdita o incoerenza dei dati.

Esempio: Quando si crea un file

- si modifica l'elemento Dir;
- si allocano blocchi per dati e FCB;
- si aggiornano le info (puntatori) su blocchi liberi;

Se si avesse una caduta del sistema mentre sono in svolgimento questi passi, potrebbero verificarsi incoerenze fra le strutture, come: il counter degli FCB liberi potrebbe indicare un FCB allocato, mentre la dir non contiene l'elemento relativo.

Verificatore di coerenza: **fsck** (file system check) in Unix, **chkdsk** (check disk) in Windows, confronta i dati nella struttura della dir con i blocchi di dati su disco e tenta di correggere eventuali incoerenze.

Copie di Riserva e Recupero: Si impiegano programmi di sistema per fare un backup, in caso di necessità si recuperano i file persi caricando suddetto backup.

12.4.2 File System con Log

I FS con **registrazione delle modifiche** salvano ogni modifica alle strutture dati del FS come una *transazione*. Le transazioni vengono registrate in un file di **log**. Una Transazione si dice **committed** quando è stata registrata sul log, anche se è committed la modifica del FS potrebbe non essere stata terminata (il Log mantiene tutte le info necessarie per completarla).

Le transazioni registrate nel Log sono attuate sul FS in modo asincrono.

12.5 Network File System (NFS)

Rappresenta sia una realizzazione che una definizione di un sistema per accesso a file remoti tramite LAN o WLAN, nata in ambiente UNIX(supportata da Linux) ed usa protocolli TCP/IP o UDP/IP (Unreliable Datagram Protocol su Ethernet) a seconda della rete di comunicazione.

Nel contesto del NFS si considera un insieme di stazioni di lavoro eterogenee interconnesse come un insieme di calcolatori indipendenti con FS indipendenti. Basata su approccio client-server garantisce condivisione tra FS su richiesta esplicita in modo trasparente.

Una dir remota viene montata sul FS locale e assume l'aspetto di un sottoalbero del FS locale. Si specifica come argomento dell'operazione di montaggio in modo esplicito: occorre fornire la locazione della dir remota.

Sia il mount che le operazioni su file sono implementate attraverso *RPC* i file nella dir remota sono accessibili in modo completamente trasparente nel FS la traduzione delle modifiche viene fatta dall'OS.

In linea di principio ogni FS può essere montato su una qualsiasi dir locale.

12.5.1 Vantaggi NFS

1. In locale si usa meno spazio disco perchè i dati possono essere conservati su una singola macchina e acceduti da tutte le altre collegate alla rete;
2. Gli utenti non devono avere home dir separate su ogni macchina in rete ma possono essere poste sul server NFS e rese disponibili tramite la rete;

3. I dispositivi di archiviazione come CD-ROM e USB possono essere utilizzati dagli altri computer collegati, riducendo così il numero di unità per supporti rimovibili presenti in rete;